



Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Le basi della programmazione grafica: il framework JavaFX

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



PROGRAMMAZIONE GRAFICA

- Sviluppare una *applicazione grafica* è molto diverso da sviluppare un'applicazione tradizionale
 - niente più Read-Eval-Print Loop (REPL)
- L'interazione non avviene più in momenti prefissati, ma *quando l'utente agisce sui controlli grafici*
 - non è il programma che ha il controllo (printf, scanf), ma l'utente
- Si programma *stabilendo come reagire agli eventi* che si verificano quando l'utente agisce sui controlli grafici
 - ciò che accade in un programma grafico è descrivibile *solo a posteriori*, come *interaction history*: non è predicibile
 - .. ed è molto difficile progettare ed effettuare testing e collaudi



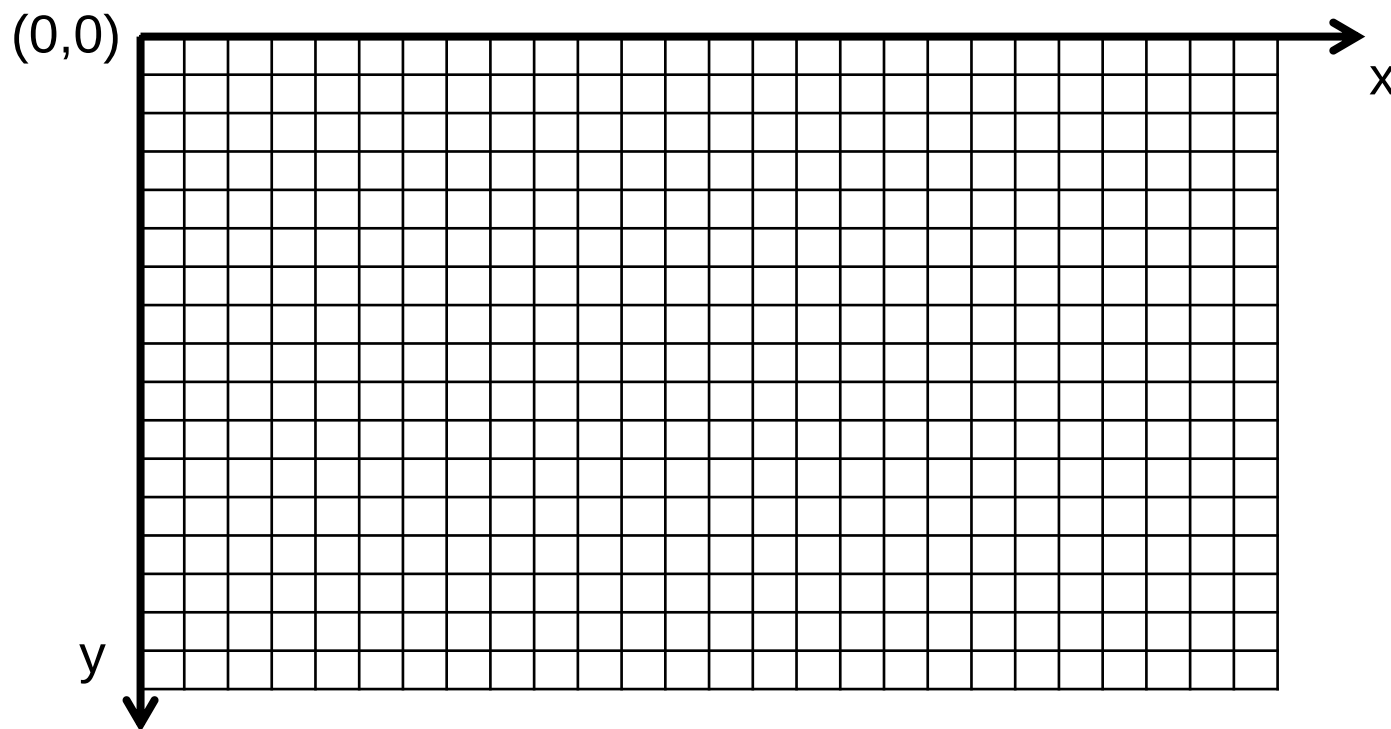
PROGRAMMAZIONE GRAFICA

In una finestra grafica si può operare a due livelli:

- a livello di **singoli pixel**
 - accendendo/spegnendo singoli pixel colorati
 - non c'è consapevolezza di cosa si stia disegnando: la finestra rimane solo una "tela con puntini colorati sopra"
- a livello di **controlli grafici evoluti**
 - collocando in essa *controlli già pronti* secondo le esigenze (bottoni, slider, liste, campi ed aree di testo.. e molto altro)
 - c'è piena consapevolezza dei componenti collocati nella finestra: la finestra in sé funge solo da supporto fisico
 - per farlo, occorre *aggiungere i componenti a un opportuno contenitore* che si relazioni col *modello della finestra*
 - il flusso di controllo è gestito dal framework

LIVELLO PIXEL

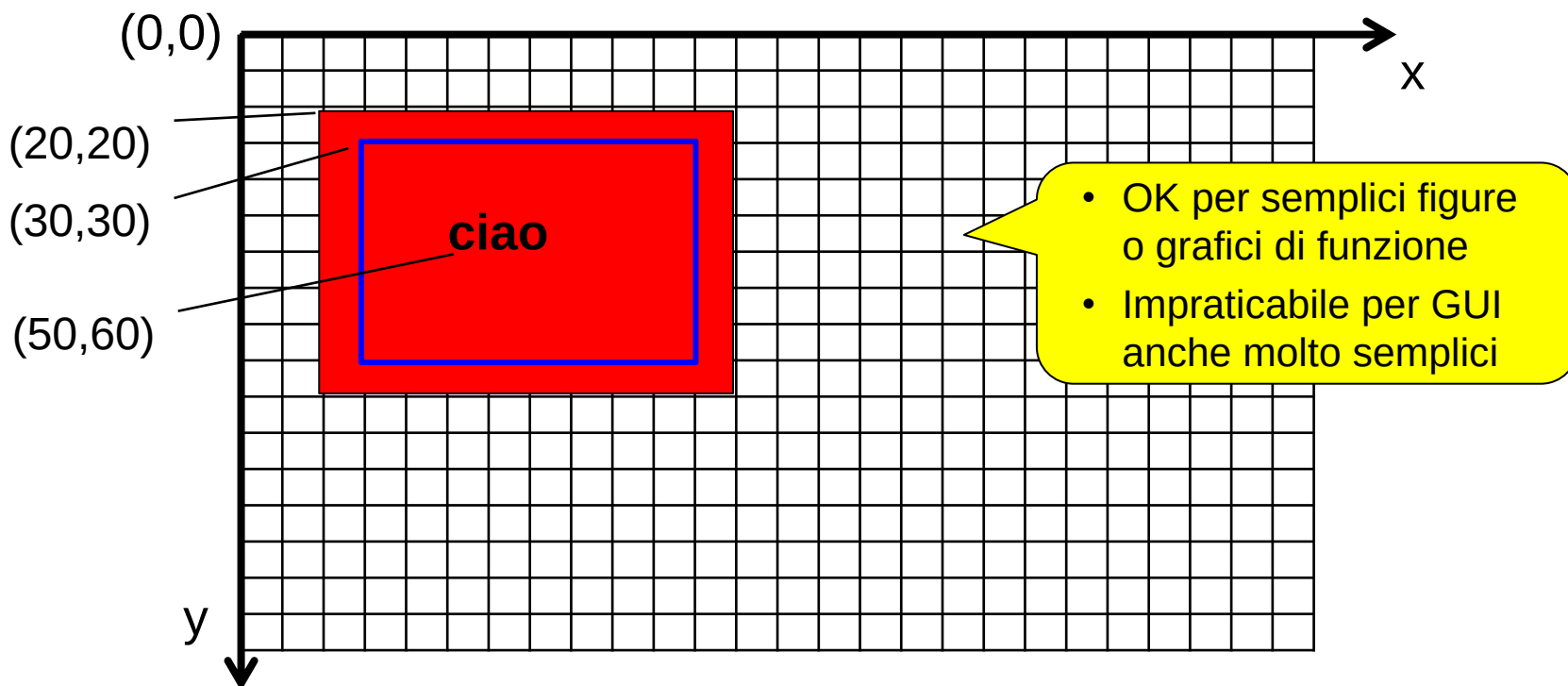
- La finestra è vista come un **piano cartesiano** con *origine in alto a sinistra* e *asse verticale orientato verso il basso*
- Si possono **accendere/spegnere/colorare singoli pixel**, formando così figure e parole



LIVELLO PIXEL – ESEMPI

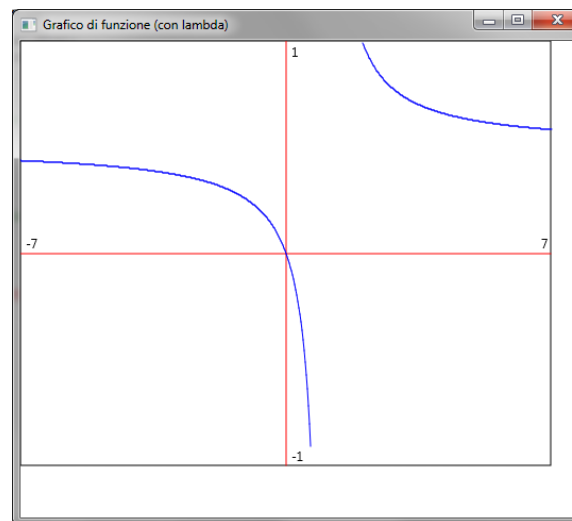
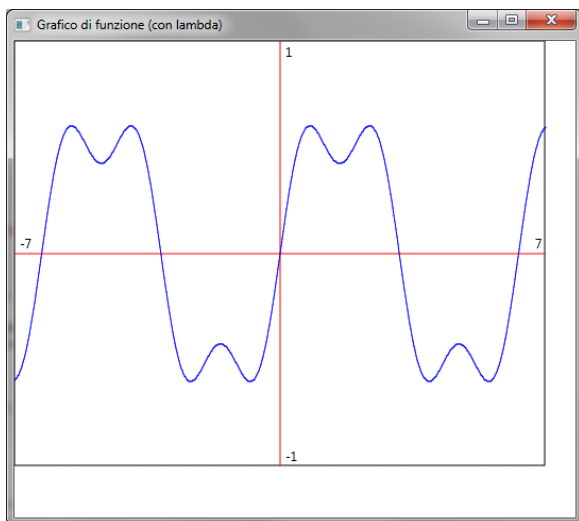
L'esempio [griglia ogni 10 pixel per comodità] **mostra:**

- un rettangolo rosso pieno, origine a (20,20), dimensioni 100x80
- un rettangolo blu interno, origine a (30,30), dimensioni 80x60
- una scritta "ciao" nera, con origine [angolo basso sinistro] a (50,60)



ESEMPIO: GRAFICO DI FUNZIONE

- Linee orizzontali rosse per assi, scritte nere per estremi x,y
- Pixel blu calcolati uno ad uno secondo la funzione desiderata
- Punti critici:
 - asse y cresce verso il basso → trasformazione di coordinate
 - i valori $x, f(x)$ sono reali, le coordinate dei pixel sono intere
 - ..e se la funzione non esiste in un punto o è discontinua..?

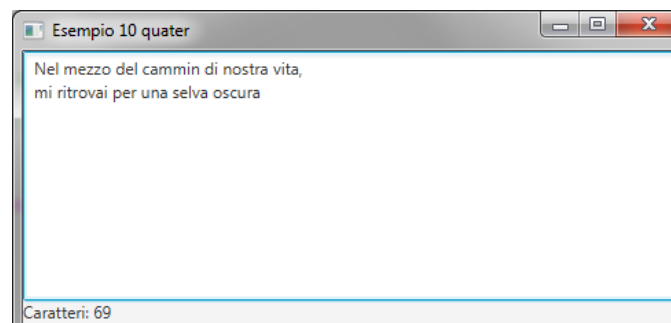
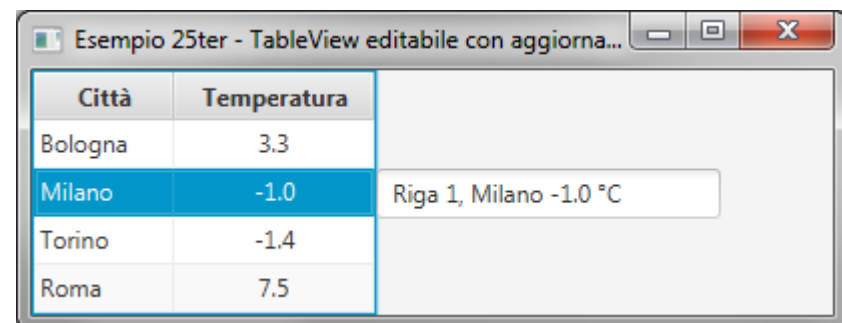
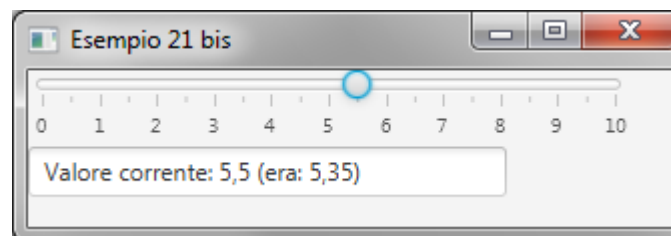
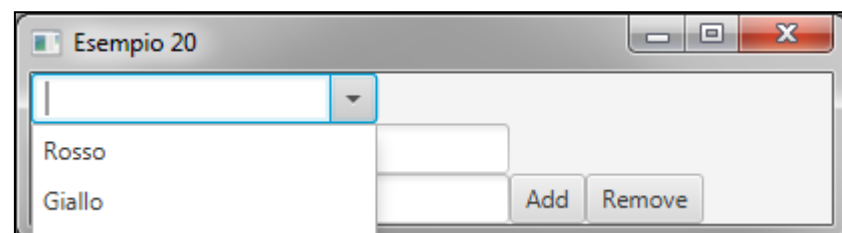
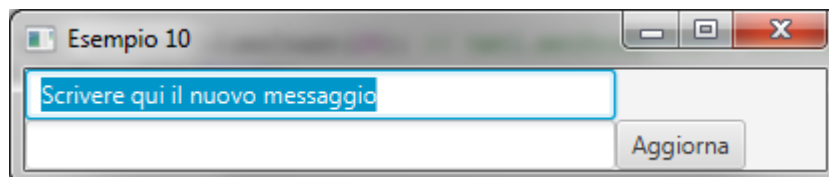
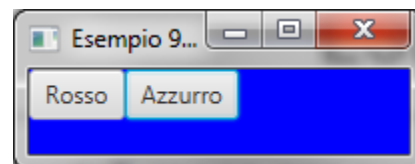
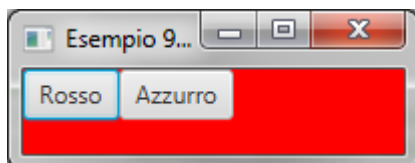




LIVELLO COMPONENTI GRAFICI

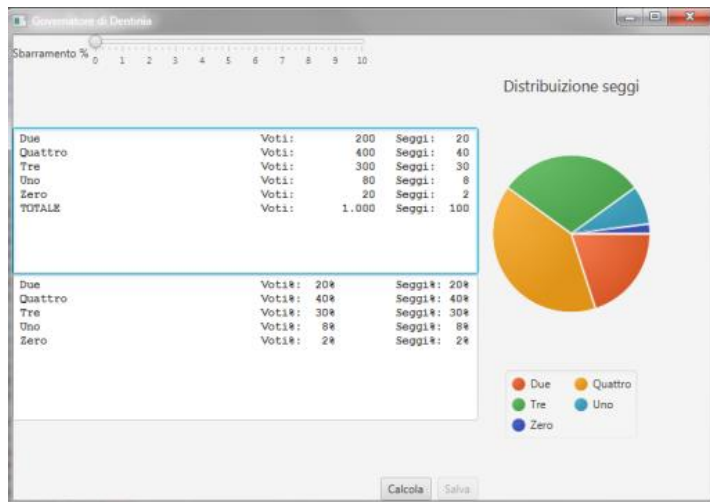
- La finestra è vista come **contenitore** su cui **appendere componenti (controlli o widget)**, come «*post it*»
 - di norma, *non* si stabilisce dove debbano stare tramite *coordinate assolute*, perché nei vari device fisici le finestre hanno dimensioni diverse, si ridimensionano, possono ruotare, etc.
 - si dota piuttosto ogni finestra di un **gestore di layout**, che colloca i widget secondo *politiche prestabilite* → *adattamento automatico*
- Ogni **framework grafico** fornisce *decine di widget già pronti*, con comportamento base già predisposto
- Il progettista della GUI deve definire:
 - la **parte strutturale** (quali e quanti widget, disposti come)
 - la **parte comportamentale** (come si reagisce agli *eventi*)

ESEMPI BASE

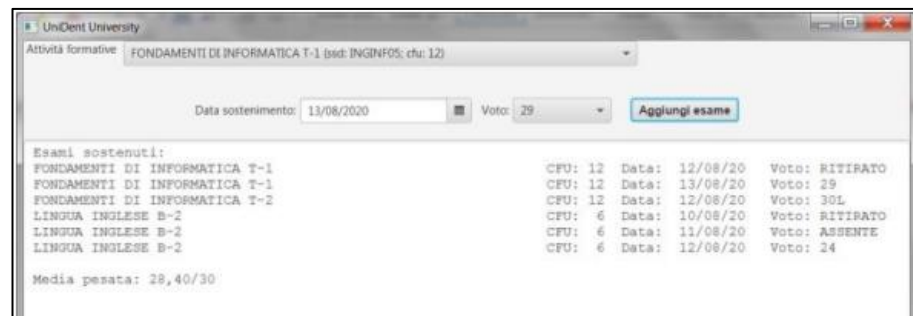


ESEMPI DA COMPITI D'ESAME (2020)

Elezioni



UniDent



Attività formative	Data sostenimento	Voto	Aggiungi esame
FONDAMENTI DI INFORMATICA T-1	13/08/2020	29	
Esami sostenuti:			
FONDAMENTI DI INFORMATICA T-1	CFU: 12	Data: 12/08/20	Voto: RITIRATO
FONDAMENTI DI INFORMATICA T-1	CFU: 12	Data: 13/08/20	Voto: 29
FONDAMENTI DI INFORMATICA T-2	CFU: 12	Data: 12/08/20	Voto: 30L
LINGUA INGLESE B-2	CFU: 6	Data: 10/08/20	Voto: RITIRATO
LINGUA INGLESE B-2	CFU: 6	Data: 11/08/20	Voto: ASSENTE
LINGUA INGLESE B-2	CFU: 6	Data: 12/08/20	Voto: 24
Media pesata: 28,40/30			

Crittocruciverba

CrittoCruciverba

1	2	3	4	5	3	6	6	7	#	8
9	10	#	6	3	8	#	11	12	2	4
10	4	12	#	13	#	9	14	#	7	#
#	12	10	15	#	#	4	10	10	2	#
16	10	1	1	10	9	3	12	16	7	#
15	14	17	4	3	13	15	2	15	5	3
12	#	10	10	4	#	10	7	#	15	7
16	15	4	3	7	#	5	18	#	5	3
10	#	10	5	2	3	4	7	#	#	7
14	6	#	3	15	4	3	#	14	3	#
9	10	17	#	17	7	12	12	3	13	15
3	18	9	4	10	1	15	1	9	7	5

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549	550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699	700	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735	736	737	738	739	740	741	742	743	744	745	746	747	748	749	750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799	800	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849	850	851	852	853	854	855	856	857	858	859	860	861	862	863	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895	896	897	898	899	900	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949	950	951	952	953	954	955	956	957	958	959	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012	1013	1014	1015	1016	1017	1018	1019	1020	1021	1022	1023	1024	1025	1026	1027	1028	1029	1030	1031	1032	1033	1034	1035	1036	1037	1038	1039	1040	1041	1042	1043	1044	1045	1046	1047	1048	1049	1050	1051	1052	1053	1054	1055	1056	1057	1058	1059	1060	1061	1062	1063	1064	1065	1066	1067	1068	1069	1070	1071	1072	1073	1074	1075	1076	1077	1078	1079	1080	1081	1082	1083	1084	1085	1086	1087	1088	1089	1090	1091	1092	1093	1094	1095	1096	1097	1098	1099	1100	1101	1102	1103	1104	1105	1106	1107	1108	1109	1110	1111	1112	1113	1114	1115	1116	1117	1118	1119	1120	1121	1122	1123	1124	1125	1126	1127	1128	1129	1130	1131	1132	1133	1134	1135	1136	1137	1138	1139	1140	1141	1142	1143	1144	1145	1146	1147	1148	1149	1150	1151	1152	1153	1154	1155	1156	1157	1158	1159	1160	1161	1162	1163	1164	1165	1166	1167	1168	1169	1170	1171	1172	1173	1174	1175	1176	1177	1178	1179	1180	1181	1182	1183	1184	1185	1186	1187	1188	1189	1190	1191	1192	1193	1194	1195	1196	1197	1198	1199	1200	1201	1202	1203	1204	1205	1206	1207	1208	1209	1210	1211	1212	1213	1214	1215	1216	1217	1218	1219	1220	1221	1222	1223	1224	1225	1226	1227	1228	1229	1230	1231	1232	1233	1234	1235	1236	1237	1238	1239	1240	1241	1242	1243	1244	1245	1246	1247	1248	1249	1250	1251	1252	1253	1254	1255	1256	1257	1258	1259	1260	1261	1262	1263	1264	1265	1266	1267	1268	1269	1270	1271	1272	1273	1274	1275	1276	1277	1278	1279	1280	1281	1282	1283	1284	1285	1286	1287	1288	1289	1290	1291	1292	1293	1294	1295	1296	1297	1298	1299	1300	1301	1302	1303	1304	1305	1306	1307	1308	1309	1310	1311	1312	1313	1314	1315	1316	1317	1318	1319	1320	1321	1322	1323	1324	1325	1326	1327	1328	1329	1330	1331	1332	1333	1334	1335	1336	1337	1338	1339	1340	1341	1342	1343	1344	1345	1346	1347	1348	1349	1350	1351	1352	1353	1354	1355	1356	1357	1358	1359	1360	1361	1362	1363	1364	1365	1366	1367	1368	1369	1370	1371	1372	1373	1374	1375	1376	1377	1378	1379	1380	1381	1382	1383	1384	1385	1386	1387	1388	1389	1390	1391	1392	1393	1394	1395	1396	1397	1398	1399	1400	140
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	-----

ESEMPI DA COMPITI D'ESAME (2021)

Ghigliottina

Gioco della Ghigliottina

La parola segreta: La tua risposta: **SVELA**

Ghigliottina

☐ Prima parola ☒ Seconda parola

VECCHIA

TIMBRO

PARTENZA

FISSARE

CAMBIAMENTO

25.000 €

MasterMind

Master Mind

GRIGIO VERDE NERO GRIGIO

Inserisci

1) ROSSO, GIALLO, VERDE, BLU
2) MARRONE, GRIGIO, BLU, VERDE
3) GIALLO, ROSSO, GRIGIO, MARRONE
4) VERDE, GIALLO, ROSSO, ROSSO
5) VERDE, MARRONE, ROSSO, VERDE
6) VERDE, MARRONE, MARRONE, ROSSO
7) GRIGIO, VERDE, ROSSO, BLU
8) BLU, VERDE, ROSSO, BLU
9) GRIGIO, VERDE, ROSSO, GRIGIO

BIANCO, BIANCO, VUOTO, VUOTO
BIANCO, BIANCO, VUOTO, VUOTO
BIANCO, BIANCO, VUOTO, VUOTO
NERO, BIANCO, VUOTO, VUOTO
NERO, BIANCO, VUOTO, VUOTO
BIANCO, BIANCO, VUOTO, VUOTO
NERO, NERO, VUOTO, VUOTO
NERO, NERO, NERO, VUOTO
NERO, NERO, NERO, NERO

Stampa report

Restart

Svela

Gioco terminato

Max tentativi: 10 Tentativi rimasti: 1 Stato del gioco: VITTORIA

Gas For Life

Gas For Life

Importo totale: 1000 Consumo totale: 800

Costo fisso: 60 Costo variabile: 940

Costo standard al m3: 1 Costo extra al m3: 1.5

Seleziona un giorno nel mese di riferimento: 21/04/2021

Calcola

Bolletta mensile aprile

Costo fisso: 60.0

Costo variabile: 940.0

Consumo complessivo: 800.0 Costo normale consumo al m3: 1.0 Costo extra consumo al m3: 1.5

Quota dell' appartamento: 1-1A di Sauron, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 1-1B di Boromir, consumo= 82,32, valore della quota= 44,99

Quota dell' appartamento: 1-2A di Saruman, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 1-2B di Gandalf, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 1-3A di Frodo, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 1-3B di Aragorn, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 1-4A di Gimli, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 1-4B di Legolas, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 1-1A di Galadriel, consumo= 82,32, valore della quota= 44,99

Quota dell' appartamento: 1-1B di Radagast, consumo= 82,32, valore della quota= 44,99

Quota dell' appartamento: 2-2A di Bilbo, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 2-2B di Balin, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 3-3A di Elrond, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 3-3B di Meriadoc, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 3-4A di Peregrino, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 3-4B di Bombail, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 5-1A di Eomer, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 5-1B di Theoden, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 5-2A di Eowyn, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 5-2B di Faramir, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 5-3A di Samwise, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 5-3B di Denethor, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 5-4A di Gollum, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 5-4B di Arwen, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 7-1A di Smaug, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 7-1B di Barbalbero, consumo= 72,97, valore della quota= 35,63

Quota dell' appartamento: 7-2A di Haldir, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 7-2B di Fana, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 7-3A di Thorin, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 7-3B di Beorn, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 7-4A di Grima, consumo= 63,61, valore della quota= 26,28

Quota dell' appartamento: 7-4B di Shelob, consumo= 72,97, valore della quota= 35,63

La differenza tra il consumo presunto nella bolletta e quello reale risulta essere di m3 -1.394,57

Battaglia Navale

Battaglia Navale

Scegli un elemento della sonda e premi il pulsante nella griglia

MAI

Dimensioni da inserire:

1) SOTTOMARINO (1 elemento) 2) CACCIA TORPEDIERO (2 elementi) 3) INCROCIATORE (3 elementi) 4) PORTAEREMI (4 elementi)

VERIFICA

ESEMPI DA COMPITI D'ESAME (2019)

Favoliere

Favoliere

Fascia d'età del bambino:

PICCOLISSIMO

Grado di impressionabilità:

abbastanza impressionabile

Favola generata:

C'era una volta la principessa Catlyn, una principessa bionda ma molto sola e Arfelius, un drago enorme e feroce; nel bosco si incontrarono; scalò la torre con una lunga corda; quindi si sposarono e vissero felici per molti anni

Genera favola

Stampa su file

Campo minato

Mines Sweeper

Mines: 3

?	1	0	0	0
1	1	0	0	0
0	0	1	1	1
1	1	1	?	1
?	1	1	1	1

Current game status:

?	1	0	0	0
1	1	0	0	0
0	0	1	1	1
1	1	1	?	1
?	1	1	1	1

YOU WIN!

M	1	0	0	0
1	1	0	0	0
0	0	1	1	1
1	1	1	M	1
M	1	1	1	1

Save status Restart

Percorsi bus urbani

Bussy - Viaggia con trasporto!

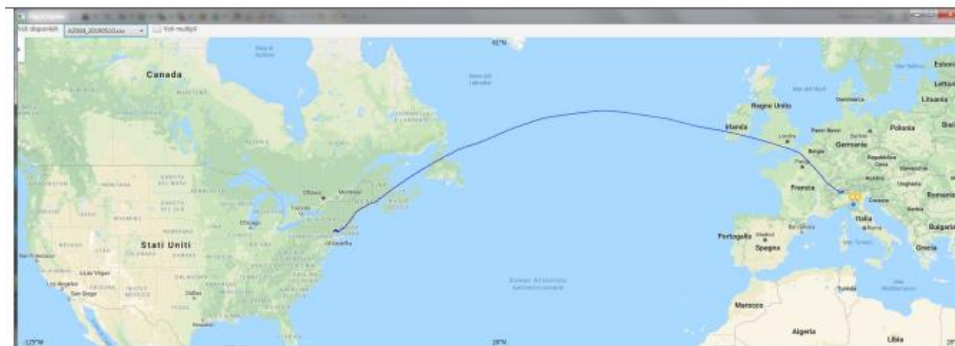
Da Stazione Centrale A Porta Saragozza - Villa Cassarini

Percorso da Stazione Centrale a Porta Saragozza - Villa Cassarini, linea 33, durata 12 min

Percorso da Stazione Centrale a Porta Saragozza - Villa Cassarini, linea 32, durata 26 min

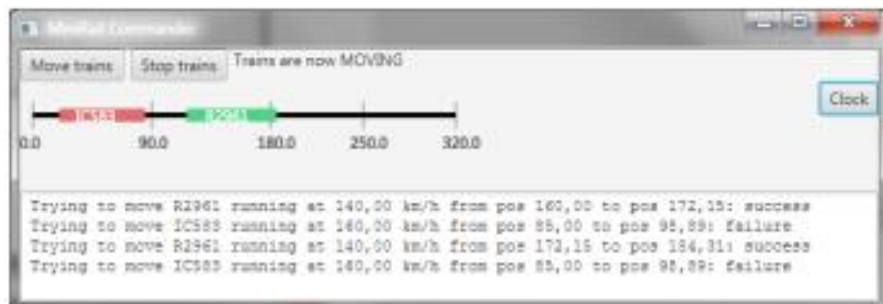
Cerca percorso

Tracciamento voli

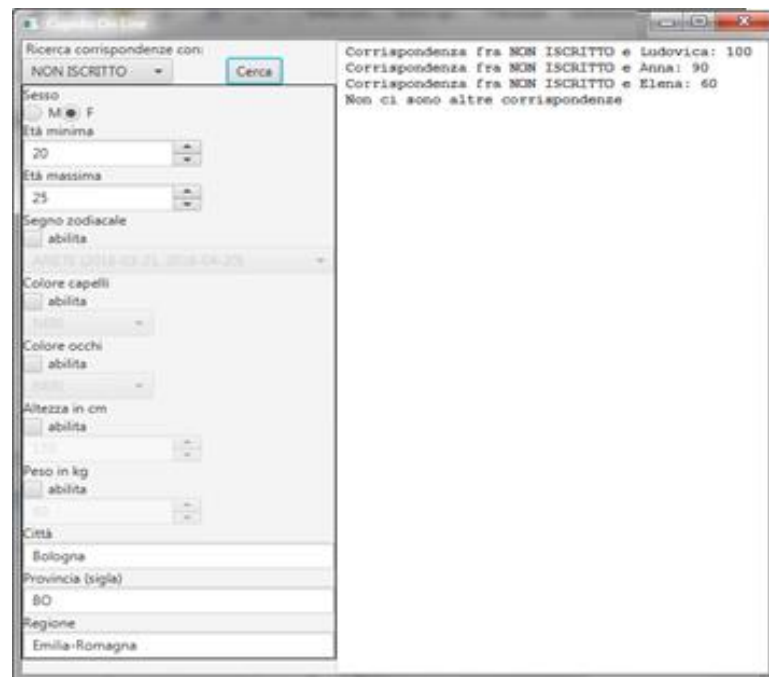


ESEMPI DA COMPITI D'ESAME (2018)

Movimento treni (plastico ferroviario) – (2019)



Anima gemella



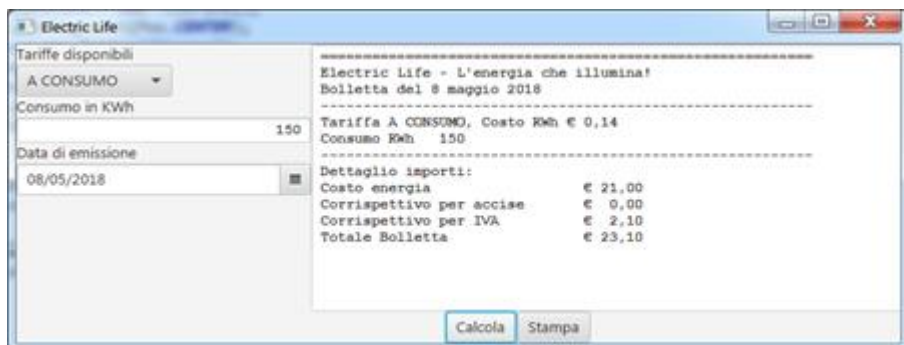
The screenshot shows a window titled "Cerca l'anima gemella" with a search bar and a "Cerca" button. Below the search bar, there are several input fields for personal information:

- Ricerca corrispondenze con: NON ISCRITTO
- Sesso: ☒ M ☐ F
- Età minima: 20
- Età massima: 25
- Segno zodiacale: abilita
- Colore capelli: abilita
- Colore occhi: abilita
- Altezza in cm: abilita
- Peso in kg: abilita
- Città: Bologna
- Provincia (sigla): BO
- Regione: Emilia-Romagna

On the right side, there is a list of search results:

```
Corrispondenza fra NON ISCRITTO e Ludovica: 100
Corrispondenza fra NON ISCRITTO e Anna: 90
Corrispondenza fra NON ISCRITTO e Elena: 60
Non ci sono altre corrispondenze
```

Bolletta elettrica



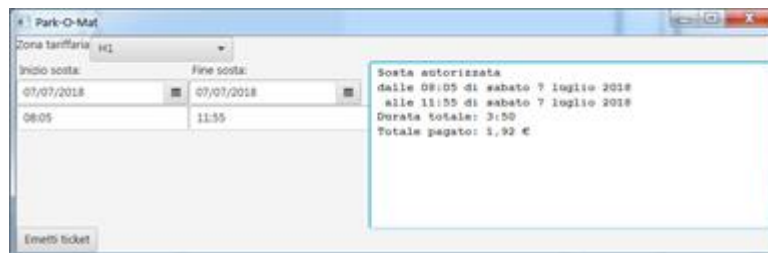
The screenshot shows a window titled "Electric Life" with a "Tariffe disponibili" dropdown menu set to "A CONSUMO". Below the dropdown, there is a "Consumo in KWh" field with the value 150. To the right, there is a section for "Data di emissione" with the date 08/05/2018. Below this, there is a table showing the breakdown of the electricity bill:

Electric Life - L'energia che illumina!	
Bolletta del 8 maggio 2018	
Tariffa A CONSUMO, Costo kWh € 0,14	
Consumo kWh	150
Dettaglio importi:	
Costo energia	€ 21,00
Corrispettivo per accise	€ 0,00
Corrispettivo per IVA	€ 2,10
Totale Bolletta	€ 23,10

At the bottom of the window, there are buttons for "Calcola" and "Stampa".

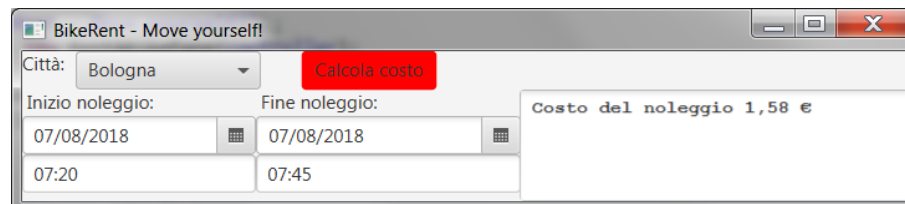
ESEMPI DA COMPITI D'ESAME (2018)

Park-O-Mat



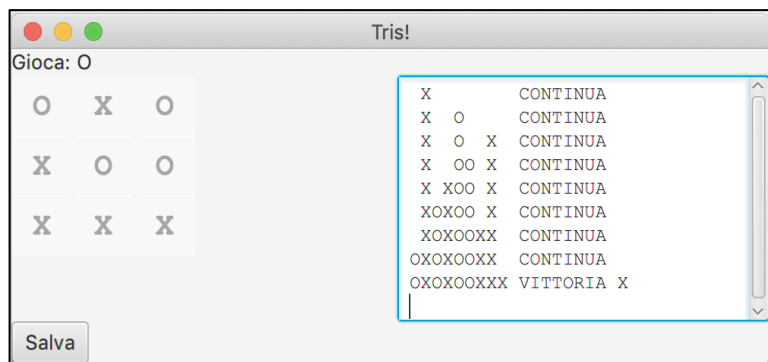
The screenshot shows a web form for Park-O-Mat. It includes a dropdown for 'Zona tariffaria' (set to 'h1'), input fields for 'Inizio sosta' (07/07/2018, 08:05) and 'Fine sosta' (07/07/2018, 11:55). A text box on the right displays: 'Sosta autorizzata dalle 08:05 di sabato 7 luglio 2018 alle 11:55 di sabato 7 luglio 2018. Durata totale: 3:50. Totale pagato: 1,92 €'. An 'Emetti ticket' button is at the bottom left.

BikeRent



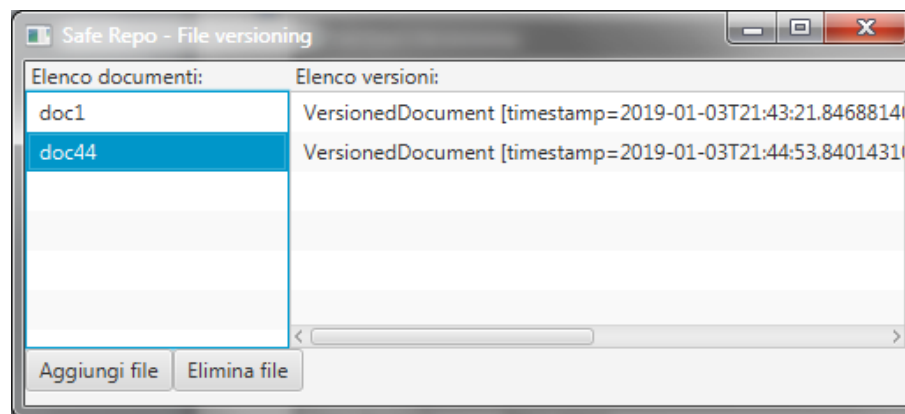
The screenshot shows the BikeRent 'Move yourself!' interface. It features a 'Città' dropdown (Bologna), a red 'Calcola costo' button, and input fields for 'Inizio noleggio' (07/08/2018, 07:20) and 'Fine noleggio' (07/08/2018, 07:45). The 'Costo del noleggio' is displayed as 1,58 €.

Tris



The screenshot shows the Tris application window. On the left, a 3x3 grid shows the game state with 'X' and 'O' markers. Below the grid is a 'Salva' button. On the right, a text area shows the game progress: 'X CONTINUA', 'X O CONTINUA', 'X O X CONTINUA', 'X OO X CONTINUA', 'X XOO X CONTINUA', 'XOXOO X CONTINUA', 'XOXOOX CONTINUA', 'OXOXOOX CONTINUA', and 'OXOXOOX VITTORIA X'.

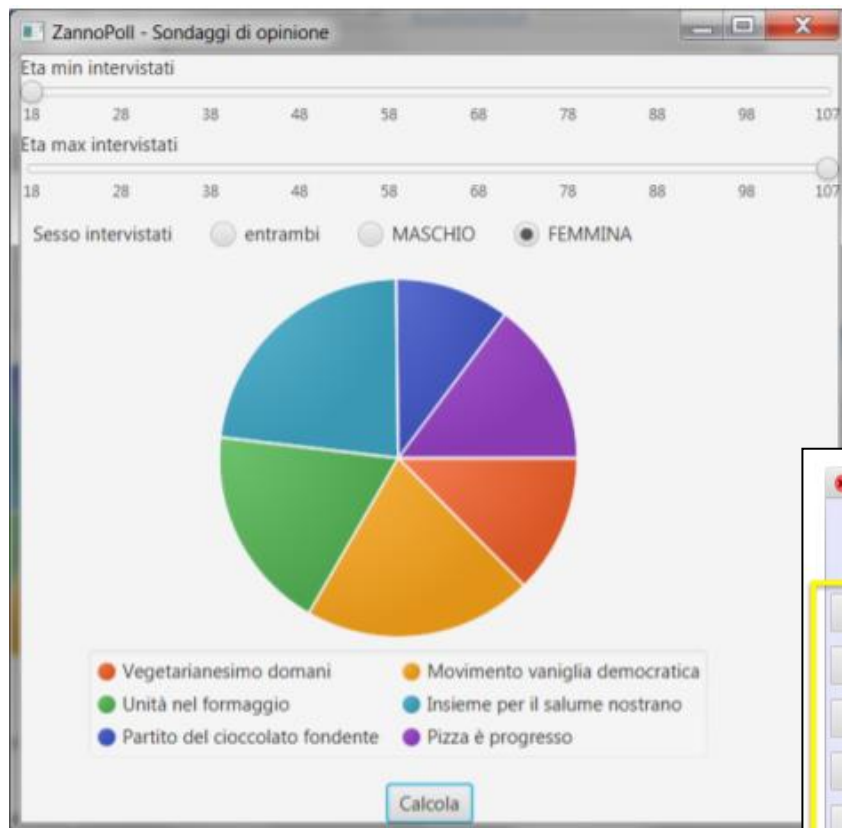
SafeRepo



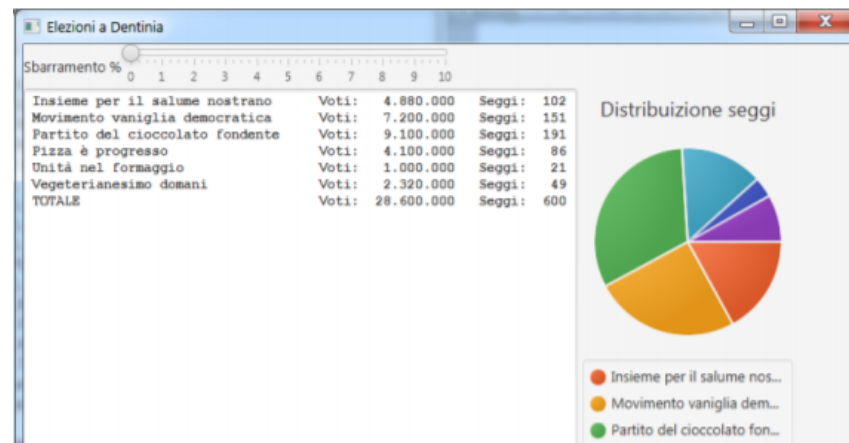
The screenshot shows the SafeRepo 'File versioning' interface. It has two panels: 'Elenco documenti:' and 'Elenco versioni:'. The 'Elenco documenti:' panel shows 'doc1' and 'doc44' (selected). The 'Elenco versioni:' panel shows 'VersionedDocument [timestamp=2019-01-03T21:43:21.8468814]' and 'VersionedDocument [timestamp=2019-01-03T21:44:53.8401431]'. At the bottom are 'Aggiungi file' and 'Elimina file' buttons.

ESEMPI DA COMPITI D'ESAME (2017)

Sondaggi



Elezioni



Gelati, che passione!

ESEMPI DA COMPITI D'ESAME (2017)

Fantacalcio

Modulo: 4-4-2

Portieri

Mirante Antonio
Sarr Mouhamadou Falk

In campo

Da Costa Angelo Esmar

Difensori

M'baye Ibrahima
Masina Adam
Oikonomou Marios
Torosidis Vasilis

In campo

Gastaldello Daniele
Helander Filip
Krafth Emil
Maietta Domenico

Centrocampisti

Pulgar Erick
Rizzo Luca
Taider Saphir
Viviani Federico

In campo

Donsah Godfred
Dzemaili Blerim
Krejci Ladislav
Nagy Adam

Attaccanti

Okonkwo Orji
Petkovic Bruno
Umar Sadiq
Verdi Simone

In campo

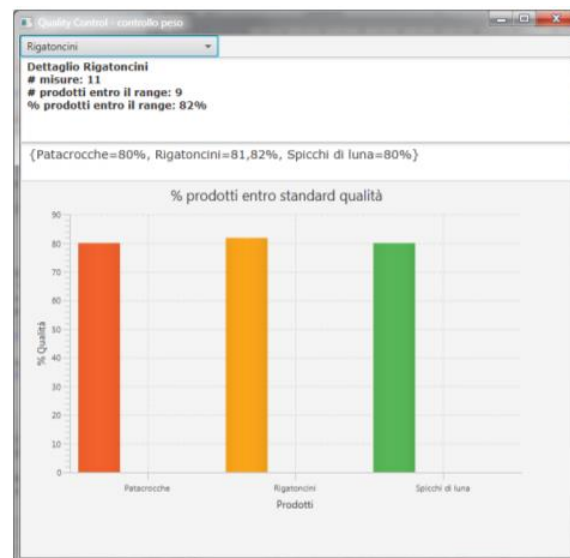
Destro Mattia
Di Francesco Federico

Conferma Formazione

Carica Risultati

Risultato:
Hai totalizzato 87.5 punti che corrispondono a 4 goal

Quality Assurance



Unione Banche di Zannonia

Tagli banconote e monete

€ 500 € 200 € 100 € 50 € 20 € 10 € 5 € 2 € 1

Quantità disponibili in cassa

5 25 30 200 200 100 100 50 50

Importo da prelevare: 307

Preleva

Tagli banconote e monete

€ 500 € 200 € 100 € 50 € 20 € 10 € 5 € 2 € 1

Prelievo

Prelievo contanti



FRAMEWORK GRAFICI IN JAVA

Un po' di storia

- in origine: AWT (Abstract Window Toolkit)
 - disponibile da Java 1 [1996], package `java.awt`
 - ormai in disuso (ne sopravvivono piccole porzioni in Swing)
- la star del quindicennio: Swing
 - disponibile da Java 2 [1998], package `javax.swing`
 - scritto in Java, indipendente dalla piattaforma
 - *usato per 15+ anni, amplissima base codice installata*
 - dal 2015 in *maintenance mode* (non più sviluppato, solo bug fix)
- la novità di Java 8: JavaFX
 - disponibile da Java 8 [2014], set di package `javafx.*`
 - da Java 11, *spostato in un modulo da scaricare separatamente*

JavaFX: installazione e supporto
vedi slide apposite

App JavaFX & JAR eseguibili
vedi slide apposite

Da Swing a JavaFX



FRAMEWORK GRAFICI IN JAVA

Un po' di storia

- in origine: AWT (Abstract Window Toolkit)
 - disponibile da Java 1 [1996], package `java.awt`
 - ormai in disuso (ne sopravvivono piccole porzioni in Swing)
- la star del quindicennio: Swing
 - disponibile da Java 2 [1998], package `javax.swing`
 - scritto in Java, indipendente dalla piattaforma
 - *usato per un decennio, amplissima base codice installata*
 - dal 2015 in *maintenance mode* (non più sviluppato, solo bug fix)
- la novità di Java 8: JavaFX
 - disponibile da Java 8 [2014], set di package `javafx.*`
 - da Java 11, *spostato in un modulo da scaricare separatamente*



SWING vs JAVAFX

- Swing e JavaFX condividono molto
 - concetti di *evento* e *ascoltatore degli eventi (event listener)*
 - tipologie di widget e controlli grafici fondamentali
 - impostazione generale dell'applicazione
- ma **JavaFX è costruito su un approccio più moderno**
 - nuovi componenti e nuovi layout (API di oltre 30 package!)
 - nuovi concetti di *palcoscenico* e *scena*
 - supporto di grafici, effetti speciali e animazioni
 - **nuova struttura dell'applicazione**, che deriva da **Application**
 - lancio diretto di applicazioni grafiche, anche senza main
 - esecuzione di applicazioni anche in un browser
 - **descrizione struttura GUI anche via (F)XML**



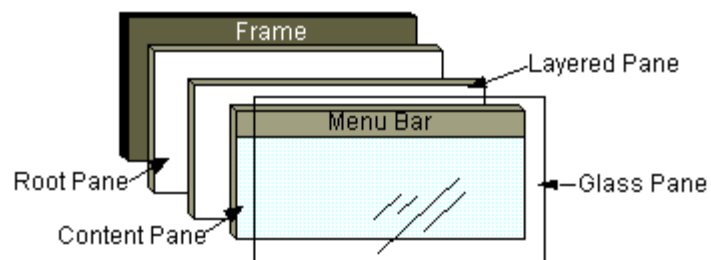
SWING vs JAVA FX

- Un'applicazione Swing richiede la creazione di un **Frame**, in cui va inserito un **Panel**
- Sul pannello si può:
 - disegnare direttamente (grafica a pixel)
 - agganciare componenti (inclusi altri pannelli)
- Organizzazione applicazione *abbastanza libera*

- Un'applicazione JavaFX estende **Application** (eredita il frame) e si basa anch'essa su un **Pane**
- Sul pannello si può:
 - agganciare una tela (**Canvas**) per disegnare direttamente (a pixel)
 - agganciare componenti (inclusi altri pannelli)
- Organizzazione applicazione *imposta dal framework* secondo *l'approccio teatrale*:
 - il pannello (**Pane**) va inserito in una *scena* (**Scene**)
 - la scena va resa visibile sul *palcoscenico* (**Stage**)

SWING vs JAVA FX

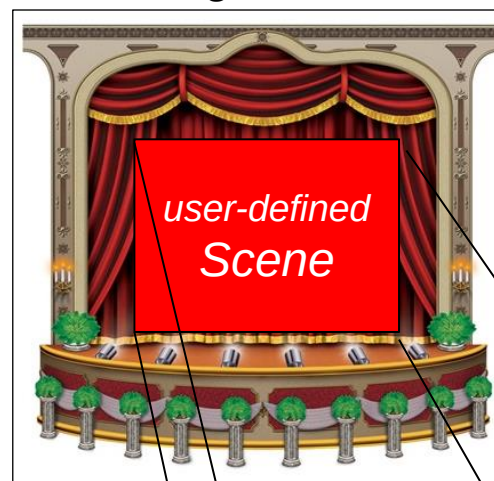
Applicazione Swing



- Sul *content pane* si può:
 - disegnare direttamente (a pixel)
 - agganciare componenti
- Per disegnare si *interferisce nel processo di disegno del pannello*
 - scomodo
 - error-prone
 - disomogeneo rispetto alla grafica a componenti

Applicazione JavaFX

Application (fornisce il frame con lo stage al suo interno)

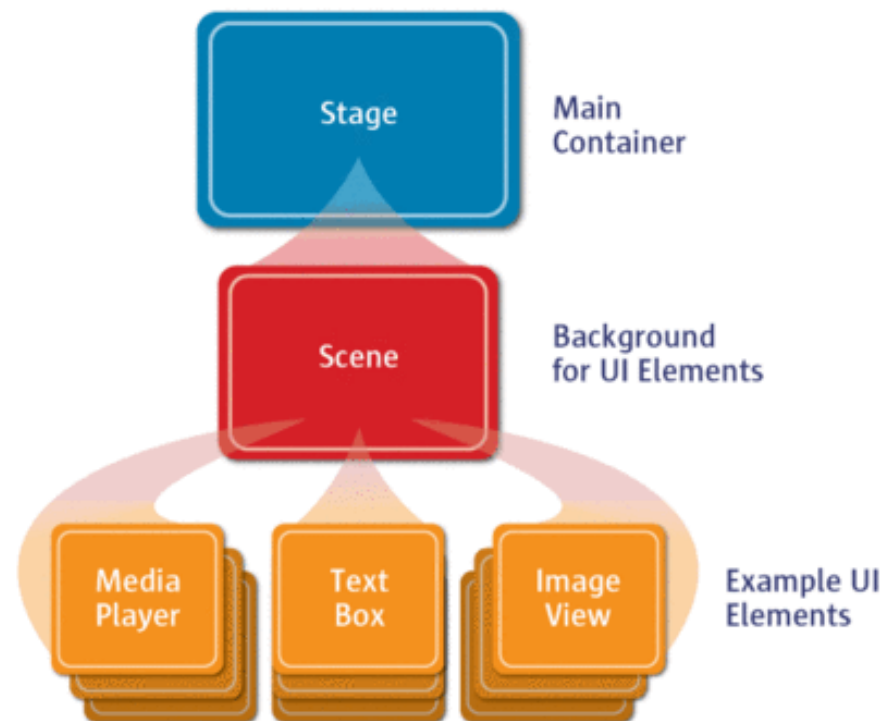


Per disegnare si mette nel Pane un Canvas (tela)
 → uniformità
 → no interferenze

Pane
 (contiene i vari componenti utente)

APPLICAZIONI JAVAFX

- L'applicazione utente estende la classe base **Application**
- **Eredita tre metodi fondamentali:**
 - **init** per eventuale set-up
 - **start** per creare la GUI
 - **stop** per eventuale shut-down
- L'unico da ridefinire sempre è **start**, il cuore dell'applicazione
 - esso rende disponibile lo **Stage** (già creato) associato alla finestra
- L'utente deve *mettere in scena* il suo spettacolo definendo la sua **Scene**: un opportuno **Pane** deve ospitare i vari componenti.

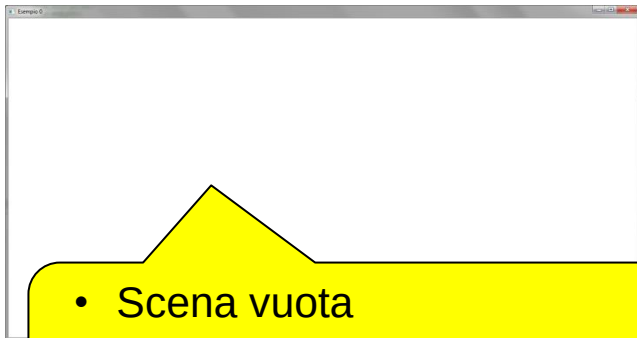


JAVAFX: UN PRIMO ESEMPIO

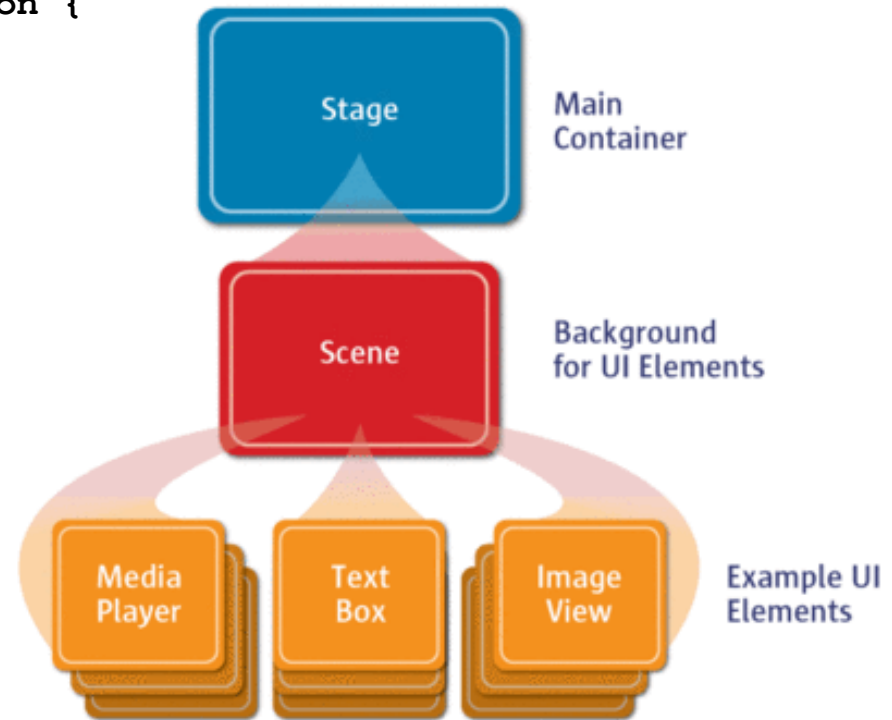
```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
```

All'inizio, implementare con
Notepad + riga di comando

```
public class EsJavaFX00 extends Application {
    public void start(Stage stage){
        stage.setTitle("Esempio 0");
        stage.show();
    }
}
```



- Scena vuota
- Stage di default, di dimensioni prefissate, a centro schermo





JAVAFX: UN PRIMO ESEMPIO

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
```

All'inizio, implementare con
Notepad + riga di comando

```
public class EsJavaFX00 extends Application {
    public void start(Stage stage){
        stage.setTitle("Esempio 0");
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

- In JavaFX, il compilatore JDK aggiunge automaticamente il main di default, che richiama il metodo ereditato *launch*
- **In Eclipse ciò avviene solo se il progetto è del tipo *JavaFX project*, previa installazione del plugin**
- Se si fa un progetto Java standard, occorre *scrivere a mano questo main e aggiungere la libreria JavaFX*

JAVAFX: UN PRIMO ESEMPIO

The screenshot displays the Eclipse IDE environment for a JavaFX application. The top window, titled 'Lab2017 - Java - Prova2/src/EsJavaFX04.java - Eclipse', shows the 'Package Explorer' on the left and the 'EsJavaFX04.java' editor on the right. The 'Package Explorer' lists a project named 'Prova2' with a sub-package 'src'. Under 'src', the 'JRE System Library [JavaSE-1.8]' and 'JavaFX SDK' are listed, with the latter highlighted by a red box. The 'EsJavaFX04.java' editor contains the following code:

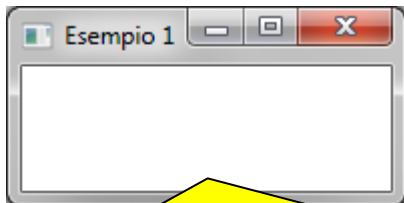
```
1 import javafx.application.Application;
2 import javafx.scene.Scene;
3 import javafx.scene.canvas.Canvas;
4 import javafx.scene.canvas.GraphicsContext;
5 import javafx.scene.layout.FlowPane;
6 import javafx.scene.paint.Color;
7 import javafx.scene.text.Font;
8 import javafx.scene.text.FontWeight;
9 import javafx.stage.Stage;
10
11 public class EsJavaFX04 extends Application {
12     public void start(Stage stage){
13         stage.setTitle("Esempio 4");
14         FlowPane panel = new FlowPane();
15         panel.setPrefSize(200,130);
16         Canvas canvas = new Canvas(150, 130);
17         panel.getChildren().add(canvas);
18         GraphicsContext g = canvas.getGraphicsContext2D();
19         g.setFont(Font.font("Serif", FontWeight.BOLD, 20));
20         g.setFill(Color.RED); g.fillRect(20,20, 100,80);
21         g.setFill(Color.BLUE); g.strokeRect(30,30, 80,60);
22         g.setFill(Color.BLACK); g.fillText("ciao",50,60);
23         Scene scene = new Scene(panel);
24         stage.setScene(scene);
25         stage.show();
26     }
27     public static void main(String[] args) {
28         Launch(args);
29     }
30 }
31
```

The bottom window, titled 'Properties for Prova2', shows the 'Java Build Path' tab. It lists the 'JRE System Library [JavaSE-1.8]' and the 'JavaFX SDK' in the 'Libraries' section. The 'JavaFX SDK' is highlighted by a red box. The 'Add Library...' button is also highlighted by a red box.

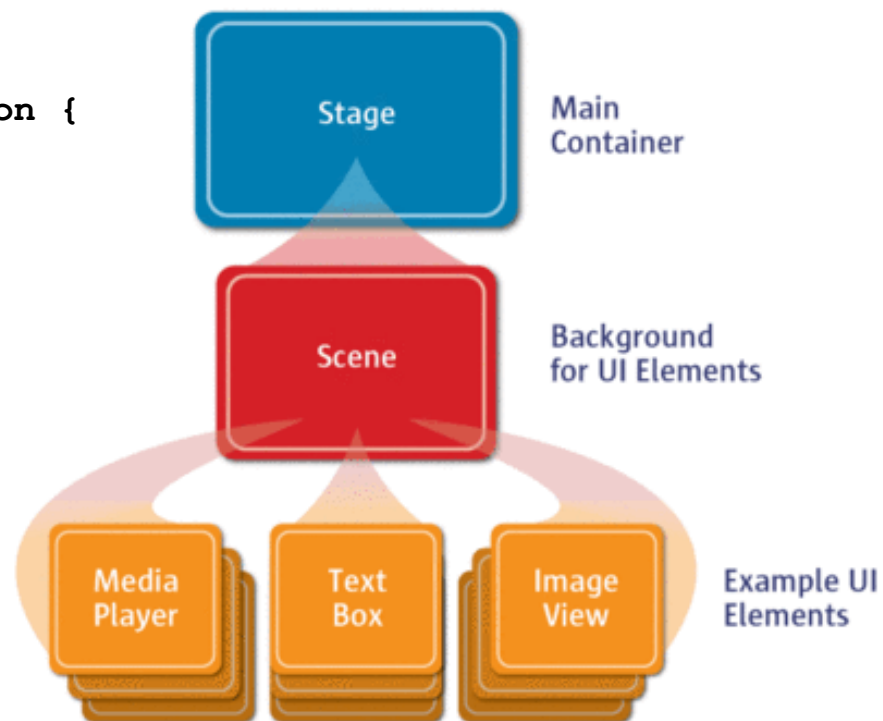
PRIMO ESEMPIO (variante)

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class EsJavaFX01 extends Application {
    public void start(Stage stage){
        stage.setTitle("Esempio 1");
        stage.setWidth(200);
        stage.setHeight(100);
        stage.setX(300);
        stage.setY(300);
        stage.show();
    }
}
```



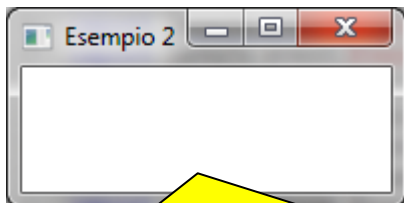
- Scena sempre vuota
- Stage dimensionato e posizionato come desiderato



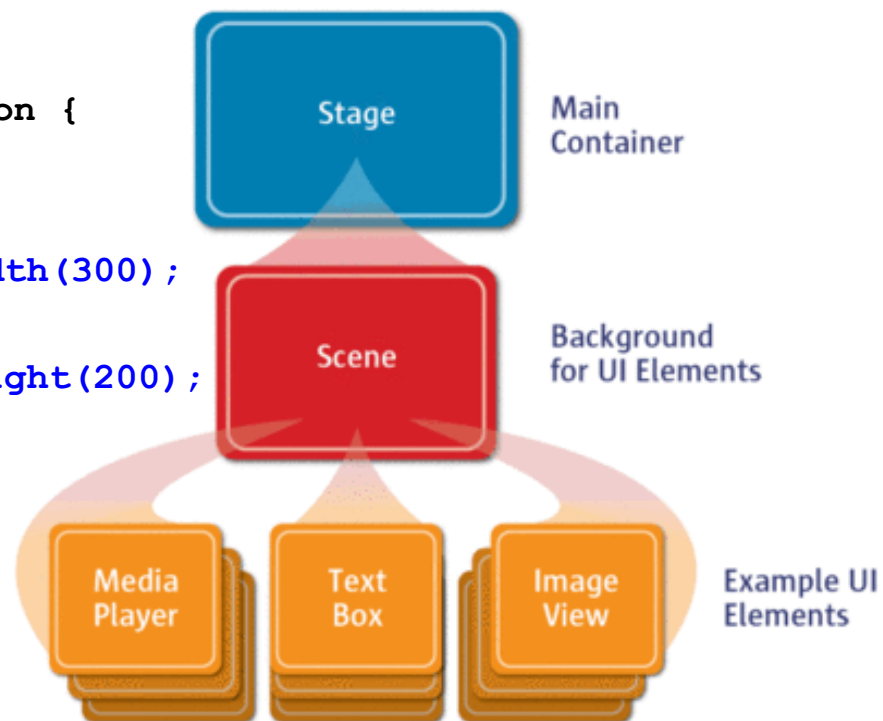
PRIMO ESEMPIO (variante)

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
```

```
public class EsJavaFX02 extends Application {
    public void start(Stage stage){
        stage.setTitle("Esempio 2");
        stage.setMinWidth(100); stage.setMaxWidth(300);
        stage.setWidth(200);
        stage.setMinHeight(50); stage.setMaxHeight(200);
        stage.setHeight(100);
        stage.show();
    }
}
```



- Stage dimensionato e posizionato come desiderato
- Stage ridimensionabile, ma solo entro i limiti dati



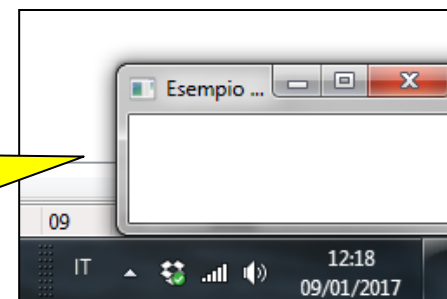


PRIMO ESEMPIO (ulteriore variante)

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class EsJavaFX02bis extends Application {
    public void start(Stage stage){
        stage.setTitle("Esempio 2");
        stage.setWidth(200);
        stage.setHeight(100);
        javafx.geometry.Rectangle2D screen = Screen.getPrimary().getVisualBounds();
        System.out.println(screen); // per me: 1920 x 1040
        stage.setX(screen.getMinX() + screen.getWidth() - stage.getWidth());
        stage.setY(screen.getMinY() + screen.getHeight() - stage.getHeight());
        stage.show();
    }
}
```

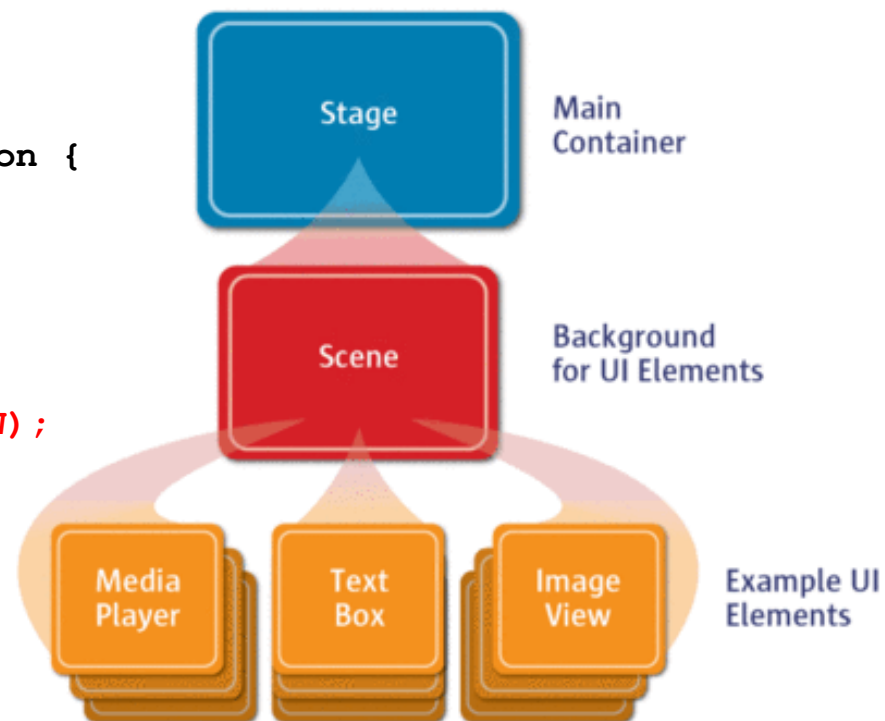
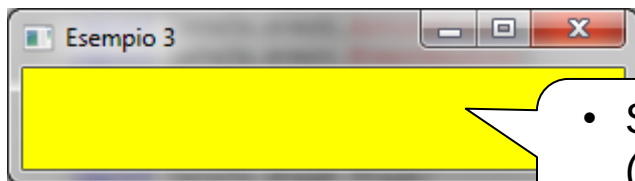
- Si recuperano le dimensioni dello schermo
- Si posiziona lo stage al bordo inferiore destro



JAVAFX: LA PRIMA SCENA

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class EsJavaFX03 extends Application {
    public void start(Stage stage){
        stage.setTitle("Esempio 3");
        Pane root = new Pane();
        Scene scene =
            new Scene(root, 300, 50, Color.YELLOW);
        stage.setScene(scene);
        stage.show();
    }
}
```



- Scena gialla, 300x50: occupa tutto lo stage (che quindi non ha dimensione propria)
- Il pannello root (vuoto) tiene tutta la scena

JAVAFX: SCENA con CANVAS

```
public class EsJavaFX04 extends Application {
```

```
    public void start(Stage stage){
```

```
        stage.setTitle("Esempio 4");
```

```
        FlowPane panel = new FlowPane();
```

```
        panel.setPrefSize(200,130);
```

```
        Canvas canvas = new Canvas(150, 130);
```

```
        panel.getChildren().add(canvas);
```

```
        GraphicsContext g = canvas.getGraphicsContext2D();
```

```
        g.setFont(Font.font("Serif", FontWeight.BOLD, 20));
```

```
        g.setFill(Color.RED);    g.fillRect(20,20, 100,80);
```

```
        g.setFill(Color.BLUE);  g.strokeRect(30,30, 80,60);
```

```
        g.setFill(Color.BLACK); g.fillText("ciao",50,60);
```

```
        Scene scene = new Scene(panel);
```

```
        stage.setScene(scene);
```

```
        stage.show();
```

```
    }
```

```
}
```

Pannello di tipo **FlowPane** (a flusso):
dispone i suoi componenti da sinistra
a destra e dall'alto in basso

Canvas: una tela per grafica a pixel

Aggiunta del **Canvas** alla lista dei figli
del pannello

Dal **Canvas** si recupera
l'entità (contesto grafico)
che serve per disegnare.

Tramite il **GraphicsContext** ricevuto si pilota
il disegno, usando i metodi **strokeXXX** (figure
con solo contorno) e **fillXXX** (figure piene)

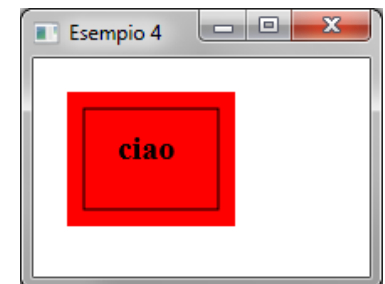


JAVAFX: SCENA con CANVAS

```
public class EsJavaFX04 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 4");  
  
        FlowPane panel = new FlowPane();  
        panel.setPrefSize(200,130);  
  
        Canvas canvas = new Canvas(150, 130);  
        panel.getChildren().add(canvas);  
  
        GraphicsContext g = canvas.getGraphicsContext2D();  
  
        g.setFont(Font.font("Serif", FontWeight.BOLD, 20));  
        g.setFill(Color.RED);      g.fillRect(20,20, 100,80);  
        g.setFill(Color.BLUE);    g.strokeRect(30,30, 80,60);  
        g.setFill(Color.BLACK);   g.fillText("ciao",50,60);  
  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

- Tramite il metodo **setFont** si può impostare il **Font** per le scritte.
- Lo specifico font va creato o con la **new** (sconsigliato) o tramite l'apposita factory internalizzata, **Font.font**

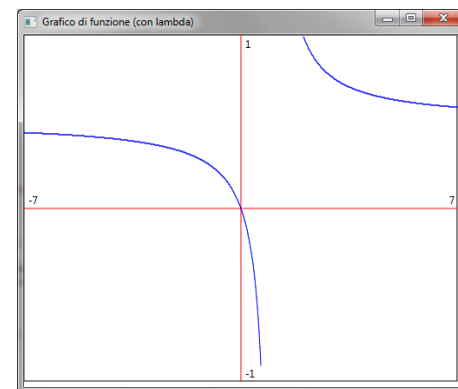
È anche possibile impostare il **Color** da usare per la diverse parti del disegno.



JAVAFX: GRAFICO DI FUNZIONE

```
public class EsJavaFX05 extends Application {  
    static Function<Float,Float> f1 =  
        x -> (float) (2.0/Math.PI*Math.sin(1.0*x)+2.0/(3.0*Math.PI)*Math.sin(3.0*x));  
    static Function<Float,Float> f2 = x -> 0.5F*x/(x-1);  
  
    private int xAxisMin=-7, xAxisMax=7, yAxisMin=-1, yAxisMax=1;  
    private int larghezza=500, altezza=400;  
    private float fattoreDiScalaX = larghezza/((float)xAxisMax-xAxisMin);  
    private float fattoreDiScalaY = altezza/((float)yAxisMax-yAxisMin);  
  
    public void start(Stage stage){  
        stage.setTitle("Grafico di funzione");  
        FlowPane panel = new FlowPane();  
        Canvas canvas = new Canvas(528, 448);  
        panel.getChildren().add(canvas);  
        GraphicsContext g = canvas.getGraphicsContext2D();  
        doMyGraphics(g, f3); // LA FUNZIONE DA GRAFICARE  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

NB: non esiste
FloatUnaryOperator



Il metodo di utilità **doMyGraphics**
è definito da noi (v. oltre)



GRAFICO DI FUNZIONE: doMyGraphics

```
private void doMyGraphics(GraphicsContext g, Function<Float,Float> f ){
    g.setStroke(Color.BLACK);          // cornice nera
    g.strokeRect(0,0,larghezza-1,altezza-1);
    g.setStroke(Color.RED);            // assi cartesiani rossi
    g.strokeLine(0,altezza/2, larghezza-1,altezza/2);
    g.strokeLine(larghezza/2,0, larghezza/2,altezza-1);
    g.fillText(""+xAxisMin, 5,altezza/2-5); // graduazione assi
    g.fillText(""+xAxisMax, larghezza-10,altezza/2-5);
    g.fillText(""+yAxisMax, larghezza/2+5,15);
    g.fillText(""+yAxisMin, larghezza/2+5,altezza-5);
    g.setStroke(Color.BLUE);          // funzione in blu
    float fXAxisMin = f.apply(new Float(xAxisMin));
    float xPrev=xAxisMin, yPrev=fXAxisMin;
    setPixel(g,xAxisMin,fXAxisMin);
    for (int ix=1; ix<g.getCanvas().getWidth(); ix++){
        float x = xAxisMin+((float)ix)/fattoreDiScalaX;
        float y = f.apply(x);
        setPixel(g,x,y); // OPPURE: setLine(g,xPrev,yPrev,x,y); SE f CONTINUA
        xPrev=x; yPrev=y;
    }
}
```

La lambda expression **f** ha come tipo la *functional interface* **Function<U,V>**

Motivo: non esiste FloatUnaryOperator (alternative: passare ai double, o definirlo)

Calcolo di f(x)
[notare che **f** è una lambda expression]

I metodi di utilità **setPixel** e **setLine** sono definiti da noi (v. oltre)

GRAFICO DI FUNZIONE: setPixel / setLine

```
void setPixel(GraphicsContext g, float x, float y){  
    if (x<xAxisMin || x>xAxisMax || y<yAxisMin || y>yAxisMax ) return;  
    int ix = Math.round((x-xAxisMin)*fattoreDiScalaX);  
    int iy = altezza-Math.round((y-yAxisMin)*fattoreDiScalaY);  
    g.strokeLine(ix,iy,ix,iy);  
}
```

Cosa succede
togliendola?

Trasformazione di coordinate e
accensione singolo pixel (ix,iy)

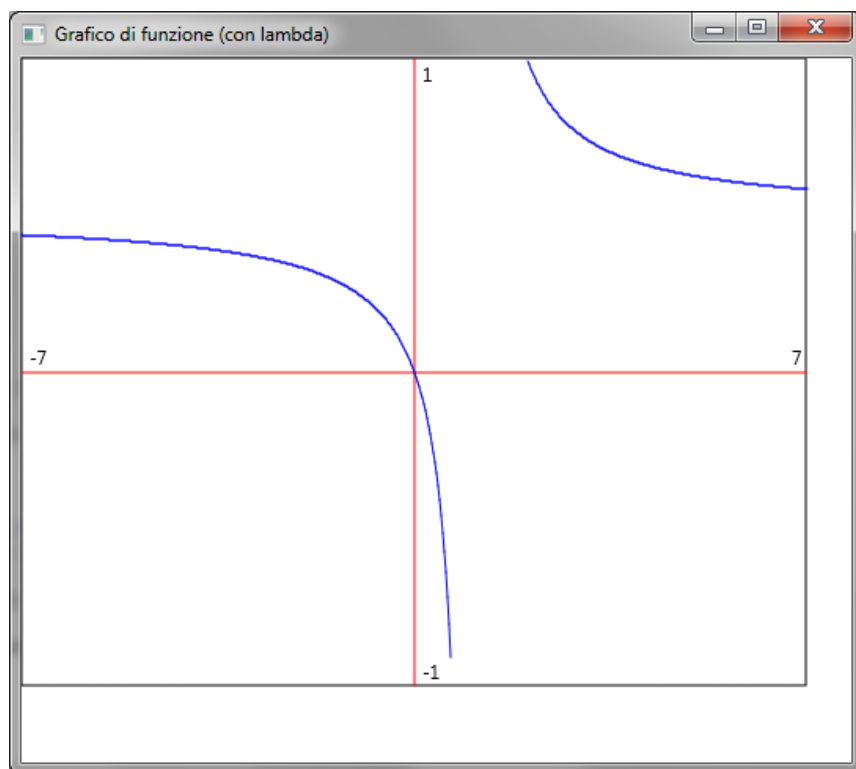
```
void setLine(GraphicsContext g, float xP, float yP, float x, float y){  
    if (x<xAxisMin || x>xAxisMax || y<yAxisMin || y>yAxisMax ||  
        xP<xAxisMin || xP>xAxisMax || yP<yAxisMin || yP>yAxisMax ) return;  
    int ix = Math.round((x-xAxisMin)*fattoreDiScalaX);  
    int iy = altezza-Math.round((y-yAxisMin)*fattoreDiScalaY);  
    int ixP = Math.round((xP-xAxisMin)*fattoreDiScalaX);  
    int iyP = altezza-Math.round((yP-yAxisMin)*fattoreDiScalaY);  
    g.strokeLine(ixP,iyP,ix,iy);  
}
```

Cosa succede
togliendola, se f
è discontinua?

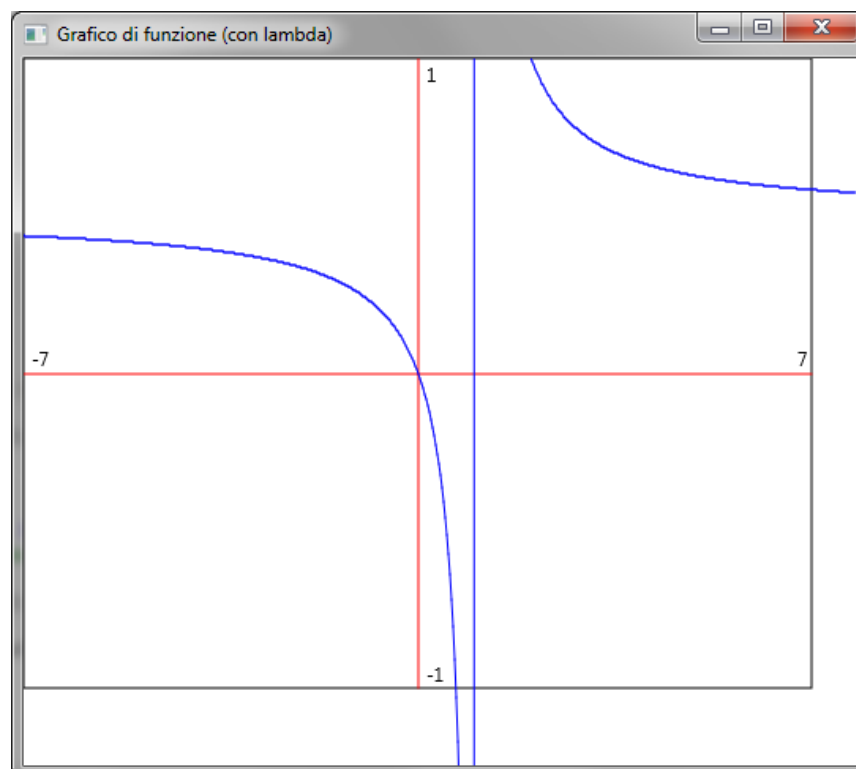
Trasformazione di coordinate e
disegno segmento dal pixel attuale
(ix,iy) al pixel precedente (ixP, iyP)

GRAFICO DI FUNZIONE con funzioni discontinue

setLine con controllo



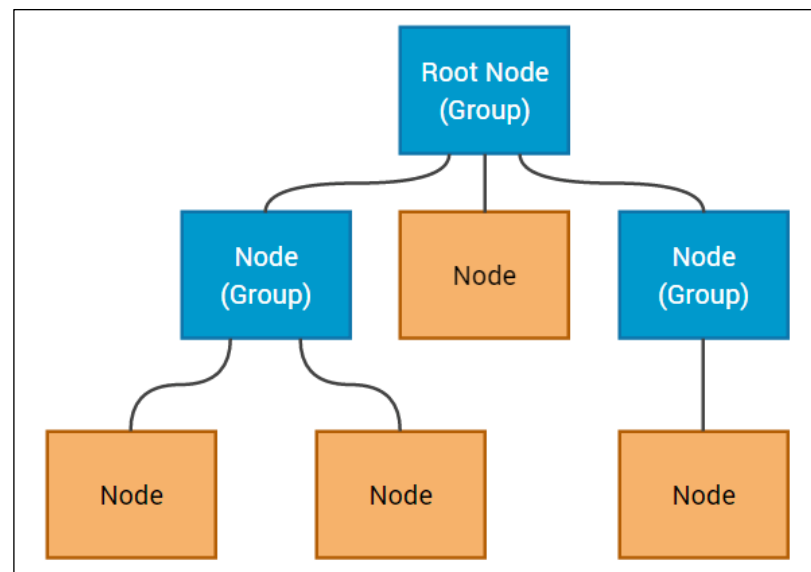
setLine **senza** controllo



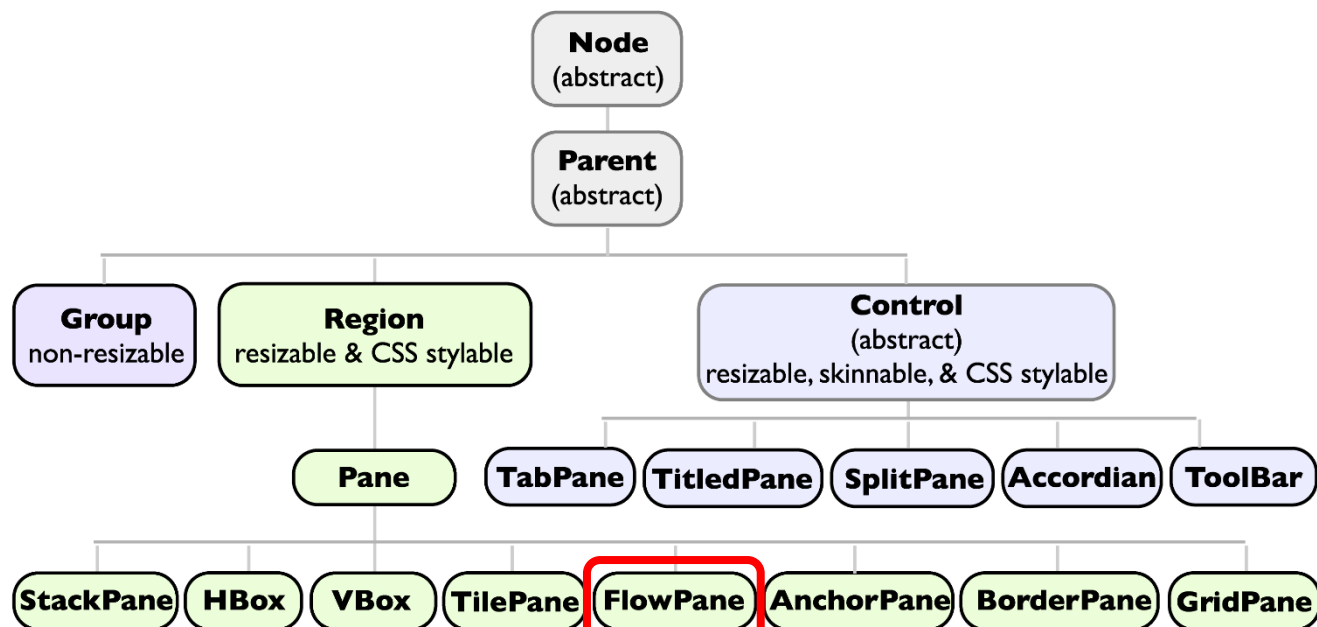
Grafica a componenti (senza gestione di eventi)

APPLICAZIONI JAVAFX

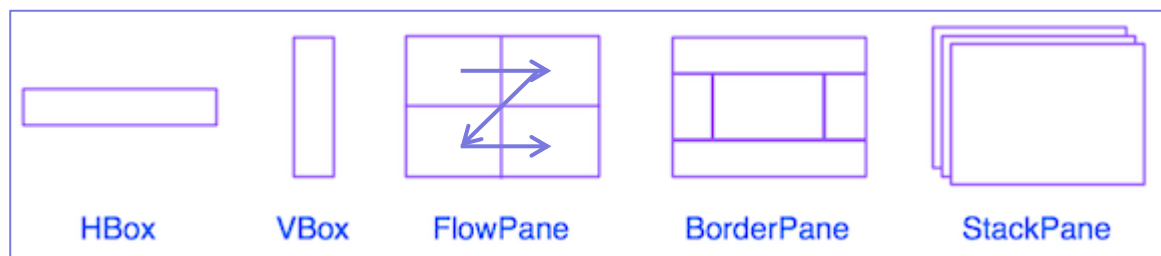
- Strutturalmente, un'applicazione JavaFX è fatta di *nodi*
- Il *pannello* inserito nella scena è il *nodo radice* (root node)
 - ogni altro componente aggiunto al pannello è un nodo-figlio
 - alcuni nodi possono essere *gruppi di altri nodi* (altri pannelli)
- Il pannello dispone i propri componenti secondo un certo *layout*
 - non si usano coordinate assolute, perché nei vari device fisici le finestre hanno dimensioni diverse, si ridimensionano, possono ruotare...
 - ogni pannello incorpora un *gestore di layout* che colloca i vari widget secondo una ben precisa *politica*, così da garantire l'adattamento *automatico* anche se finestra e pannelli vengono ruotati, ridimensionati, etc.



LAYOUT PANES

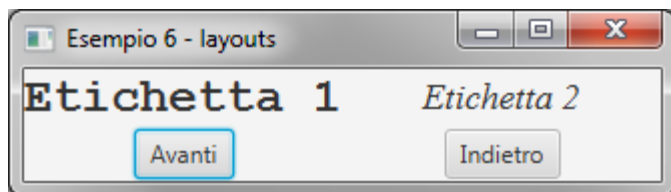


Il più semplice



LAYOUT PANES: CONFRONTO

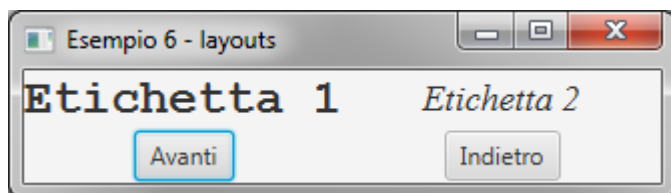
FlowPane



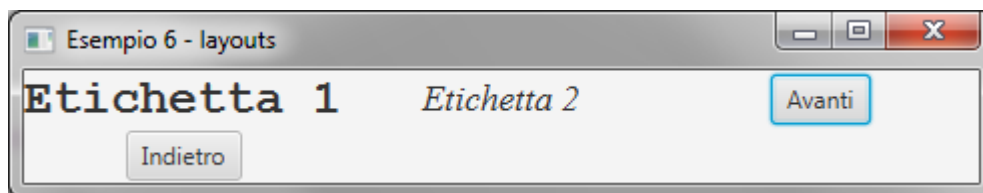
HBox



TilePane a due colonne...



... e a tre colonne



BorderPane



GridPane (griglia 3x3)



VBox





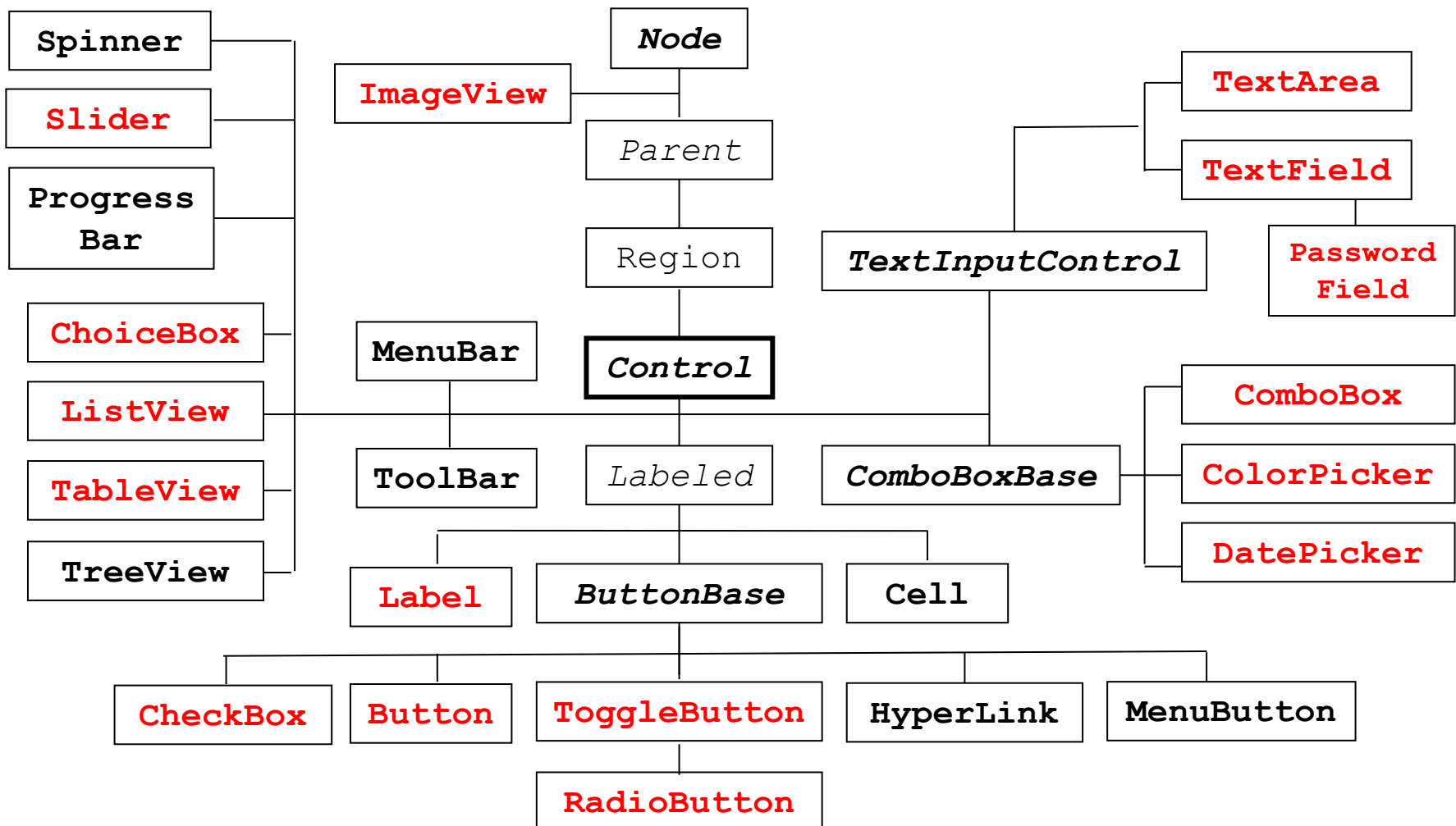
CONTROLLI JAVAFX: PANORAMICA

- *Bottoni*
 - pulsante, interruttore on/off, casella di opzione, bottone radio
- *Componenti di testo*
 - etichetta*, campo a riga singola, password, area di testo a più righe
(*) unico componente di solo output, non interattivo
- *Caselle combinate, liste e tabelle*
 - lista classica, casella pop-up, casella combinata, tabella di valori
- *Regolazioni*
 - barra di regolazione, selettore di date, selettore di colore
- *Dialoghi*
 - finestra di input, conferma, warning, apertura / salvataggio file
- *Grafici*
 - a linee, istogrammi, a torta..



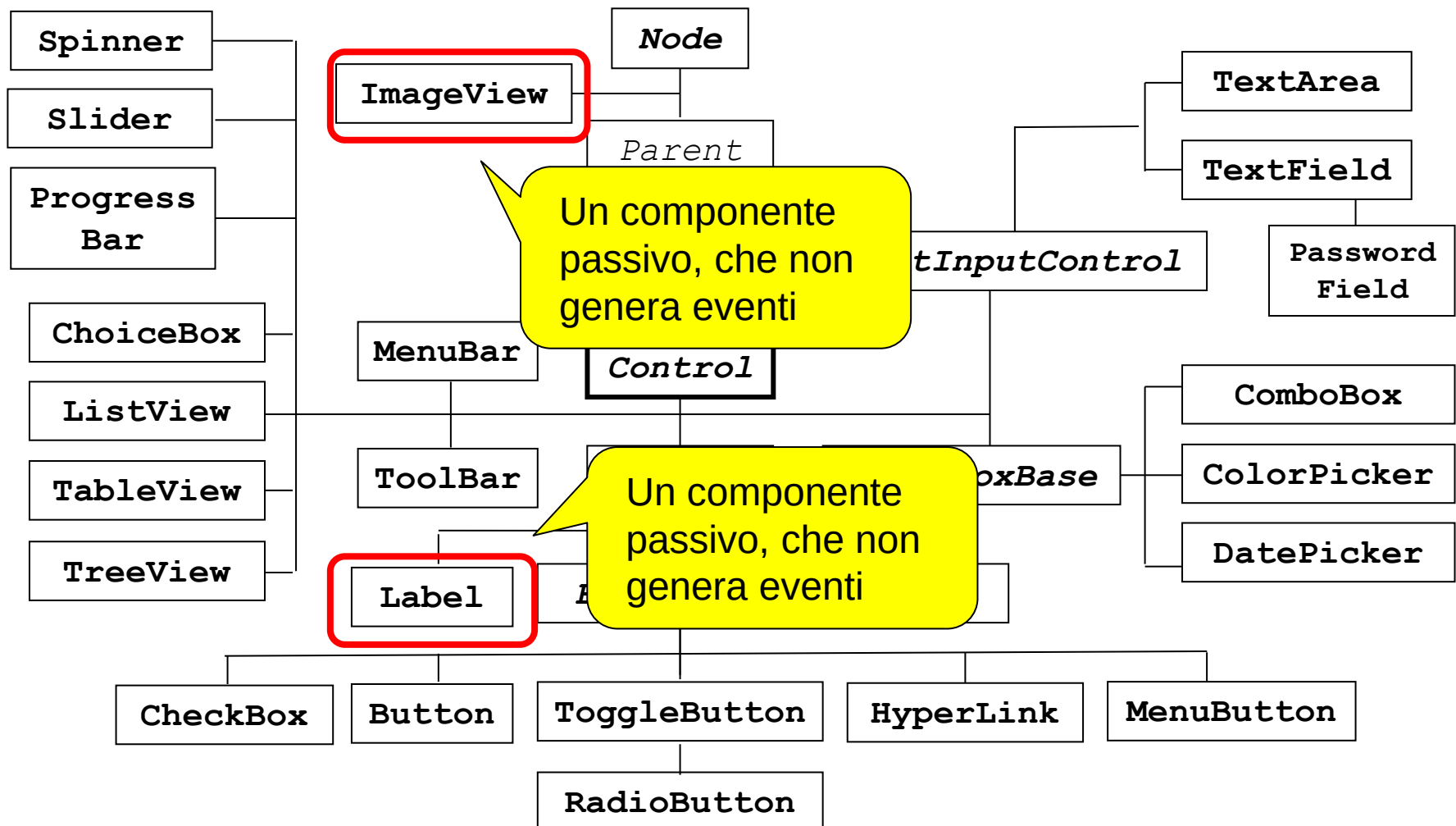
GERARCHIA DEI CONTROLLI

`javafx.scene.control`



GERARCHIA DEI CONTROLLI

`javafx.scene.control`





IL COMPONENTE Label

```
public class EsJavaFX06 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 6");  
        FlowPane panel = new FlowPane();  
        Label l1 = new Label("Etichetta 1");  
        Label l2 = new Label("Etichetta 2");  
        l1.setFont(Font.font("Courier New", FontWeight.BOLD, 24));  
        l2.setFont(Font.font("Times New Roman", FontPosture.ITALIC, 18));  
        panel.getChildren().addAll(l1,l2);  
        Scene scene = new Scene(panel,Color.WHITE);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

NB: aggiungere le opportune `import`

Due `Label` con `Font` diverso

Aggiunta delle due `Label` alla lista dei figli del panel

...ma si può ridimensionarlo

Di default, `FlowPane` si estende quanto serve per ospitare tutti i componenti...



IL COMPONENTE ImageView

```
public class EsJavaFX07 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 7");  
        BorderPane panel = new BorderPane();  
        panel.setCenter(new ImageView(  
            new Image("alberi.jpg", 400, 300, true, false)));  
        Scene scene = new Scene(panel, Color.WHITE);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

NB: aggiungere le opportune **import**

Un diverso tipo di pannello

Ogni **ImageView** mostra una **Image**

La **Image** incapsula una immagine (qui, letta da file)

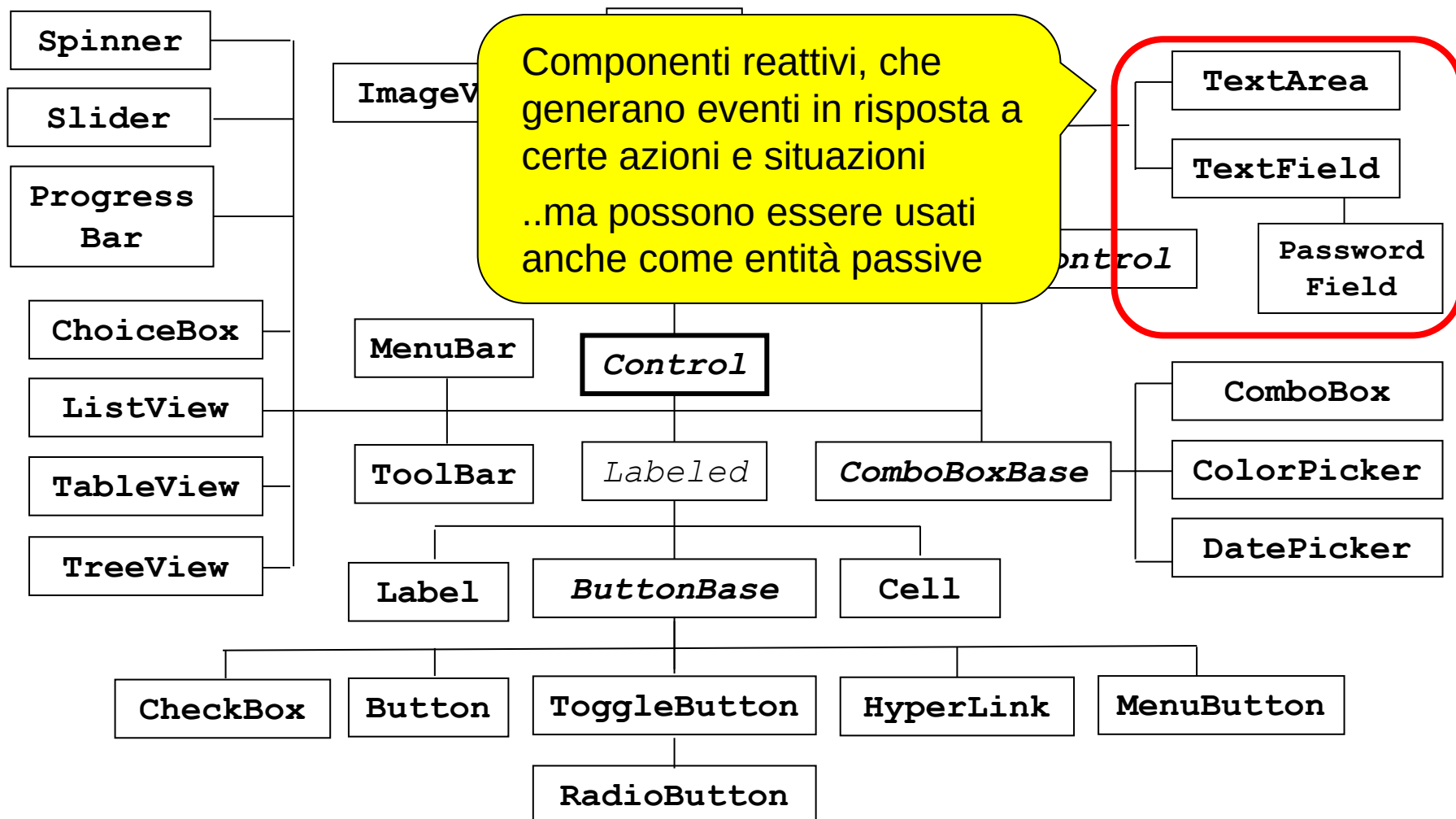
Image

- supporta JPG, BMP, PNG, GIF
- ha vari costruttori (da file, da URL): questo accetta le dimensioni preferite e due boolean (preserva aspetto, effetto smooth)



GERARCHIA DEI CONTROLLI

`javafx.scene.control`





COMPONENTI DI TESTO

- Consentono l'immissione di testo
 - a riga singola: **TextField** (e **PasswordField**)
 - su più righe: **TextArea**
- Pilotabili da programma
 - metodi base: **setText** , **getText** , **setEditable**
 - per **TextArea**: **appendText**, **setPrefColumnCount**
- Generano opportuni eventi se
 - si preme ENTER sulla tastiera (tranne **TextArea**)
 - cambia il testo contenuto
- .. ma non è detto che servano sempre!

Ricordate **GraphicApplication** ..? ☺



GraphicApplication

Una applicazione grafica con un'area di testo

ed.utils

Class GraphicApplication

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method and Description
void	main() Contiene la logica dell'applicazione.
void	print (java.lang.String txt) Mostra la stringa data nell'area di output della finestra.
void	setTitolo (java.lang.String titolo) Imposta il titolo della finestra.

Prova2

Area di output

Ecco il risultato di 5 + 9
14

Ora siamo in grado di capire com'è realizzata



GraphicApplication (1/2)

```
public class GraphicApplication extends Application {  
    protected TextArea ta;  
    private Stage primaryStage;  
    private String titolo;  
  
    public void start(Stage stage){  
        this.primaryStage=stage;  
        stage.setTitle(titolo);  
        Label l = new Label("Area di output");  
        ta = new TextArea();  
        ta.setPrefColumnCount(25);  
        ta.setEditable(false);  
        FlowPane panel = new FlowPane();  
        panel.getChildren().addAll(l, ta);  
        stage.setScene(new Scene(panel, 320, 220));  
        stage.show();  
        main();  
    }  
    ...  
}
```

Imposta la dimensione preferita in termini di colonne



GraphicApplication (2/2)

...

// METODI OFFERTI ALL'UTENTE

```
public void main(){  
    ta.setText("Welcome");  
}
```

Implementazione di default: è destinato a essere sovrascritto dalla logica dell'utente

```
public void setTitolo(String titolo){  
    this.titolo=titolo;  
    primaryStage.setTitle(titolo);  
}
```

```
public void print(String txt){  
    ta.appendText(txt);  
}  
}
```

OSSERVA: il contenuto dell'area è *pilotato in toto* da programma. Non ci sono "reazioni a eventi grafici" dell'utente.

Grafica a componenti con gestione di eventi



EVENT-DRIVEN PROGRAMMING

- I controlli grafici *generano eventi* quando:
 - si compie una qualche *azione* su di essi
 - talune loro *proprietà* cambiano
- Occorre stabilire:
 - *chi* debba essere informato di tali accadimenti
 - *come reagire* a tali accadimenti
- La *parte strutturale* dell'applicazione stabilisce *chi* sia *notificato* dell'accaduto.
- La *parte comportamentale* stabilisce invece *come reagire* all'accaduto.

ARCHITETTURA

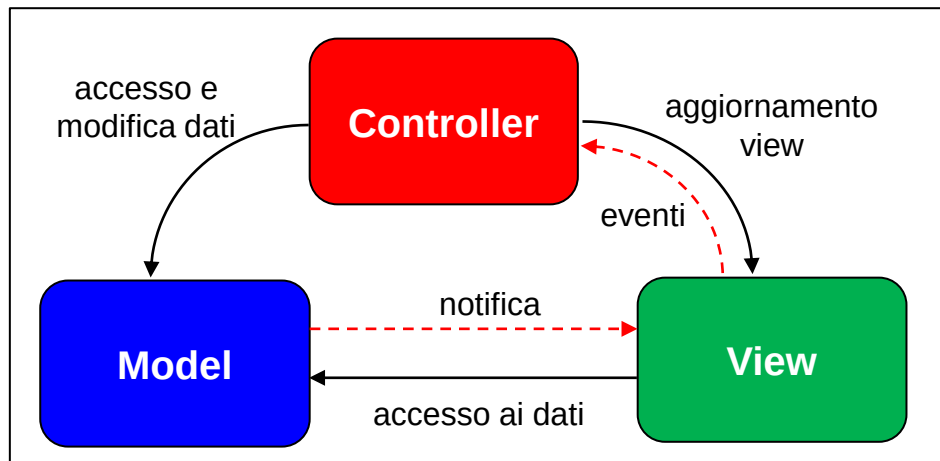
- Ogni framework grafico è progettato secondo un *ben preciso pattern* che implica una ben precisa *architettura* per le relative applicazioni
 - OBIETTIVO 1: massimo disaccoppiamento
 - OBIETTIVO 2: massima *testability*
- Entità fondamentali in gioco
 - *il modello dei dati*: i dati gestiti
 - *la/e vista/e grafica/e* mostrata/e
 - *il burattinaio che li controlla e "incolla"*
- Approcci tipici: MVC, MVVM (,MVP)

Model

View

Controller
(ViewModel)

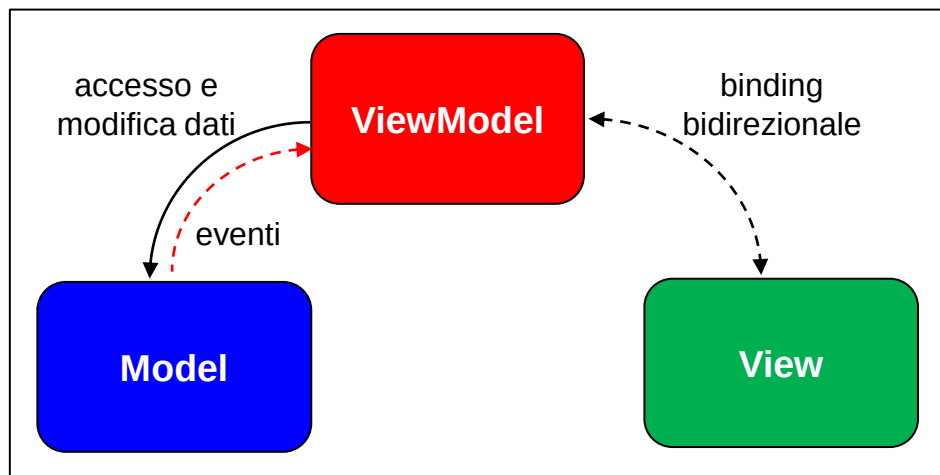
ARCHITETTURA



Schema

Model-View-Controller

- introduce controller esplicito
- disaccoppiamento fra Model e View non totale



Schema

Model-View-ViewModel

- disaccoppia Model e View
- **JavaFX**

Legenda

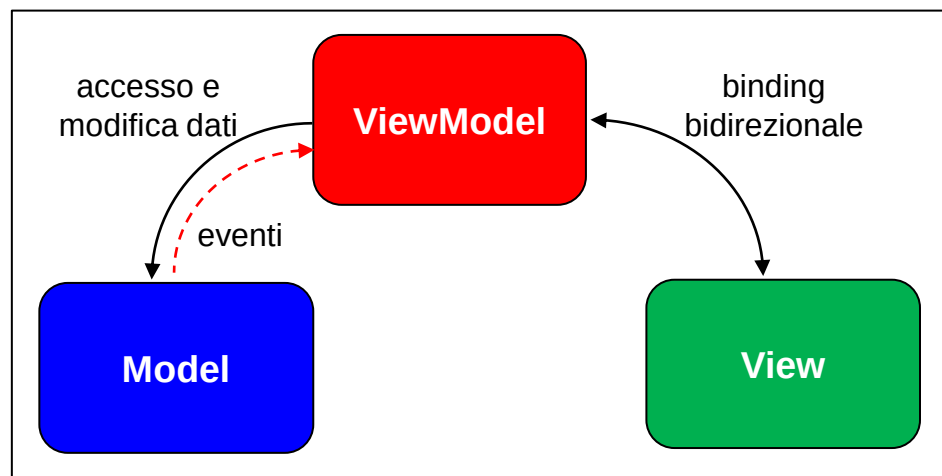
- associazioni dirette
- - - - -→ associazioni indirette

ARCHITETTURA

Lo schema è adottato dal framework

- non significa che NOI dobbiamo per forza fare tre classi...
- ..ma dobbiamo capire come LUI è organizzato, per capirne i metodi e la logica di funzionamento

Nelle nostre applicazioni, potremo anche usare MVC

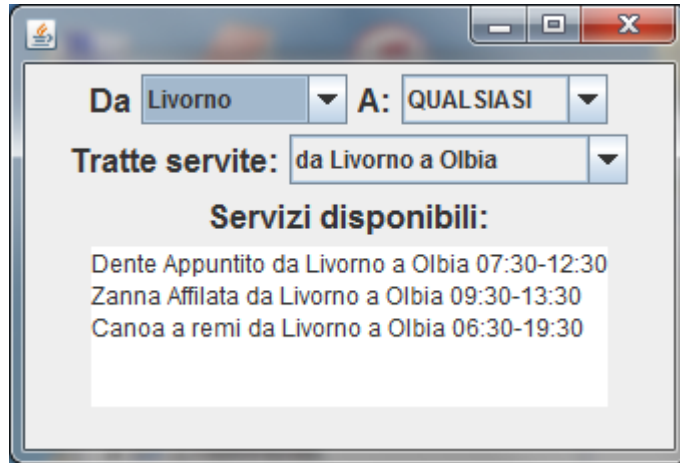


Schema

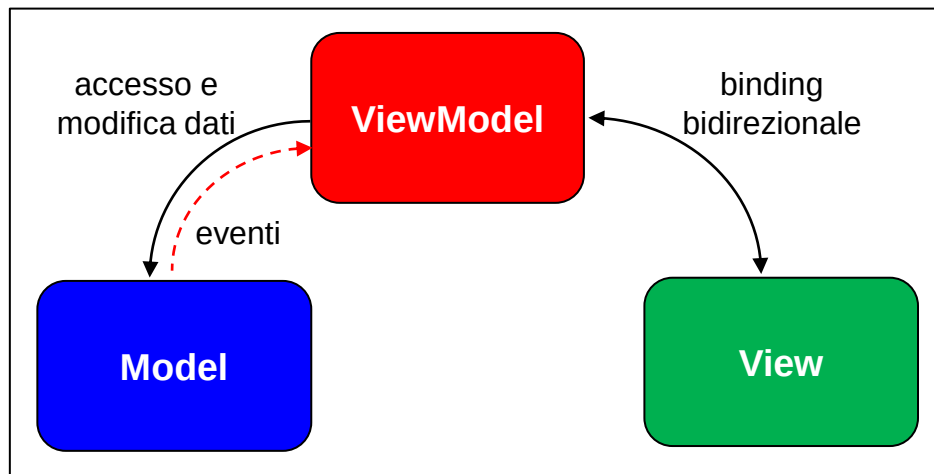
Model-View-ViewModel

- disaccoppia totalmente Model e View
- **JavaFX**

ESEMPIO: TeethFerries



- Il *model* rappresenta i *porti*, le *tratte* e i *servizi* svolti.
- La *view* è la finestra grafica coi controlli che vedete.
- Il "*burattinaio*" *governa l'interazione*: su richiesta della view, cerca i servizi, fornisce l'elenco delle tratte, etc.



Schema

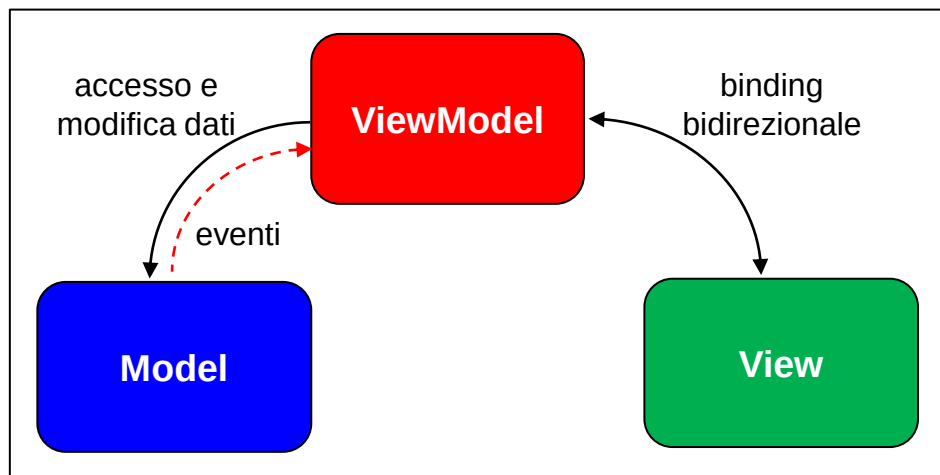
Model-View-ViewModel

- disaccoppia totalmente Model e View
- **JavaFX**

EVENTI & GESTIONE

Gli *eventi* sono il modo di trasmettere informazione fra i widget e il "burattinaio"

- Quando l'utente agisce sul widget, questo genera un *evento*
- L'evento è recapitato *solo al componente preventivamente collegato a esso* – *l'ascoltatore (listener) dell'evento*



Schema

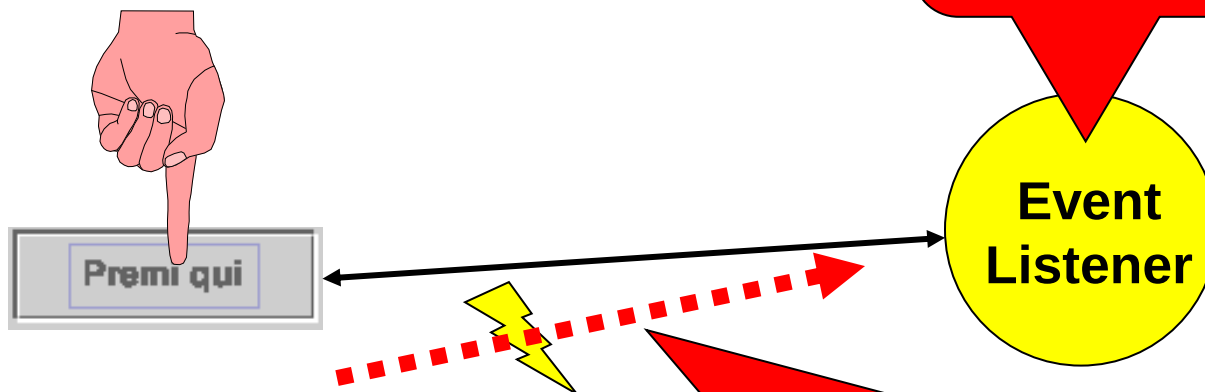
Model-View-ViewModel

- disaccoppia totalmente Model e View
- **JavaFX**

EVENTI & GESTIONE

Ogni componente viene associato a un *ascoltatore degli eventi* (un oggetto che implementa una interfaccia **Listener**)

2. L'ascoltatore riceve e gestisce l'evento – tipicamente, invocando metodi del burattinaio



L'attività non è più algoritmica (Read-Eval-Print Loop): è *interattiva e reattiva*.

1. Quando si agisce sul componente (ovvero, qui, si preme il pulsante) si genera un evento che è inviato all'ascoltatore



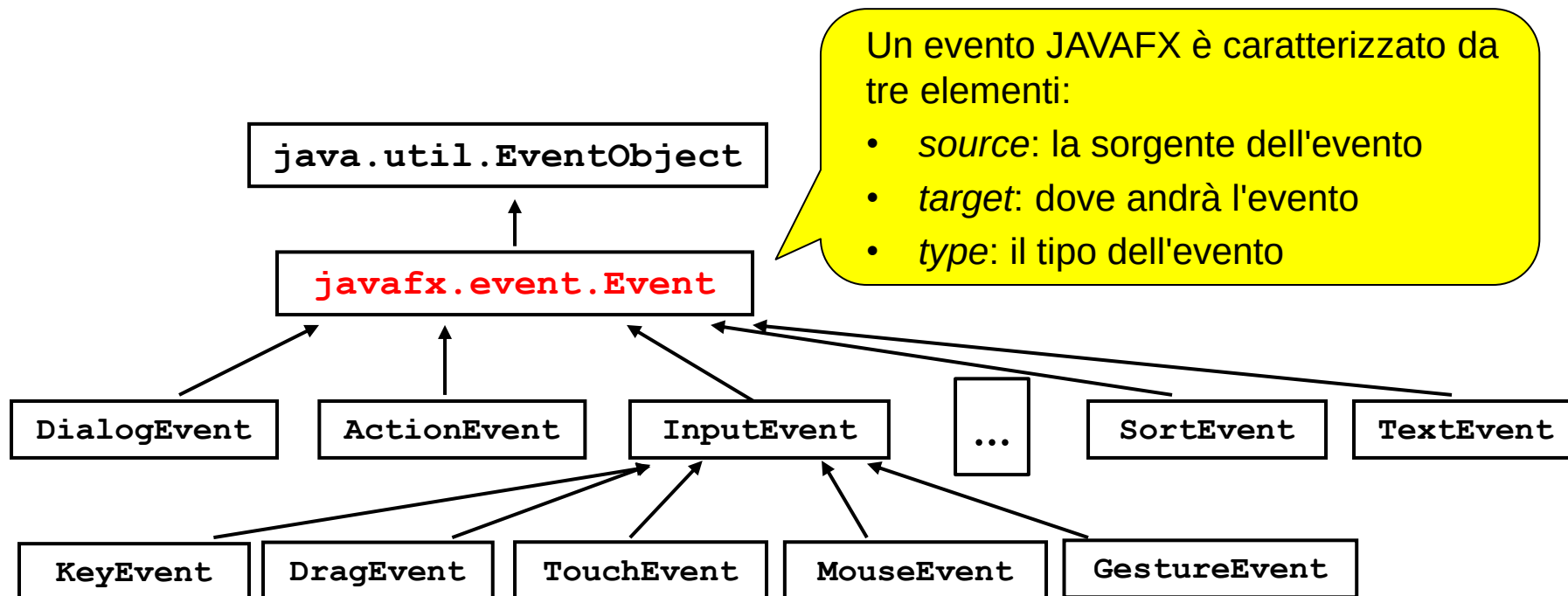
TIPI DI EVENTI

Il **tipo** dell'evento descrive **cosa** è accaduto

- un **bottone** o una **combo** premuti generano l'evento **“azione”**
 - una **lista di voci** genera l'evento **“item selezionato”**
 - un **campo di testo** genera l'evento **“azione”** se viene premuto ENTER sulla tastiera (ma non solo quello..)
- In situazioni diverse, lo stesso widget può generare *eventi diversi*
 - ad esempio, il bottone radio genera tre tipi di eventi: *l'azione, l'item selezionato* e *il cambiamento*
 - si sfrutta il più appropriato al caso concreto.

EVENTI IN JAVAFX

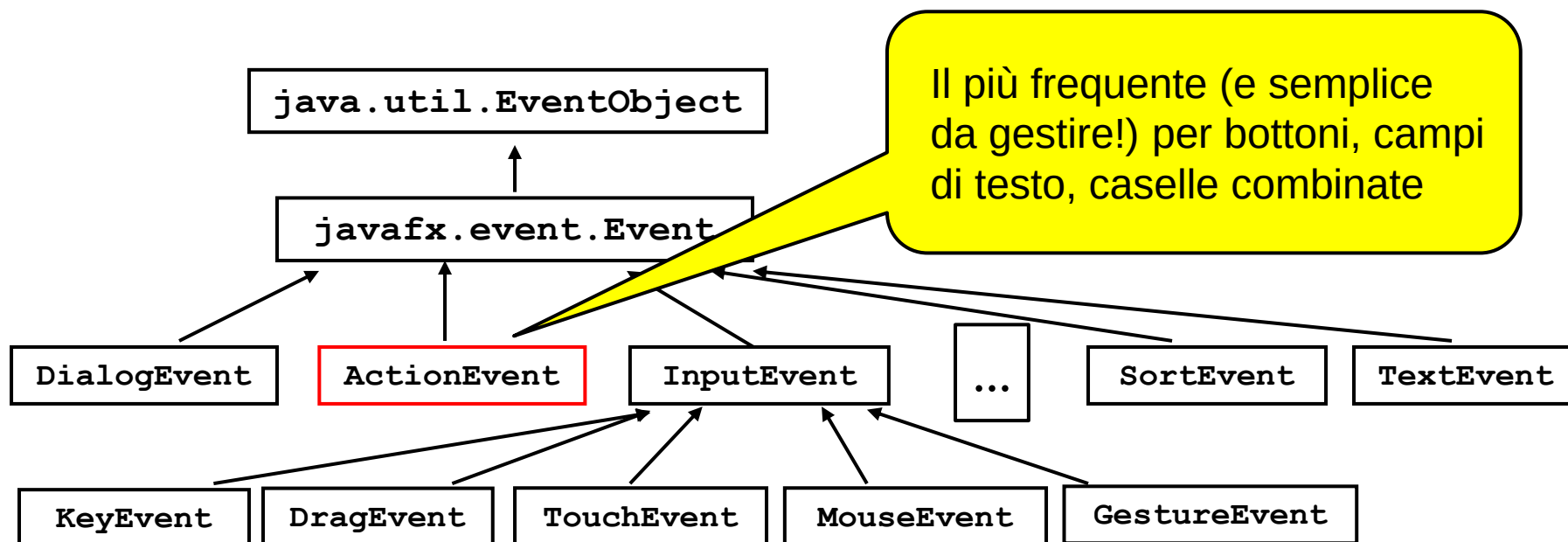
Un *evento* è un oggetto, istanza di una qualche sottoclasse di `java.util.EventObject`



..e molti, molti altri.

EVENTI IN JAVAFX

Un *evento* è un oggetto, istanza di una qualche sottoclasse di `java.util.EventObject`



Gli `InputEvent` sono di basso livello, solitamente da NON gestire direttamente



IL COMPONENTE `Button`

- Un **`Button`** premuto *genera un evento di azione*, ossia un'opportuna istanza di **`ActionEvent`**
 - se nessun ascoltatore è stato preventivamente associato a quel bottone, l'evento andrà perso e non accadrà nulla
- L'evento *viene notificato all'ascoltatore* che era stato in precedenza *associato a quel bottone*
 - l'ascoltatore dev'essere del giusto tipo
 - il tipo è espresso dall'interfaccia **`EventHandler<ActionEvent>`** che dichiara il metodo **`void handle(ActionEvent ev)`**
 - l'ascoltatore può essere un'istanza di una qualunque classe (spesso anonima o lambda) che implementi tale interfaccia



ESEMPIO CON Button

```
public class EsJavaFX08 extends Application {
```

```
    private Label l;
```

```
    public void start(Stage stage) {
```

```
        stage.setTitle("Esempio 8");
```

```
        FlowPane panel = new FlowPane();
```

```
        l = new Label("Tizio");
```

```
        Button b = new Button("Tizio/Caio");
```

```
        b.setOnAction(...);
```

```
        b.setTooltip(new Tooltip("Premere per commutare"));
```

```
        panel.getChildren().addAll(l,b);
```

```
        Scene scene = new Scene(panel,Color.WHITE);
```

```
        stage.setScene(scene);
```

```
        stage.show();
```

```
    }
```

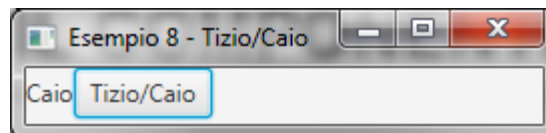
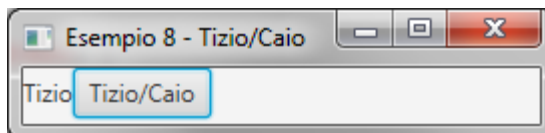
```
}
```

NB: aggiungere le opportune **import**

Estratta da **start** per poterla riusare..

Si aggancia al **Button** il suo ascoltatore

L'ascoltatore implementa il metodo **handle**





ESEMPIO CON Button

```
public class EsJavaFX08 extends Application
    implements EventHandler<ActionEvent>{

    private Label l;
    public void start(Stage stage){
        stage.setTitle("Esempio 8");
        FlowPane panel = new FlowPane();
        l = new Label("Tizio");
        Button b = new Button("Tizio/Caio");
        b.setOnAction(this);
        panel.getChildren().addAll(l,b);
        Scene scene = new Scene(panel,Color.WHITE);
        stage.setScene(scene);
        stage.show();
    }
    public void handle(ActionEvent event) {
        l.setText(l.getText().equals("Tizio") ? "Caio" : "Tizio");
    }
}
```

Qui l'ascoltatore è la classe stessa

Si aggancia al **Button** il suo ascoltatore: *QUI, l'application stessa*

L'ascoltatore, *chiamato automaticamente quando si verifica l'evento*, gestisce la situazione.



VARIANTE: ASCOLTATORE SEPARATO

```
public class EsJavaFX08bis extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 8 bis");  
        FlowPane panel = new FlowPane();  
        Label l = new Label("Tizio");  
        Button b = new Button("Tizio/Caio");  
        b.setOnAction(new Es8bisListener(l));  
        b.setTooltip(new Tooltip("Premere per commutare"));  
        panel.getChildren().addAll(l,b);  
        Scene scene = new Scene(panel,Color.WHITE);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

Aggancio al **Button** il suo ascoltatore

Qui l'ascoltatore è un oggetto di una classe diversa

Ergo, non è più necessario estrarre l'etichetta da `start`

L'ascoltatore implementa il metodo **handle** ma è necessario passargli gli oggetti su cui deve operare (qui, l'etichetta **l**), perché non può più accedere a essi direttamente



VARIANTE: ASCOLTATORE SEPARATO

```
class Es8bisListener implements EventHandler<ActionEvent> {
```

```
    private Label l;
```

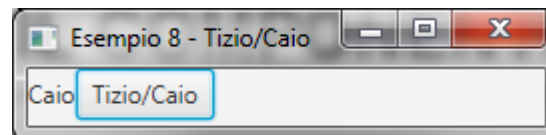
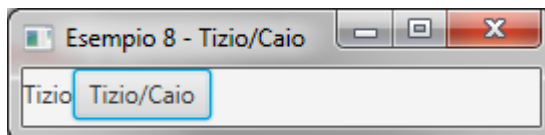
L'ascoltatore separato

```
    public Es8bisListener(Label label) {  
        l=label;  
    }
```

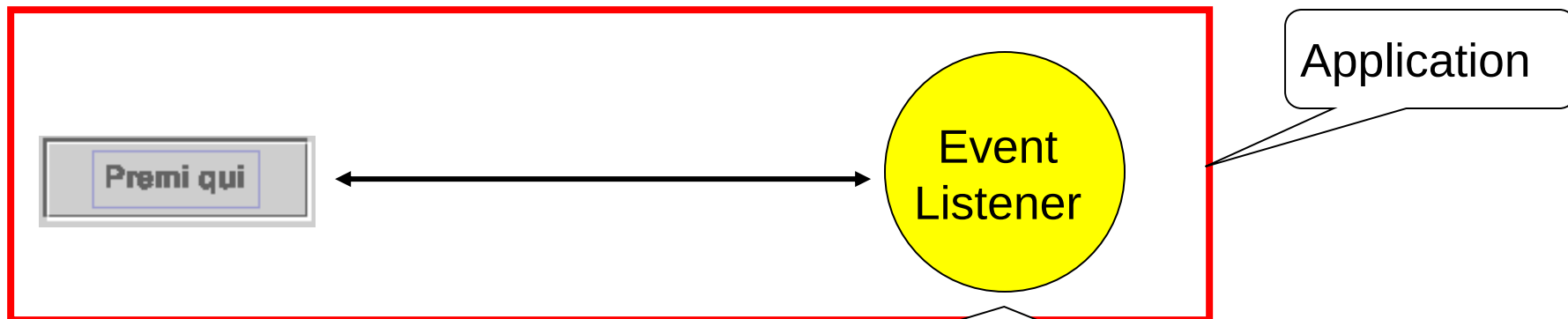
Occorre un costruttore che memorizzi il riferimento alla etichetta su cui deve operare

```
    public void handle(ActionEvent event) {  
        l.setText(l.getText().equals("Tizio") ? "Caio" : "Tizio");  
    }  
}
```

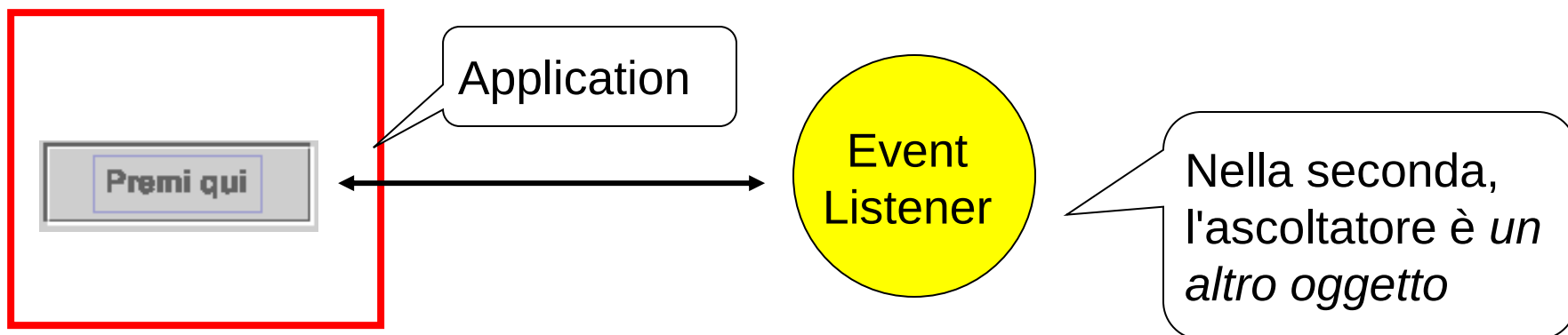
L'ascoltatore implementa il metodo **handle** (identico a prima)



ARCHITETTURE A CONFRONTO



Nella prima versione, l'ascoltatore è *l'application stessa*





ULTERIORE VARIANTE: ASCOLTATORE COME LAMBDA EXPRESSION

L'ascoltatore può essere implementato come *lambda expression* (che sostituisce una classe esterna anonima)

```
public class EsJavaFX08ter extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 8 ter");  
        FlowPane panel = new FlowPane();  
        Label l = new Label("Tizio");  
        Button b = new Button("Tizio/Caio");  
        b.setOnAction(.....);  
        b.setTooltip(new Tooltip("Premere per commutare"));  
        panel.getChildren().addAll(l, b);  
        Scene scene = new Scene(panel, 300, 200);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

- Il metodo **handle** non è più citato direttamente: *diventa anonimo*, riassunto dalla lambda expression
- RICORDA: la lambda è *tutto ciò che resta* del metodo!

ULTERIORE VARIANTE: ASCOLTATORE COME LAMBDA EXPRESSION

L'ascoltatore può essere implementato come *lambda expression*

```
public class F
{
    public void
        stage.set
        FlowPane
        Label l =
        Button b = new Button("Caio");
        b.setOnAction(this::myHandle);
        b.setTooltip(new Tooltip("Premere per c
        panel.getChildren().addAll(l,b);
        Scene scene = new Scene(panel,Color.WHIT
        stage.setScene(scene);
        stage.show();
    }
}
```

- Nel caso più semplice, la gestione può consistere nel **richiamare un metodo ausiliario** (es. **myHandle**)
- La lambda expression assume allora la forma semplificata di **method reference**: **obj::method**
- Se il metodo ausiliario è nella stessa classe, tale forma diviene banalmente **this::myHandle**

NB: l'ascoltatore sembra interno, ma è in realtà **esterno** in quanto istanza di una classe anonima generata dal compilatore.

```
void myHandle(ActionEvent event) {
    l.setText( l.getText().equals("Tizio")
        ? "Caio" : "Tizio");
}
```

ULTERIORE VARIANTE: ASCOLTATORE COME LAMBDA EXPRESSION

Se il codice è breve si può anche fare a meno del metodo ausiliario, implementando la *lambda expression* in loco:

```
public class EsJavaFX08ter extends Application {
```

```
    public void start(Stage stage) throws Exception {
```

```
        stage
```

```
        Flow
```

```
        Label
```

```
        Button b
```

```
        Button("Tizio/Caio");
```

```
        b.setOnAction(
```

```
            event -> l.setText(l.getText().equals("Tizio") ? "Caio" : "Tizio"));
```

```
        b.setTooltip(new Tooltip("Premere per commutare"));
```

```
        panel.getChildren().addAll(l,b);
```

```
        Scene scene = new Scene(panel,Color.WHITESMOKED);
```

```
        stage.setScene(scene);
```

```
        stage.show();
```

```
    }
```

```
}
```

La variabile locale **event** (che può chiamarsi anche diversamente) è l'unica cosa che ricorda che "in origine" c'era un metodo **handle**

L'ascoltatore sembra interno, ma è concettualmente esterno in quanto istanza di una classe anonima generata dal compilatore.

Approccio molto pratico quando un ascoltatore è *breve e serve una volta sola*; altrimenti è poco appropriato.



UN ALTRO ESEMPIO

APPLICAZIONE "ROSSO/BLU"

```
public class EsJavaFX09bis extends Application {
```

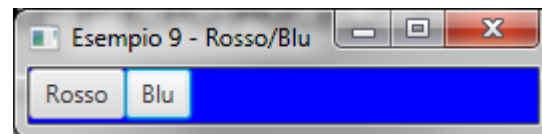
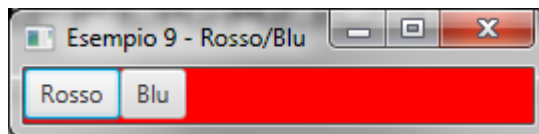
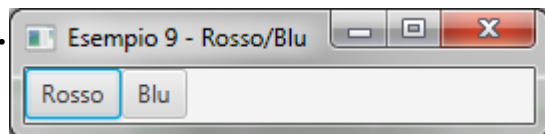
```
    private Button b1, b2;  
    private FlowPane panel;
```

Variabili dichiarate a livello di classe
(non più nel metodo `start`) per
essere visibili anche in `myHandle`

```
    public void start(Stage stage) {  
        stage.setTitle("Esempio 9 - Rosso/Blu");  
        panel = new FlowPane();  
        b1 = new Button("Rosso");  
        b2 = new Button("Azzurro");  
        b1.setOnAction(this::myHandle);  
        b2.setOnAction(this::myHandle);  
        panel.getChildren().addAll(b1,b2);  
        Scene scene = new Scene(panel,Color.WHITE);  
        stage.setScene(scene);  
        stage.show();  
    }
```

Aggancio a ogni Button *lo stesso ascoltatore* nella
forma di *method reference*

Qui va implementato il
metodo ausiliario `myHandle`



APPLICAZIONE ROSSO/BLU

```
public class EsJavaFX09bis extends Application {
```

```
...
```

```
private void myHandle(ActionEvent event) {
```

```
    if (event.getSource() == b1)
```

```
        panel.setStyle("-fx-background: red;");
```

```
    else
```

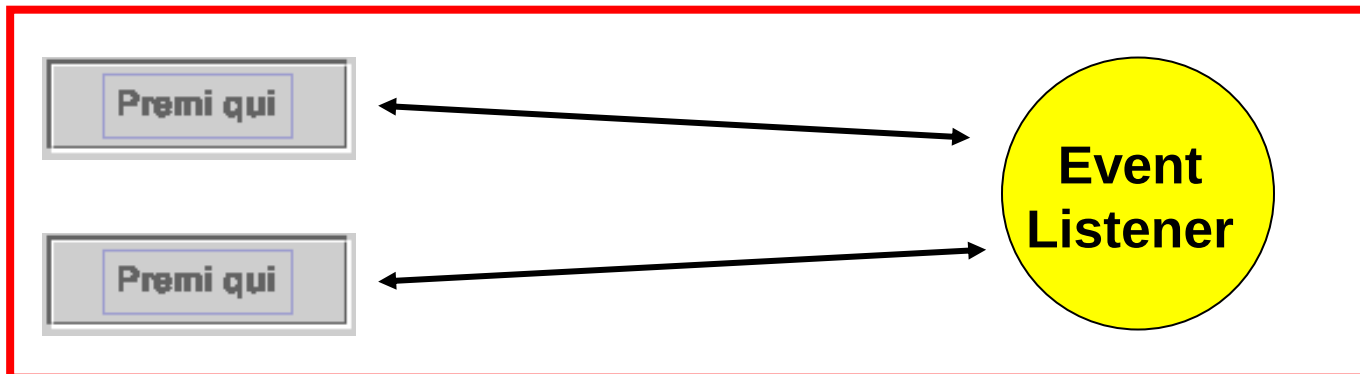
```
        panel.setStyle("-fx-background: blue;");
```

```
}
```

```
}
```

Necessità di discriminare la sorgente dell'evento (MA: spiacevole dover cablare qui il nome della variabile `b1`)

L'ascoltatore imposta lo stile del pannello (tramite codice HTML / CSS)

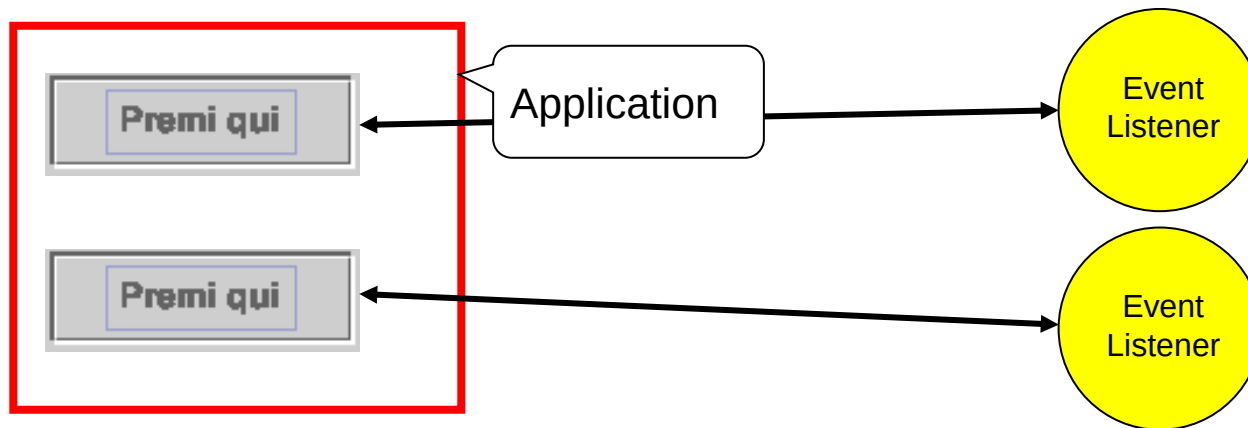


ALTERNATIVA

- Fattorizzare la gestione in un listener unico ha senso se le operazioni da fare sono molte e "praticamente uguali"
 - in questo caso, ci ha solo costretto a inserire un `if` per discriminare la sorgente dell'evento
- In alternativa, si può optare per *due listener separati*, ovvero due lambda distinte
 - scompare la necessità di discriminare quale sia il pulsante premuto
 - più semplice usare le lambda expression, poiché ogni lambda equivale a una *istanza singleton* di una *classe anonima diversa* generata dal compilatore
 - non più necessario il metodo ausiliario

APPLICAZIONE "ROSSO/BLU" CON DUE LISTENER SEPARATI

- Applicazione con *due pulsanti* per cambiare il colore di sfondo: un pulsante lo rende rosso, l'altro blu.
- *Utilizzando le lambda, è pratico predisporre due ascoltatori diversi (uno per ogni pulsante) anche se il compito da svolgere è quasi identico*
→ si evita di dover *discriminare quale sia* il pulsante premuto

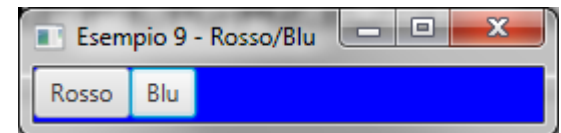
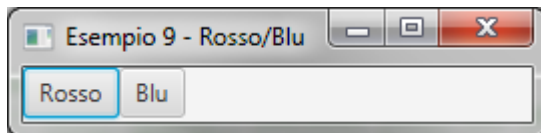


APPLICAZIONE "ROSSO/BLU" CON DUE LISTENER SEPARATI

```
public class EsJavaFX09 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 9 - Rosso/Blu");  
        FlowPane panel = new FlowPane();  
        Button b1 = new Button("Rosso");  
        Button b2 = new Button("Azzurro")  
        b1.setOnAction(event -> panel.setStyle("-fx-background: red;"));  
        b2.setOnAction(event -> panel.setStyle("-fx-background: blue;"));  
        panel.getChildren().addAll(b1,b2);  
        Scene scene = new Scene(panel,Color.WHITE);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

Aggancio a **ogni** Button
il suo ascoltatore

Eventualmente si può fattorizzare
in un metodo ausiliario la parte di
codice che non cambia, passando
il colore come argomento.



UN ESEMPIO CON TextField (passivo)

```
public class EsJavaFX10 extends Application
```

L'unico componente reattivo è sempre il **Button**: i campi di testo sono pilotati da programma (non ci sono ascoltatori su di essi)

```
public void start(Stage stage){
```

```
    stage.setTitle("Esempio 10");
```

```
    FlowPane panel = new FlowPane();
```

```
    TextField txt1 = new TextField("Scrivere qui il nuovo messaggio");
```

```
    txt1.setPrefColumnCount(25);
```

```
    TextField txt2 = new TextField(); txt2.setEditable(false);
```

```
    txt2.setPrefColumnCount(25);
```

```
    Button b1 = new Button("Aggiorna");
```

```
    b1.setOnAction(event -> txt2.setText(txt1.getText()));
```

```
    panel.getChildren().addAll(txt1,txt2,b1);
```

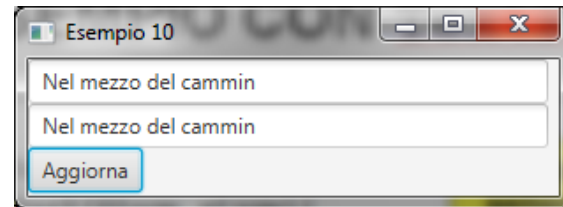
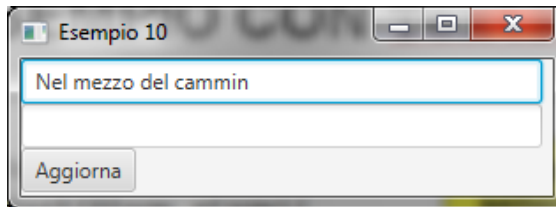
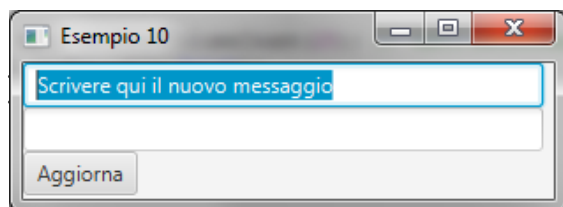
```
    Scene scene = new Scene(panel,Color.WHITE);
```

```
    stage.setScene(scene);
```

Solo output

Premendo il pulsante, il testo sopra è copiato sotto.

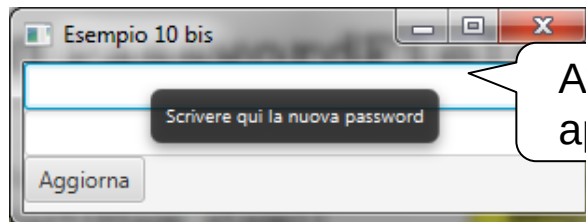
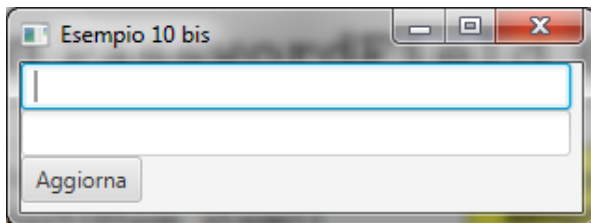
```
}
```



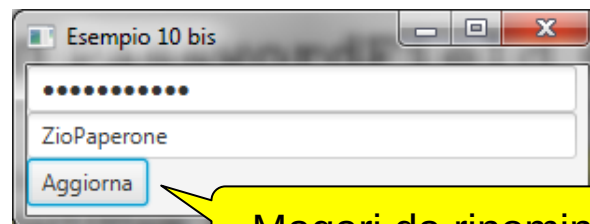
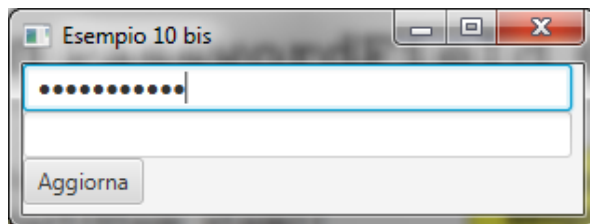
VARIANTE con PasswordField

```
public class EsJavaFX10bis extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 10 bis");  
        FlowPane panel = new FlowPane();  
  
        PasswordField txt1 = new PasswordField(); txt1.setPrefColumnCount(25);  
        txt1.setTooltip(new Tooltip("Scrivere qui la nuova password"));  
  
        ...  
    }  
}
```

Tooltip



Avvicinando il mouse,
appare il tooltip



Magari da rinominare «Svela»

VARIANTE con TextArea

...

```
TextArea area = new TextArea();
```

```
Label label = new Label("Caratteri: 0");
```

```
Button b1 = new Button("Aggiorna");
```

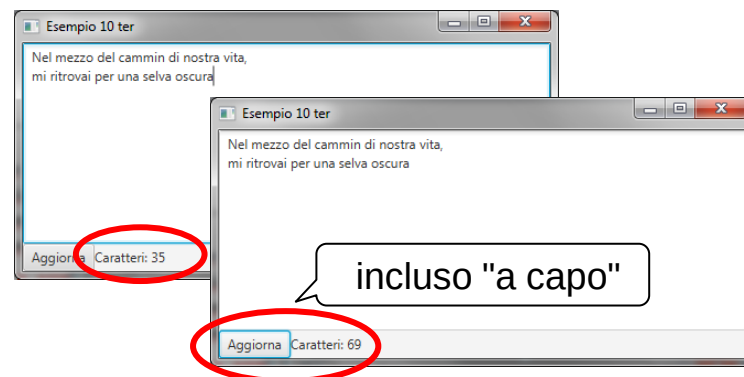
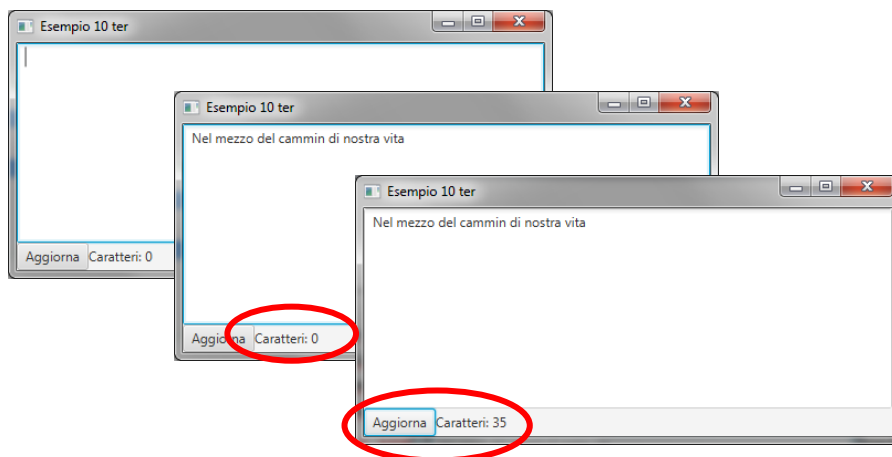
```
b1.setOnAction(
```

```
event -> label.setText("Caratteri: " + area.getText().length()));
```

...

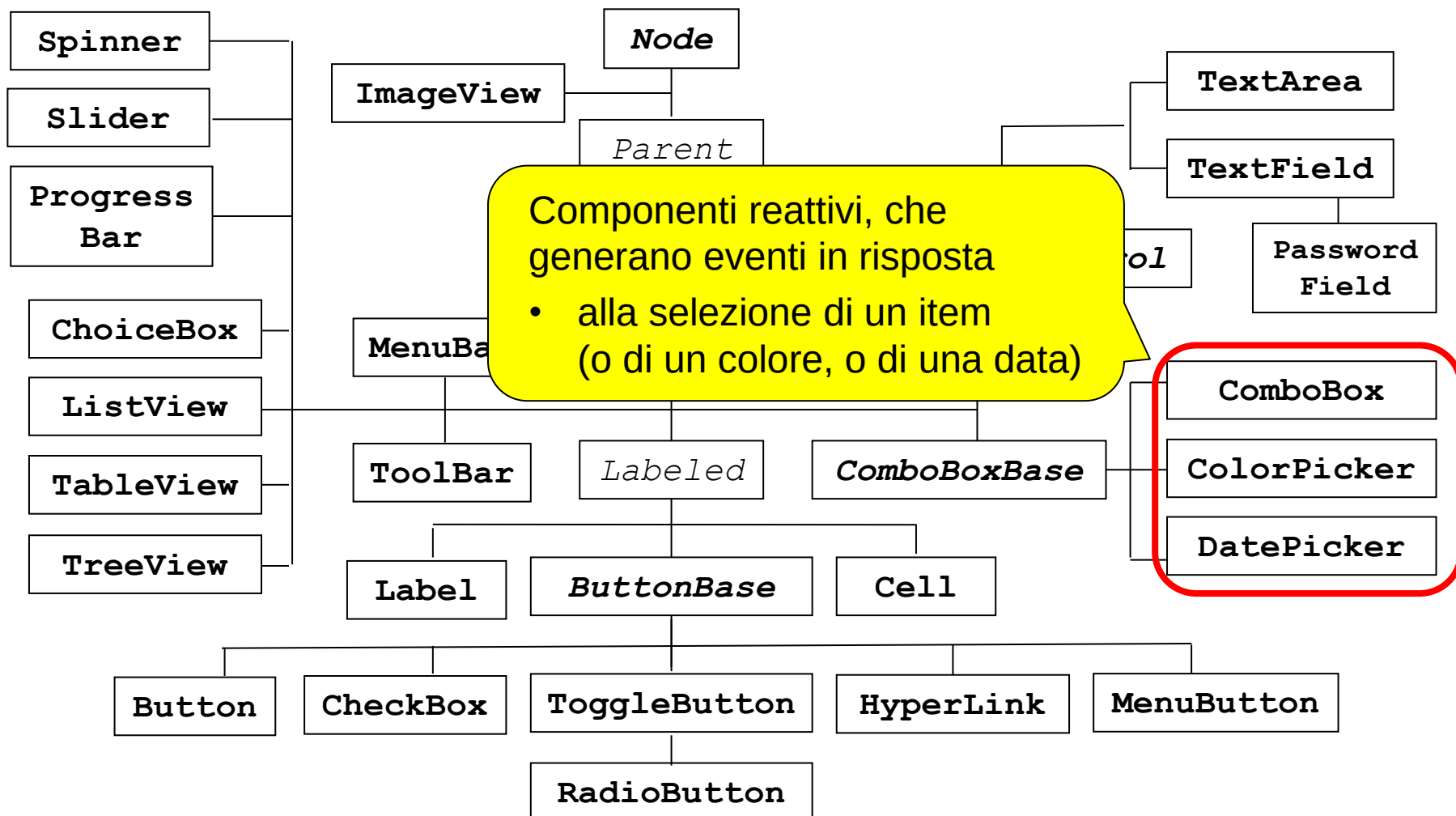
Label anziché campo di testo non modificabile

La label mostra il numero di caratteri digitati, *ma lo aggiorna solo solo quando si preme il pulsante* 😊



GERARCHIA DEI CONTROLLI

`javafx.scene.control`

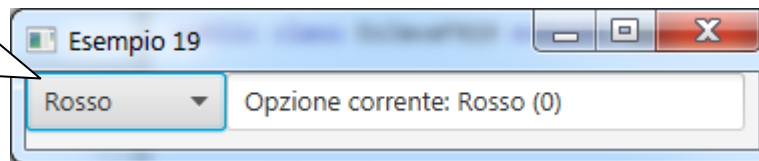


IL COMPONENTE ComboBox

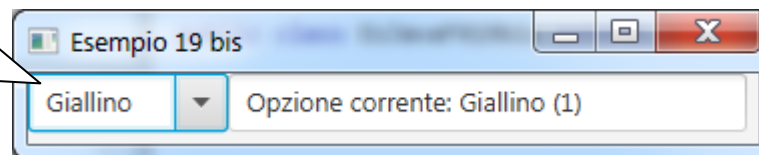
Una **ComboBox** presenta una lista di scelte a discesa

- uso tipico: scegliere elementi da una lista pop-up prestabilita
- unisce le funzionalità di `ChoiceBox` e `TextField`
- se editabile, permette anche all'utente di scrivere un valore *diverso* da quelli proposti nella lista prestabilita ($\sim$ `TextField`)
- come tutti i figli di `ComboBoxBase`, *genera un evento di azione* (come un bottone) quando viene «premuto», ossia *quando si sceglie un elemento* → molto semplice e pratica da gestire

ComboBox non editabile:
funziona come una
ChoiceBox



ComboBox editabile: si
possono scrivere valori
non previsti in elenco





IL COMPONENTE ComboBox

```
public class EsJavaFX19 extends Application {  
    private ComboBox<String> cb;  
    private TextField txt1;  
  
    public void start(Stage stage){  
        stage.setTitle("Esempio 19");  
        FlowPane panel = new FlowPane();  
        cb = new ComboBox<>(); cb.setPrefWidth(100);  
        cb.setItems(  
            FXCollections.observableArrayList("Rosso", "Giallo", "Verde", "Blu");  
        panel.getChildren().add(cb, txt1);  
        cb.setOnAction(  
            event -> txt1.setText("Opzione corrente: " + cb.getValue()  
                + " (" + cb.getSelectionModel().getSelectedIndex() + ")")  
        );  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

Tipizzata

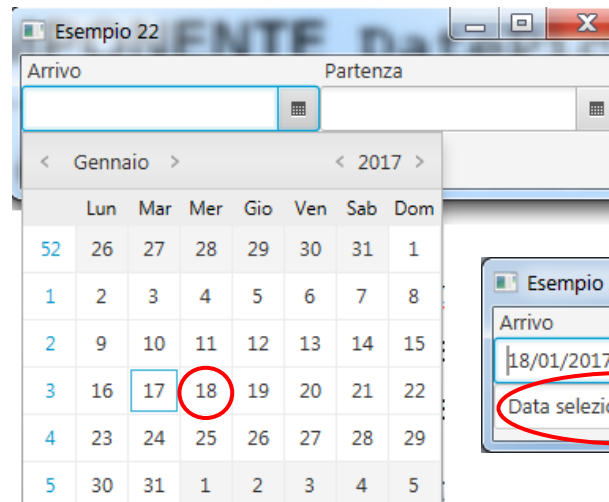
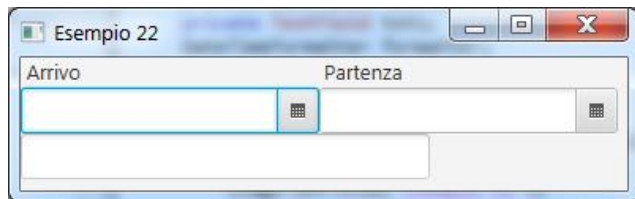
Si popola tramite il metodo
factory
FXCollections.observableList

getValue recupera l'elemento
selezionato
Per recuperare l'indice occorre
invece rivolgersi al model.

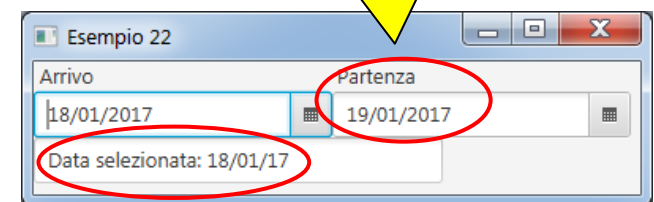
IL COMPONENTE DatePicker

Un **DatePicker** consente di scegliere una data dal classico calendarietto pop-up

- il formato dipende dal `Locale` e dal `Formatter` impostati
- di default, si posiziona sulla data corrente
- ma è possibile forzarlo su una data diversa (esempio: arrivo/partenza in hotel)



Data di partenza
impostata al
giorno successivo





IL COMPONENTE DatePicker

```
public class EsJavaFX22 extends Application {  
    private DatePicker picker, picker2;  
    private DateTimeFormatter formatter;  
    private TextField txt1;  
  
    public void start(Stage stage){  
        stage.setTitle("Esempio 22");  
        FlowPane panel = new FlowPane();  
        picker = new DatePicker();  
        picker2 = new DatePicker();  
        picker.setOnAction(  
            event -> {  
                java.time.LocalDate date = (java.time.LocalDate) (picker.getValue());  
                txt1.setText("Data selezionata: " + formatter.format(date) );  
                picker2.setValue( picker.getValue().plusDays(1) );  
            }  
        );  
        ...  
    }  
}
```

Vale il Locale corrente

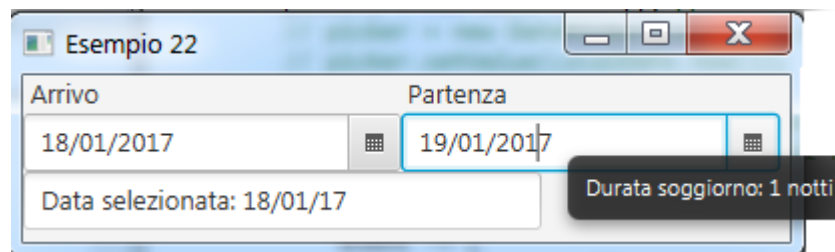
Recupero valore (data) corrente

Forzatura data corrente nel secondo picker

IL COMPONENTE DatePicker

```
...  
picker2.setOnAction(  
    event -> {  
        if (picker.getValue()==null) return;  
        long p = java.time.temporal.ChronoUnit.DAYS.between(  
            picker.getValue(), picker2.getValue());  
        picker2.setToolTipText(new Tooltip("Durata soggiorno: " + p + " notti"));  
    }  
);  
Scene scene = new Scene(panel);  
stage.setScene(scene);  
stage.show();  
}  
}
```

Quando si sceglie la seconda data,
si calcola la durata del soggiorno





IL COMPONENTE ColorPicker

```
public class EsJavaFX23 extends Application {
```

```
    private ColorPicker picker;  
    private TextField txt1,txt2,txt3;
```

ColorPicker è analogo, ma serve per scegliere colori

```
    public void start(Stage stage){
```

```
        stage.setTitle("Esempio 23");
```

```
        FlowPane panel = new FlowPane();
```

```
        txt1 = new TextField(); txt1.setEditable(false);
```

```
        txt2 = new TextField(); txt2.setEditable(false);
```

```
        txt3 = new TextField(); txt3.setEditable(false);
```

```
        picker = new ColorPicker();
```

```
        picker.setOnAction(
```

```
            event -> {
```

```
                txt1.setText("Rosso: " + picker.getValue().getRed());
```

```
                txt2.setText("Verde: " + picker.getValue().getGreen());
```

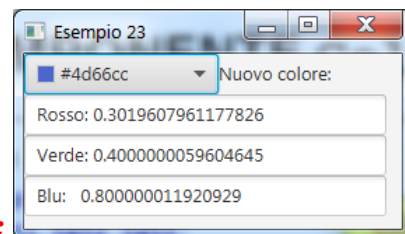
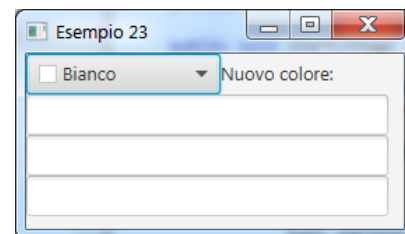
```
                txt3.setText("Blu:   " + picker.getValue().getBlue());
```

```
            }
```

```
        );
```

```
        ...
```

```
    }
```

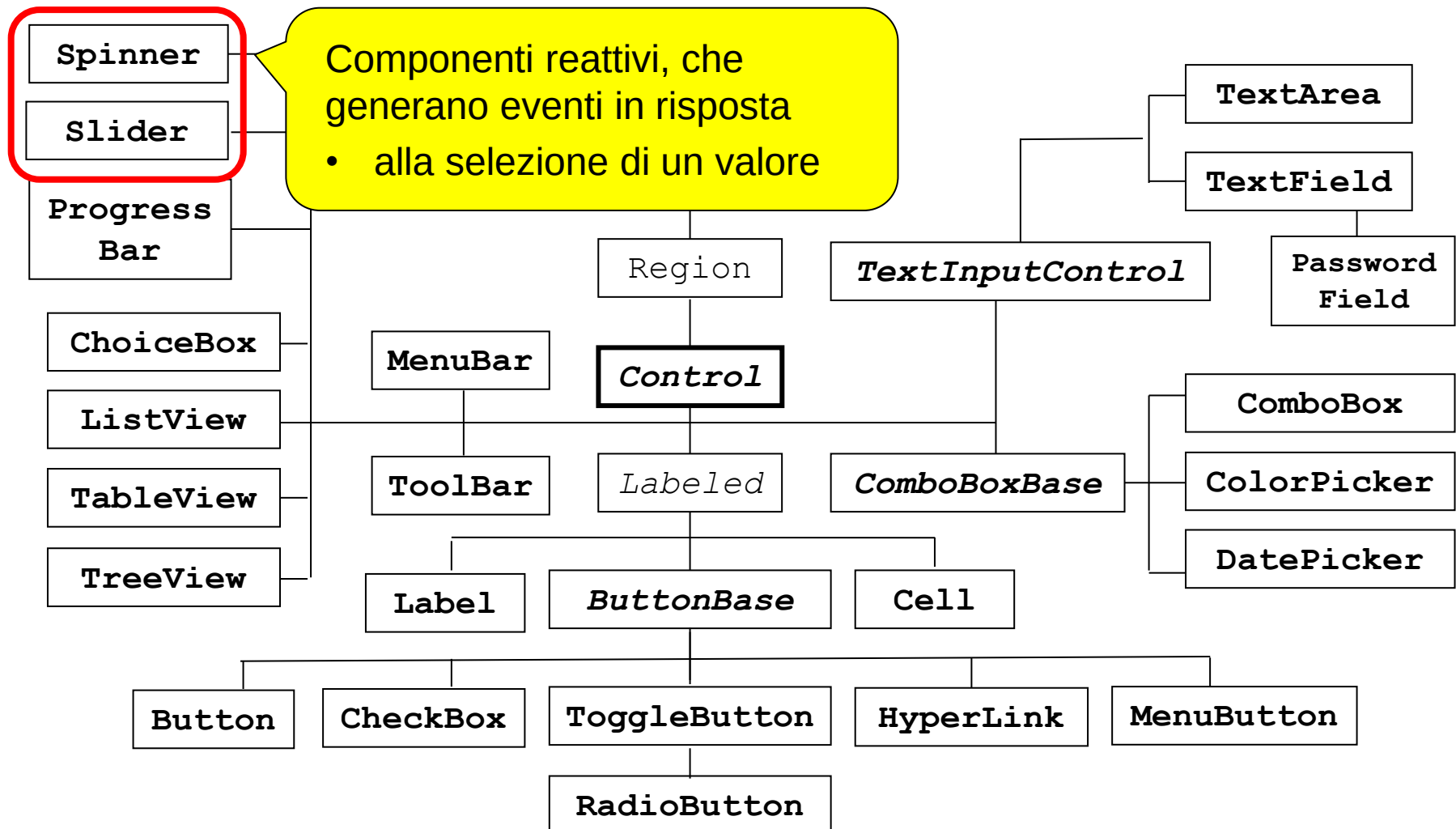


Recupero colore corrente

Recupero valori RGB

GERARCHIA DEI CONTROLLI

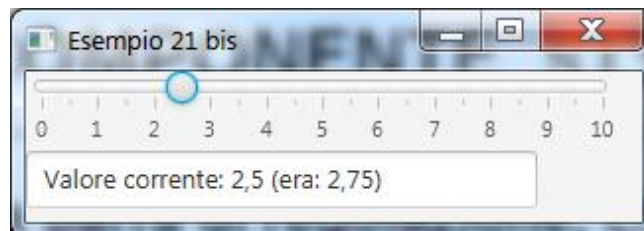
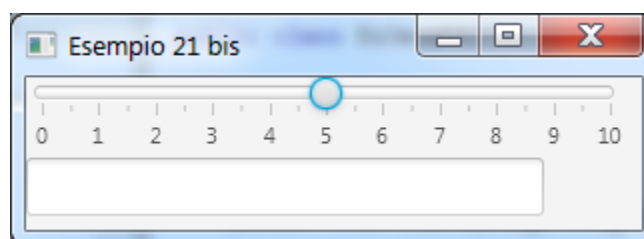
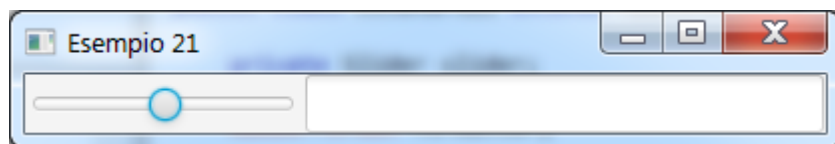
`javafx.scene.control`



IL COMPONENTE Slider

Lo **Slider** è una barra di regolazione, che consente di scegliere un valore muovendo un cursore su una scala

- sono configurabili sia i limiti min/max, sia la graduazione della scala
- graduazione distingue fra *major tick units* e *minor tick units*
- orientamento orizzontale (default) o verticale
- eventuale listener sulla proprietà osservabile **valueProperty**



IL COMPONENTE Slider

```
public class EsJavaFX21 extends Application {  
    private Slider slider;  
    private TextField txt1;  
    private NumberFormat formatter;  
    public void start(Stage stage){  
        formatter = NumberFormat.getInstance();  
        formatter.setMaximumFractionDigits(2);  
        stage.setTitle("Esempio 21");  
        FlowPane panel = new FlowPane();  
        txt1 = new TextField(); txt1.setEditable(false);  
        slider = new Slider(0,10,5); // min, max, initvalue  
        slider.valueProperty().addListener( (changed, oldval, newval) ->  
            txt1.setText("Valore corrente: " + formatter.format(newval)  
                + " (era: " + formatter.format(oldval) + ")")  
        );  
        panel.getChildren().add(slider,txt1);  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
    }  
}
```



Versione base, senza scala graduata

Argomenti del costruttore:

- valore inizio scala
- valore fine scala
- posizione iniziale cursore

Slider: SCALA GRADUATA

Configurabilità della scala

Major tick

- mostrare, con etichette, ogni 1 unità

Minor tick

- mostrare ogni 1 unità

Spostamento cursore

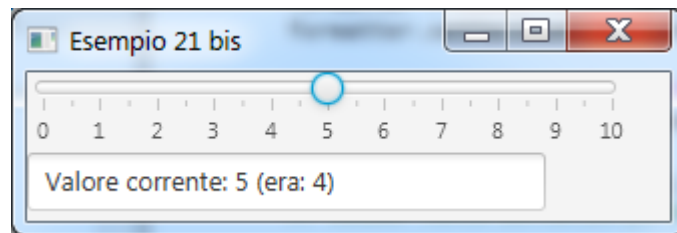
- in caso di movimento tramite tasti freccia: muovere a blocchi di 1 unità
- forza il cursore a stare sui tick

```
slider.setShowTickMarks(true);  
slider.setShowTickLabels(true);  
slider.setMajorTickUnit(1);
```

```
slider.setMinorTickCount(1);
```

```
slider.setBlockIncrement(1);
```

```
slider.setSnapToTicks(true);
```



Orientamento (eventuale)

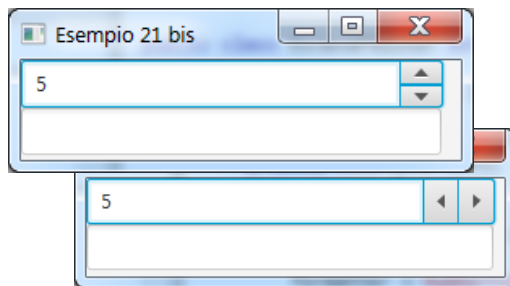
```
slider.setOrientation(  
    Orientation.VERTICAL);
```



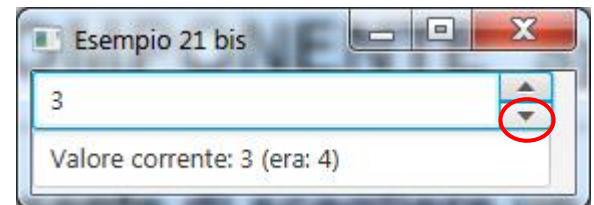
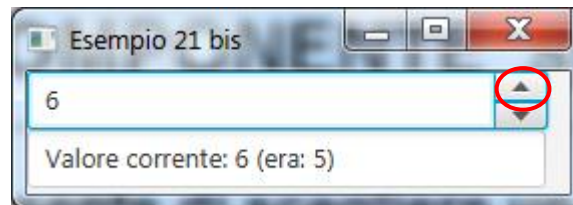
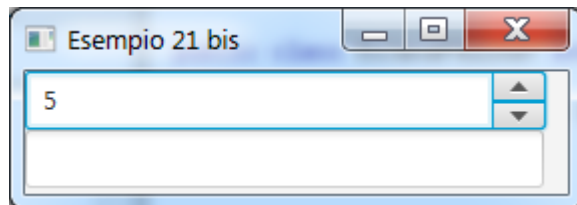
IL COMPONENTE **Spinner**

Lo **Spinner** consente di scegliere un valore muovendo freccine su/giù (o sx/dx), entro limiti predeterminati

- i valori possono essere interi, reali, liste o user-defined
- necessaria factory per produrre i valori (automatica per numeri e liste)
- eventuale listener sulla proprietà osservabile **valueProperty**



```
Spinner.STYLE_CLASS_ARROWS_ON_LEFT_HORIZONTAL  
Spinner.STYLE_CLASS_ARROWS_ON_LEFT_VERTICAL  
Spinner.STYLE_CLASS_ARROWS_ON_RIGHT_HORIZONTAL  
Spinner.STYLE_CLASS_ARROWS_ON_RIGHT_VERTICAL (default)  
Spinner.STYLE_CLASS_SPLIT_ARROWS_HORIZONTAL  
Spinner.STYLE_CLASS_SPLIT_ARROWS_VERTICAL
```





IL COMPONENTE Spinner

```
public class EsJavaFX21ter extends Application {  
    private Spinner<Integer> spinner;  
    private TextField txt1;  
    private NumberFormat formatter;  
    public void start(Stage stage){  
        formatter = NumberFormat.getInstance();  
        formatter.setMaximumFractionDigits(2);  
        stage.setTitle("Esempio 21 ter");  
        FlowPane panel = new FlowPane();  
        txt1 = new TextField(); txt1.setEditable(false);  
        spinner = new Spinner(0,10,5);  
        spinner.valueProperty().addListener( (changed, oldval, newval) ->  
            txt1.setText("Valore corrente: " + formatter.format(newval)  
                + " + formatter.format(oldval) + ")")  
        );  
        panel.getChildren().add(txt1);  
        Scene scene = new Scene(panel, 300, 100);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

Tipizzato

Argomenti come in Slider

Eventualmente:

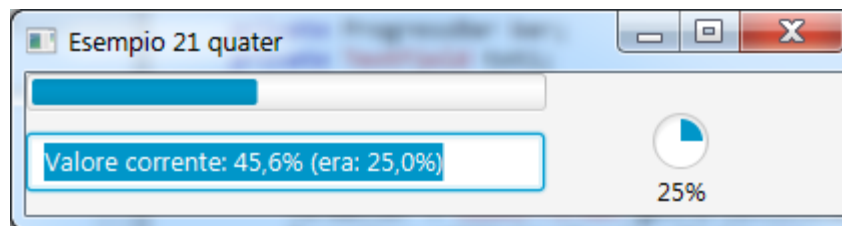
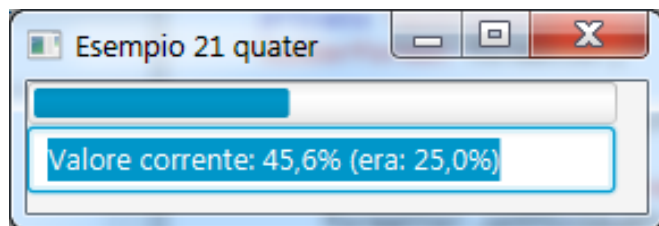
```
spinner.getStyleClass().add(  
    Spinner.STYLE_CLASS_ARROWS_ON_RIGHT_HORIZONTAL);
```

I COMPONENTI ProgressBar E ProgressIndicator

La **ProgressBar** indica a che punto si è arrivati (%)

- è per sua natura principalmente un dispositivo di output
- eventuale listener sulla proprietà osservabile **progressProperty**

Il **ProgressIndicator** fa lo stesso in un grafico a torta



```
ProgressBar bar = new ProgressBar(0.25);
ProgressIndicator pid = new ProgressIndicator(0.25);
bar.progressProperty().addListener( (changed, oldval, newval) ->
    txt1.setText("Valore corrente: " + formatter.format(newval)
        + " (era: " + formatter.format(oldval) + ")")
);
bar.setProgress(0.456);
```

Scatena aggiornamento proprietà

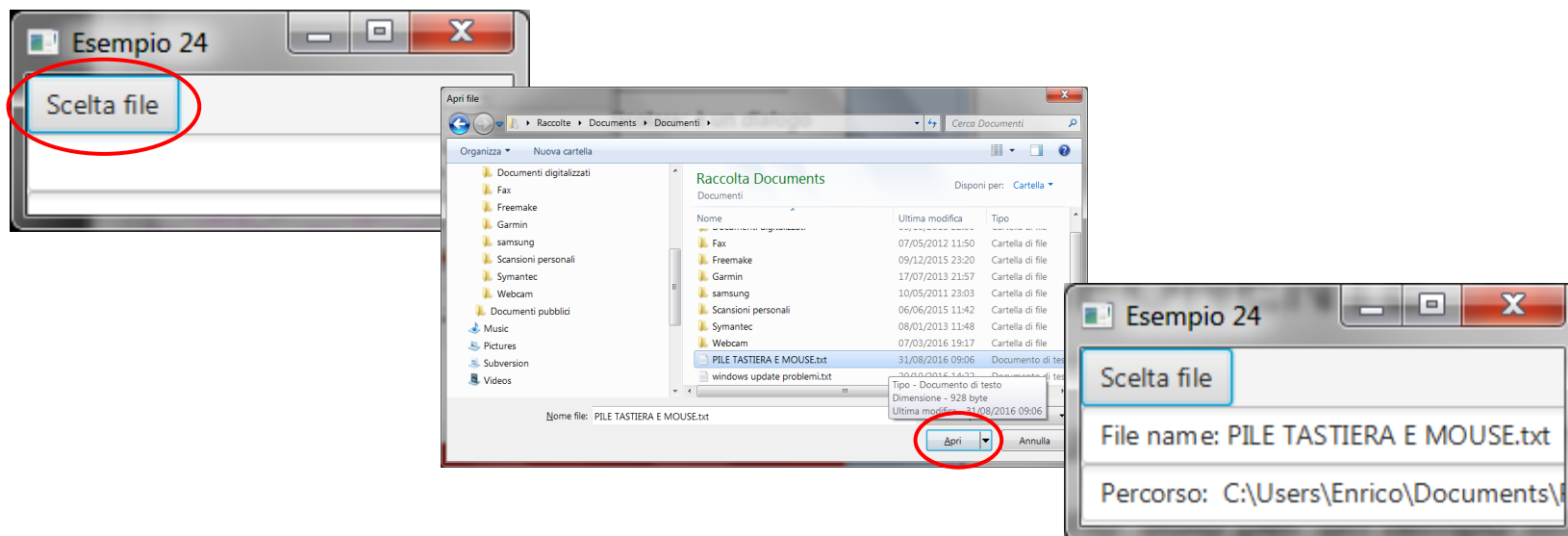
Formattatore percentuali

Un componente particolare: il dialogo FileChooser

IL COMPONENTE FileChooser

FileChooser è un componente particolare: è un *dialogo*

- i dialoghi *compaiono solo per un tempo limitato*, per supportare lo svolgimento di una data azione (non restano visibili stabilmente)
- **FileChooser** si usa per aprire/salvare un file
- solitamente lo si fa comparire in seguito alla pressione di un bottone





IL COMPONENTE FileChooser

```
public class EsJavaFX24 extends Application {  
    private FileChooser chooser;  
    private File selectedFile;  
    private Button button;  
    private TextField txt1, txt2;  
    public void start(Stage stage){  
        stage.setTitle("Esempio 24 ter");  
        FlowPane panel = new FlowPane();  
        button = new Button("Scelta file");  
        button.setOnAction( event -> {  
            chooser = new FileChooser();  
            chooser.setTitle("Apri file");  
            selectedFile = chooser.showOpenDialog(stage); // OPPURE showSaveDialog  
            txt1.setText("File name: " + selectedFile.getName());  
            txt2.setText("Percorso:  " + selectedFile.getPath());  
        }  
    );  
    ...  
}
```

Si può scegliere fra dialogo di
APERTURA o SALVATAGGIO file



IL COMPONENTE FileChooser

Si possono filtrare solo certi tipi di file:

```
chooser = new FileChooser();
chooser.setTitle("Apri file");

chooser.getExtensionFilters().addAll(
    new ExtensionFilter("Text Files", "*.txt"),
    new ExtensionFilter("Image Files", "*.png", "*.jpg", "*.gif"),
    new ExtensionFilter("Audio Files", "*.wav", "*.mp3", "*.aac"),
    new ExtensionFilter("All Files", "*.*"));
```

Si possono accettare selezioni multiple di file:

```
// selezione singola:
File selectedFile = chooser.showOpenDialog(stage);

// selezione multipla:
List<File> selectedFiles = chooser.showOpenMultipleDialog(stage);
```

IL COMPONENTE `FileChooser`

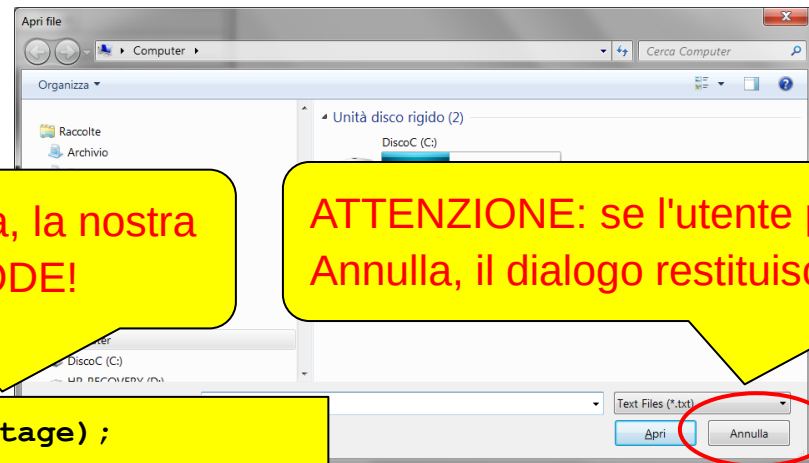
Completamento dell'esempio

```
public class EsJavaFX24 extends Application {  
  
    ...  
  
    txt1 = new TextField(); txt1.setEditable(false);  
    txt2 = new TextField(); txt2.setEditable(false);  
    panel.getChildren().addAll(button, txt1, txt2);  
    stage.setScene(new Scene(panel));  
    stage.setMaxWidth(270);  
    stage.setMinHeight(115);  
    stage.show();  
}  
}
```

..e per come è fatta ora, la nostra gestione eventi ESPLODE!

ATTENZIONE: se l'utente preme Annulla, il dialogo restituisce *null*

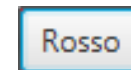
```
selectedFile = chooser.showOpenDialog(stage);  
txt1.setText("File name: " + selectedFile.getName());  
txt2.setText("Percorso: " + selectedFile.getPath());
```



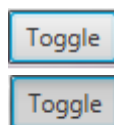
The Toggles family

THE TOGGLES FAMILY

- Il **Button** base è un classico pulsante
 - una sola posizione stabile (quella OFF)
 - attivo (ON) solo finché lo si mantiene premuto
 - appena lo si rilascia, torna nella posizione di riposo
- I **Toggle** sono invece **interruttori a due posizioni**
 - premendoli, si attivano e commutano nella posizione ON
 - per riportarli nella posizione OFF occorre ripremerli
 - ne sono disponibili diversi, per varie esigenze



Toggle



CheckBox



RadioButton

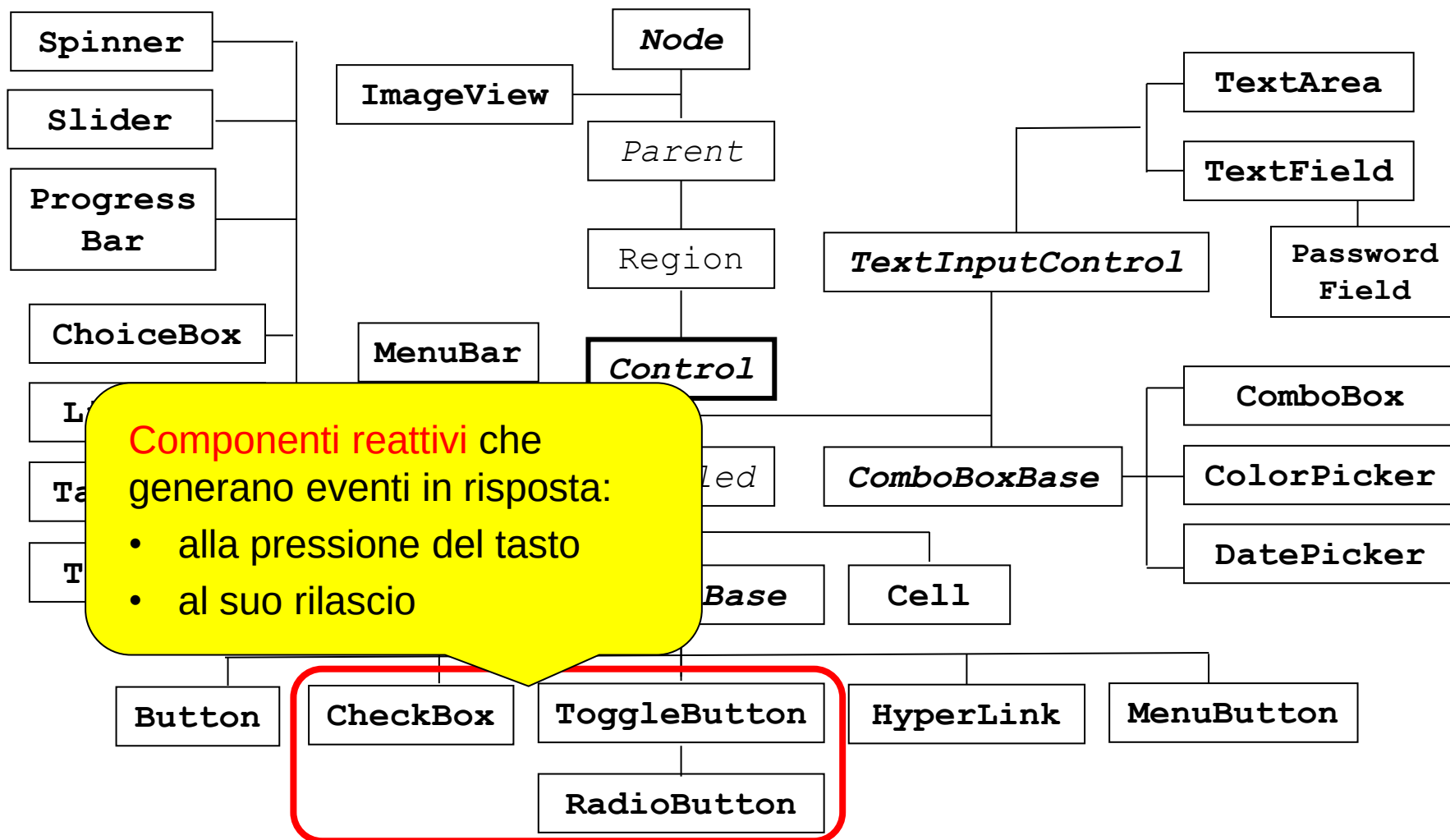


Usati singolarmente (per opzioni *non* mutuamente esclusive)

Solitamente usati a gruppi (opzioni mutuamente esclusive)

GERARCHIA DEI CONTROLLI

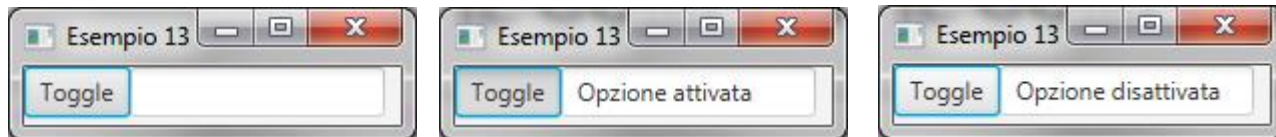
javafx.scene.control



IL COMPONENTE Toggle

```
public class EsJavaFX13bis extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 13bis");  
        FlowPane panel = new FlowPane();  
        ToggleButton tgl = new ToggleButton("Toggle");  
        TextField txt2 = new TextField(); txt2.setEditable(false);  
        tgl.setOnAction( event -> txt2.setText(  
            tgl.isSelected() ? "Opzione attivata" : "Opzione disattivata" ) );  
        panel.getChildren().add(tgl,txt2);  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

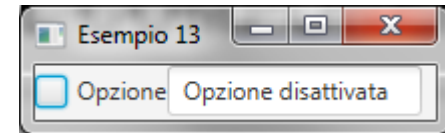
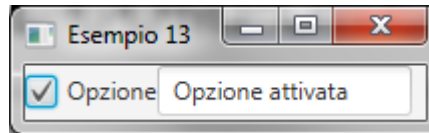
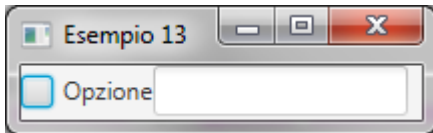
Solito evento di azione,
stesso listener del Button



IL COMPONENTE CheckBox

```
public class EsJavaFX13 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 13");  
        FlowPane panel = new FlowPane();  
        CheckBox ck1 = new CheckBox("Opzione");  
        TextField txt2 = new TextField(); txt2.setEditable(false);  
  
        ck1.setOnAction( event -> txt2.setText(  
            ck1.isSelected() ? "Opzione attivata" : "Opzione disattivata" ) );  
        panel.getChildren().add(ck1,txt2);  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

Solito evento di azione,
stesso listener del Button



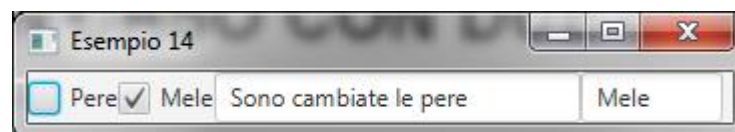
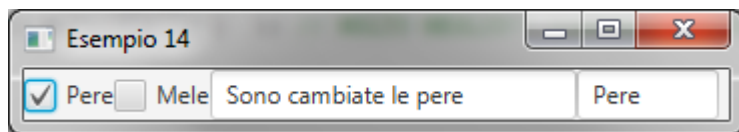
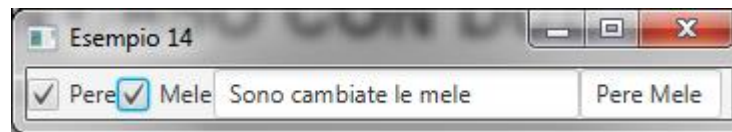
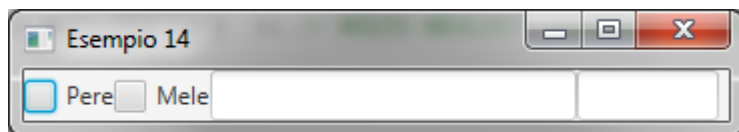
UN CASO CON DUE CheckBox

Obiettivo: gestire due **CheckBox** in modo che:

- un primo **TextField** rifletta l'ultima azione eseguita
- un secondo **TextField** dia invece lo stato complessivo

Gestione eventi:

- un primo **TextField** va gestito per singola checkbox (due listener separati, uno per ciascuna casella)
- il secondo **TextField** va gestito invece collettivamente (stesso listener per entrambe le caselle)





UN CASO CON DUE CheckBox

```
public class EsJavaFX14 extends Application {
```

```
    private CheckBox ck1, ck2;
```

```
    private TextField txt1, txt2;
```

```
    public void start(Stage stage){
```

```
        stage.setTitle("Esempio 14");
```

```
        FlowPane panel = new FlowPane();
```

```
        ck1 = new CheckBox("Pere"); ck2 = new CheckBox("Mele");
```

```
        txt1 = new TextField(); txt1.setEditable(false);
```

```
        txt2 = new TextField(); txt2.setEditable(false);
```

```
        ck1.setOnAction(event -> {
```

```
            txt1.setText( "Sono cambiate le pere" ); myAuxHandler(); } );
```

```
        ck2.setOnAction(event -> {
```

```
            txt1.setText( "Sono cambiate le mele" ); myAuxHandler(); } );
```

```
        panel.getChildren().add(ck1,ck2,txt1,txt2);
```

```
        Scene scene = new Scene(panel);
```

```
        stage.setScene(scene);
```

```
        stage.show();
```

```
    }
```

```
    ...
```

Variabili spostate a livello di classe
per essere visibili anche fuori dal
metodo `start`

Due listener distinti, ma
con una parte comune

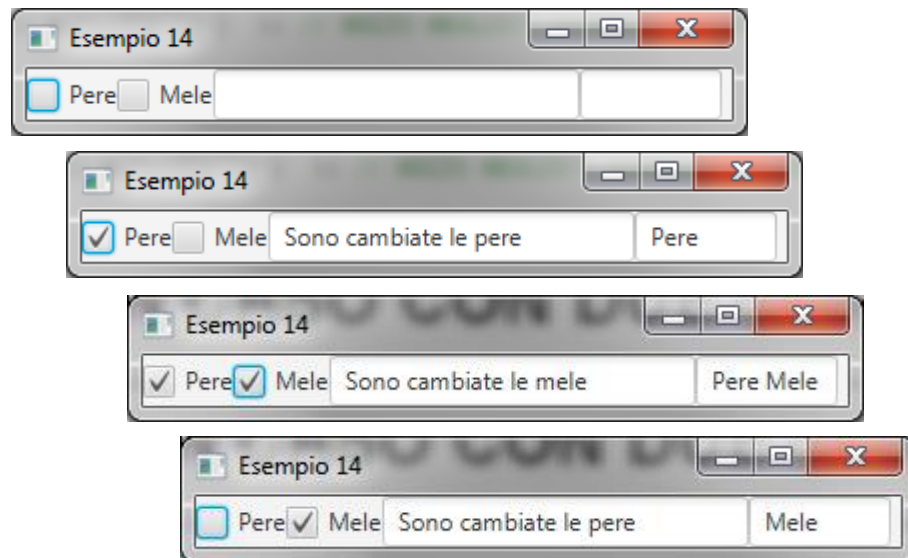
Nostro metodo di utilità
per fattorizzare la parte
comune ai due casi

UN CASO CON DUE CheckBox

...

```
private void myAuxHandler() {  
    String frase = (ck1.isSelected() ? "Pere " : "")  
        + (ck2.isSelected() ? "Mele" : "");  
    txt2.setText(frase);  
}  
}
```

Sintetizza la frase descrittiva dello status globale



Il primo textfield riflette l'ultima azione.

Il secondo riflette lo stato complessivo delle opzioni



UN CASO CON DUE CheckBox

Alternativa: un listener unico

```
ck1 = new CheckBox("Pere"); ck2 = new CheckBox("Mele");
```

```
txt1 = new TextField(); txt1.setEditable(false);
```

```
txt2 = new TextField(); txt2.setEditable(false);
```

```
ck1.setAction(this::myHandle);
```

```
ck2.setAction(this::myHandle);
```

```
...
```

```
}
```

```
private void myHandle(ActionEvent event) {
```

```
    txt1.setText(
```

```
        "Sono cambiate le " + ( event.getSource()==ck1 ? "pere" : "mele" ));
```

```
    String frase = (ck1.isSelected() ? "Pere " : "")
```

```
        + (ck2.isSelected() ? "Mele " : "");
```

```
    txt2.setText(frase);
```

```
}
```

```
}
```

Method reference per fattorizzare la gestione in un unico listener

Occorre ispezionare l'argomento `event` per capire quale checkbox è stata premuta per ultima

Sgradevole dover cablare un confronto con una specifica variabile...

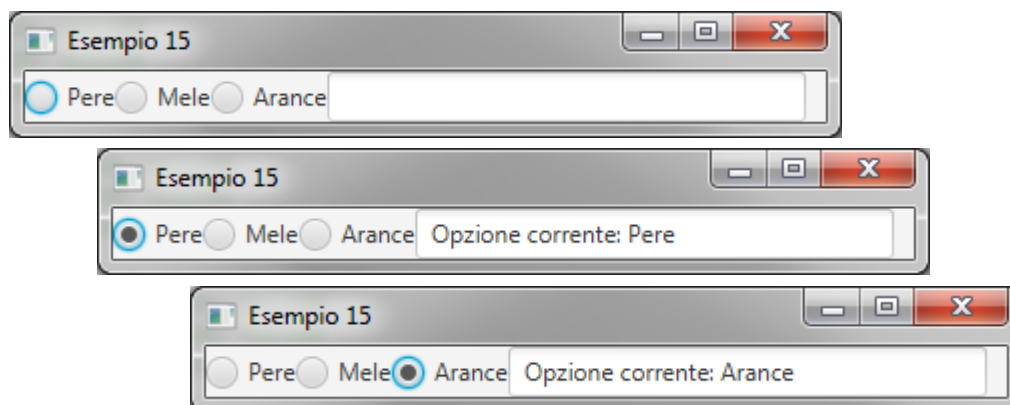
IL COMPONENTE `RadioButton`

A differenza dei precedenti, il `RadioButton` rappresenta un'opzione *mutuamente esclusiva*

- non è mai usato da solo: ce ne sono sempre almeno due
- il concetto di gruppo è catturato da un'istanza di `ToggleGroup`

Gestione eventi:

- si possono gestire i pulsanti singolarmente (scomodo)...
- ..oppure sfruttare il `ToggleGroup` (più comodo)





IL COMPONENTE RadioButton

```
public class EsJavaFX15 extends Application {  
    private RadioButton b1, b2, b3;  
    private ToggleGroup tg;  
    private TextField txt1;  
  
    public void start(Stage stage){  
        stage.setTitle("Esempio 15");  
        FlowPane panel = new FlowPane();  
        tg = new ToggleGroup();  
        b1 = new RadioButton("Pere");    b1.setToggleGroup(tg);  
        b2 = new RadioButton("Mele");    b2.setToggleGroup(tg);  
        b3 = new RadioButton("Arance"); b3.setToggleGroup(tg);  
        txt1 = new TextField(); txt1.setEditable(false);  
        b1.setOnAction(this::myHandle);  
        b2.setOnAction(this::myHandle);  
        b3.setOnAction(this::myHandle);  
        panel.getChildren().add(b1,b2,b3,txt1);  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

Ogni RadioButton va aggiunto anche al suo gruppo

Method reference per fattorizzare la gestione in un unico listener



IL COMPONENTE `RadioButton`

```
private void myHandle(ActionEvent event) {  
    txt1.setText("Opzione corrente: " +  
        ( (RadioButton) event.getSource() ).getText());  
}
```

Occorre ispezionare l'argomento `event` per capire quale radiobutton è stato premuto per ultimo

Necessario il cast a `RadioButton` per poter invocare `getText`

Alternativa: sfruttare (male) il `ToggleGroup`

```
private void myHandle(ActionEvent event) {  
    txt1.setText("Opzione corrente: " +  
        ( (RadioButton) tg.getSelectedToggle() ).getText());  
}
```

Non ispezioniamo più `event`

Necessario il cast a `RadioButton` per poter invocare `getText`



IL COMPONENTE `RadioButton`

Alternativa migliore: sfruttare la proprietà

`selectedToggleProperty` di `ToggleGroup`

- non agganciamo più tanti listener ai singoli pulsanti
- **agganciamo un solo `ChangeListener` alla proprietà**

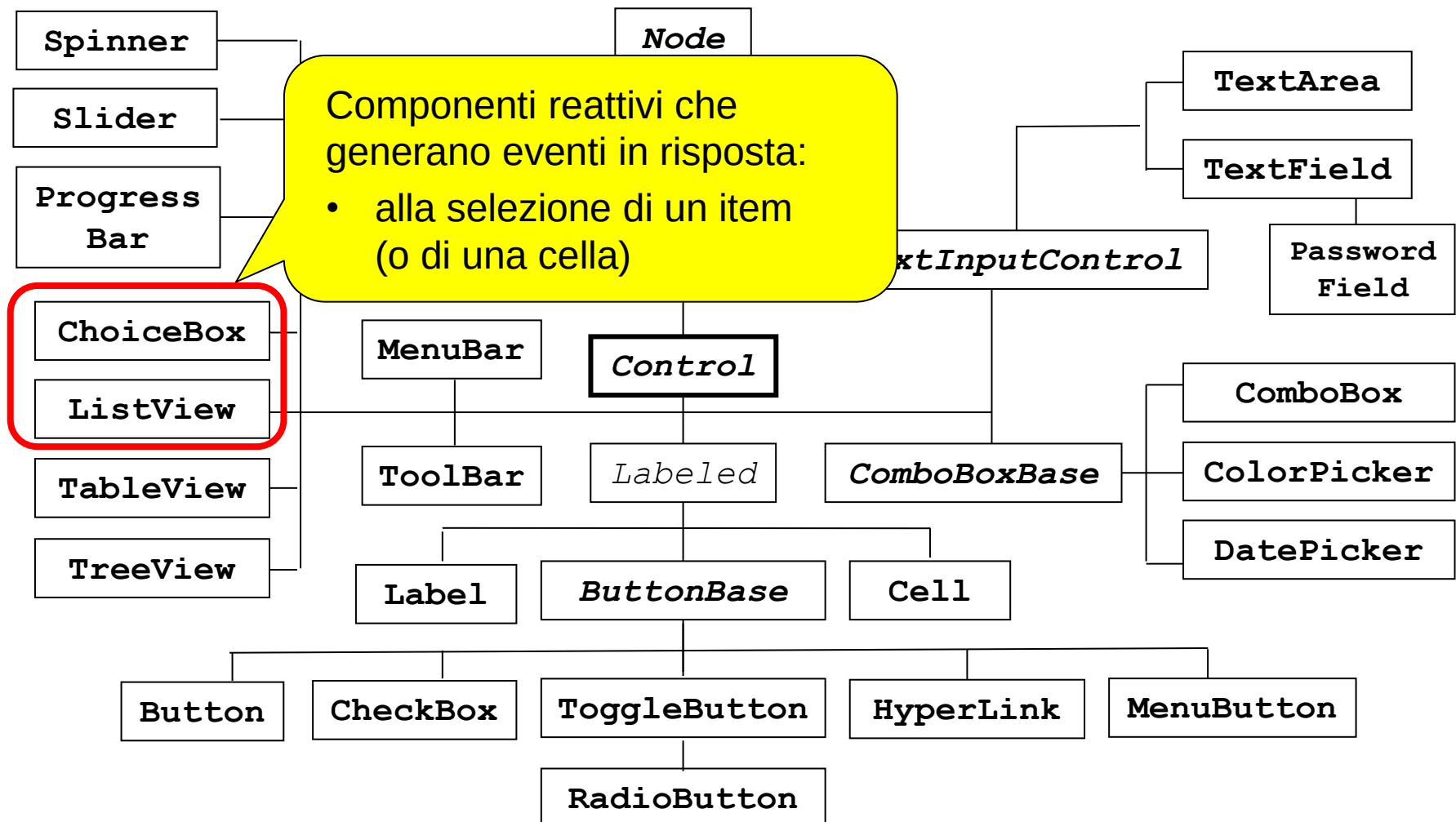
```
...
tg.selectedToggleProperty().addListener(
    (changed, oldval, newval) ->
    txt1.setText("Opzione corrente: " + ((RadioButton)newval).getText())
);
panel.getChildren().add(b1,b2,b3,txt1);
Scene scene = new Scene(panel);
stage.setScene(scene);
stage.show();
}
```

Necessario il cast a `RadioButton`
per poter invocare `getText`

ListView & ChoiceBox

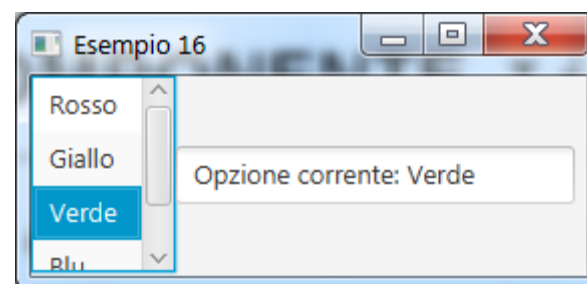
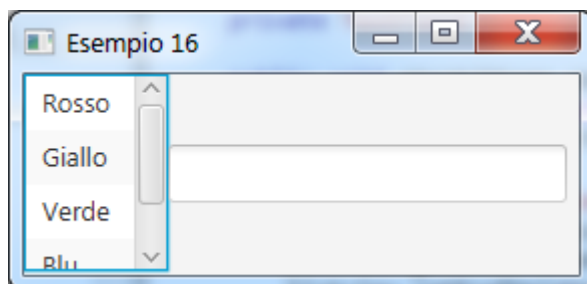
GERARCHIA DEI CONTROLLI

`javafx.scene.control`



IL COMPONENTE `ListView`

- Un **`ListView`** è una lista (con eventuali scrollbar) che permette di scegliere una voce da un elenco
 - di default, è abilitata la selezione singola
 - ma si può abilitare la selezione multipla
- Quando una voce viene scelta, nel model associato viene modificata la proprietà **`selectedItemProperty`**
 - MVVM: se si vuole intercettare l'evento, occorre recuperare la proprietà dal model e agganciarle un **`ChangeListener`**





IL COMPONENTE `ListView`

```
public class EsJavaFX16 extends Application {  
    private ListView<String> listview;  
    private TextField txt1;  
    public void start(Stage stage){  
        stage.setTitle("Esempio 16");  
        FlowPane panel = new FlowPane();  
        listview = new ListView<>();  
        listview.setItems(  
            FXCollections.observableArrayList("Rosso", "Giallo", "Verde", "Blu") );  
        listview.setPrefHeight(listview.getItems().size()*24+ 2);  
        listview.setPrefWidth(72);  
        listview.getSelectionModel().selectedItemProperty().addListener(  
            (obj, oldval, newval) -> txt1.setText("Opzione corrente: " + newval) );  
        txt1 = new TextField(); txt1.setEditable(false);  
        panel.getChildren().add(listview,txt1);  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

È tipizzata: occorre specificare il tipo degli elementi.

Per popolare la `ListView` usare i metodi factory di `FXCollections`

Di default mostra 10 item (brutto) → impostiamo l'altezza

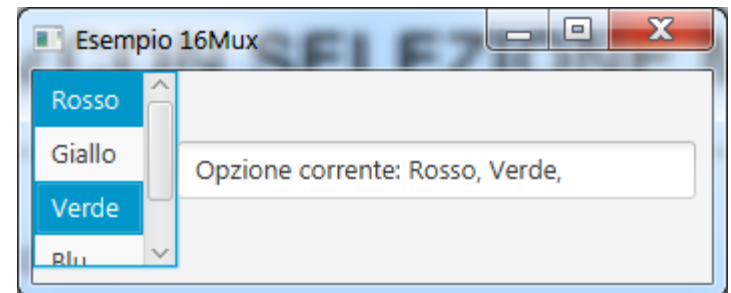
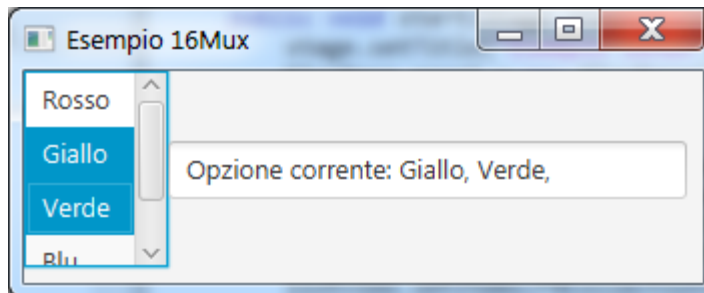
Gestione evento: `ChangeListener` implementato come lambda expression

ListView

CON SELEZIONE MULTIPLA

```
...
listview = new ListView<>();
listview.setItems(
    FXCollections.observableArrayList("Rosso", "Giallo", "Verde", "Blu") );
listview.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);
listview.setPrefHeight(listview.getItems().size()*24+ 2);
listview.setPrefWidth(72);
listview.getSelectionModel().selectedItemProperty().addListener(
    (obj, oldval, newval) -> {
        StringBuilder sb = new StringBuilder();
        for(String item : listview.getSelectionModel().getSelectedItems())
            sb.append(item+"", " ");
        txt1.setText("Opzione corrente: " + sb) ;
    }
);
...
```

Selezione multipla tenendo premuti i tasti SHIFT (item adiacenti) o CTRL



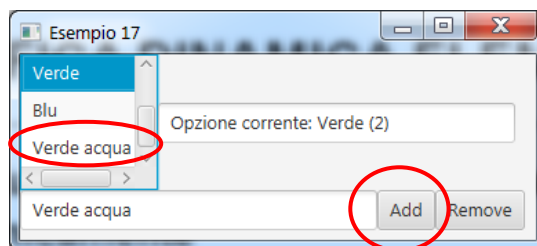
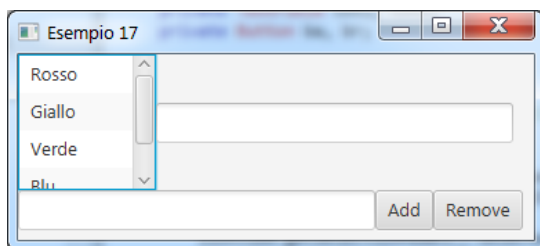
MODIFICA DINAMICA ELEMENTI

È possibile *aggiungere o togliere elementi* da una **ListView** *dinamicamente*

- la lista degli elementi è recuperabile con **getItems ()**
- si possono aggiungere/togliere elementi normalmente, come in una qualsiasi lista

Nell'esempio che segue:

- un campo di testo consente di scrivere il nome dell'elemento
- due bottoni Add/Remove effettuano l'aggiornamento richiesto





MODIFICA LISTA ELEMENTI

```
public class EsJavaFX17 extends Application {  
    private ListView<String> listview;  
    private TextField txt1, txt2;  
    private Button ba, br;  
    public void start(Stage stage){  
        ...  
        txt2 = new TextField(); txt2.setPrefColumnCount(20);  
        ba = new Button("Add");  
        br = new Button("Remove");  
        panel.getChildren().add(listview,txt1,txt2,ba,br);  
        ba.setOnAction(e -> listview.getItems().add(txt2.getText()));  
        br.setOnAction(e -> listview.getItems().remove(txt2.getText()));  
        Scene scene = new Scene(panel);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

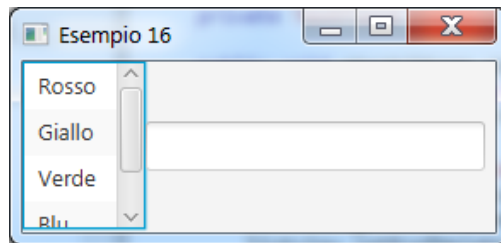
OCCHIO: `List.add` non controlla la presenza di eventuali duplicati!

Se elemento non presente non fa nulla (tipico comportamento di `List.remove`)

IL COMPONENTE **ChoiceBox**

Una **ChoiceBox** è molto simile a **ListView**

- semplicemente, gli elementi sono mostrati *in una lista a discesa pop-up*, che si apre solo cliccandoci sopra, invece che in una lista fissa
- la gestione eventi e le proprietà sono identiche



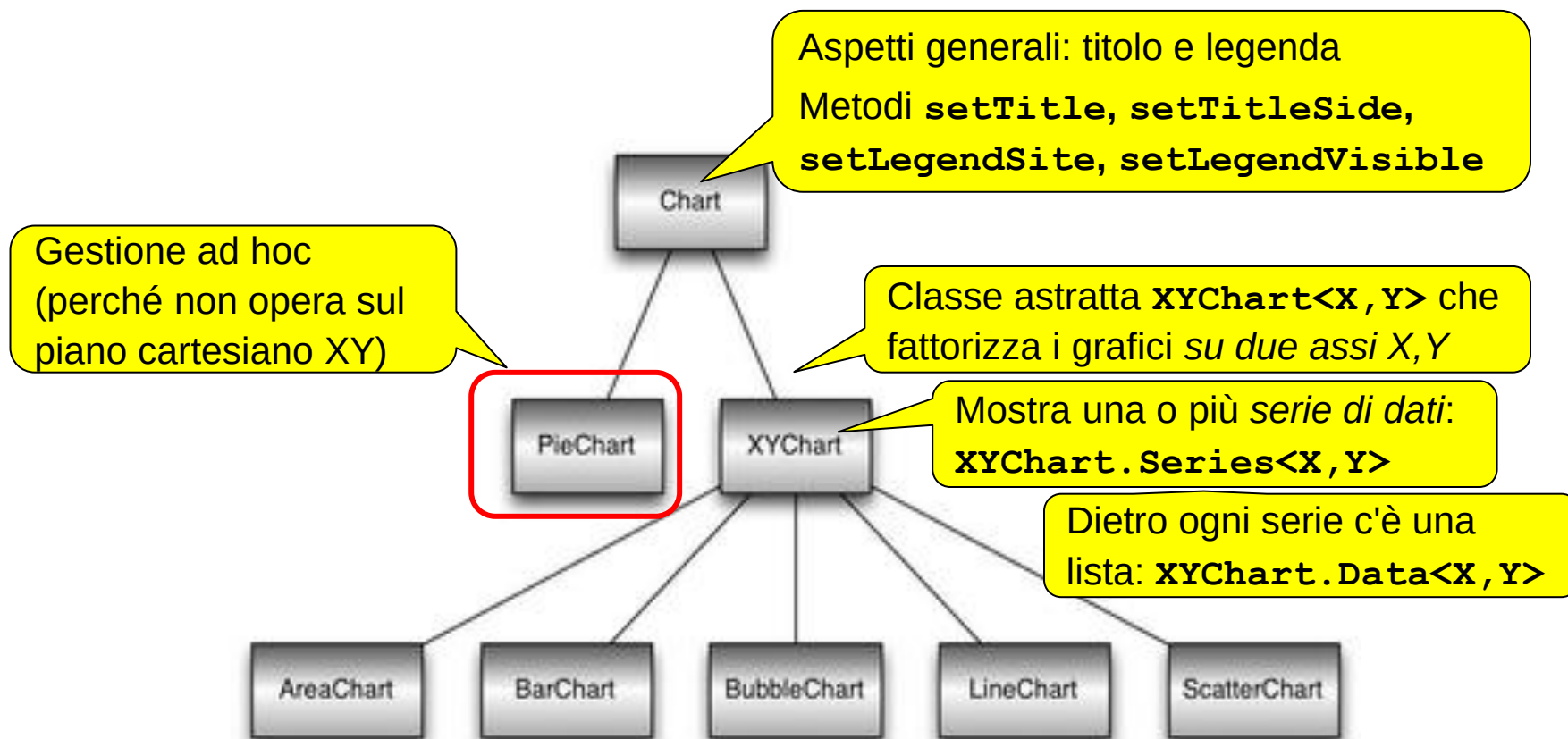
Esercizio lasciato al lettore:

nell'esempio precedente, mettere un **ChoiceBox** al posto di **ListView**

Grafici in JavaFX: the Charts family

THE Chart FAMILY

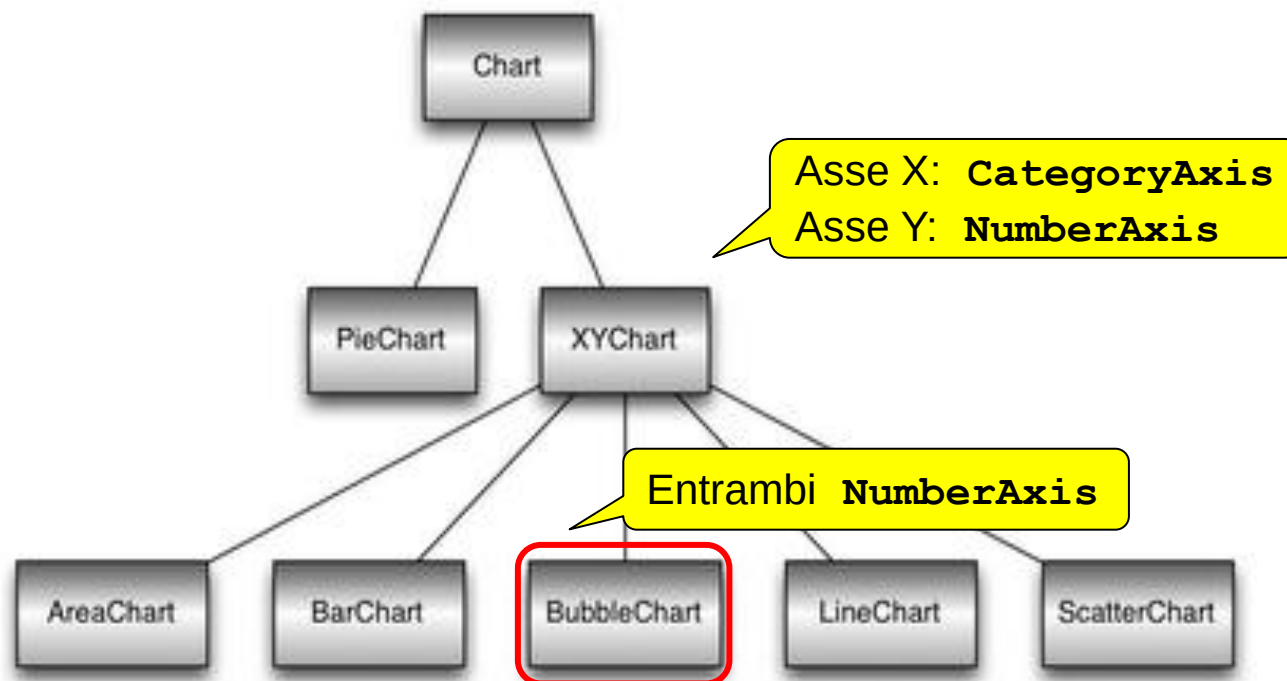
JavaFX offre un ampio ventaglio di grafici 2D



THE Chart FAMILY

Nei grafici XY, i due assi rappresentano

- l'asse orizzontale: categorie (numeri solo nel caso di **BubbleChart**)
- l'asse verticale: numeri





SCHEMA GENERALE

Passi da compiere

1. predisporre gli assi
2. creare l'oggetto Chart con quegli assi e fissarne il titolo
3. predisporre le serie di dati (una o più)
4. popolare ciascuna serie caricando le sue coppie (x,y) di dati
5. aggiungere al grafico le serie

Esempio: vendite di frutta

- asse orizzontale: categorie (tipo di frutta: es. mele, pere, arance)
- asse verticale: valori (vendite di quel tipo di frutta, es. 25, 50, 23)
- una serie per ogni città (es. Modena, Bologna, Imola, Ferrara)
- ogni serie contiene i valori di frutta venduta in quella città (es. modena → quantità di mele, pere, arance vendute a Modena)

IL COMPONENTE BarChart

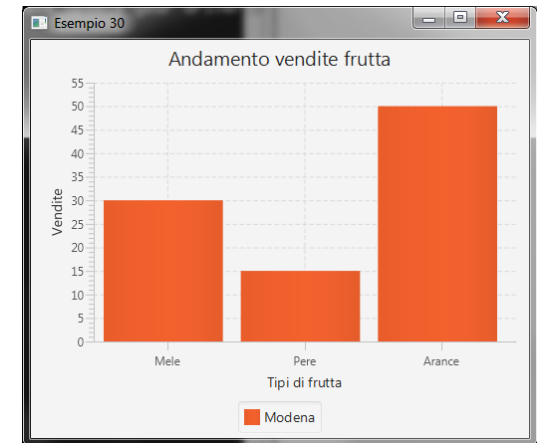
```
public class EsJavaFX30 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 30");  
        FlowPane panel = new FlowPane();  
  
        CategoryAxis asseOrizz = new CategoryAxis();  
        asseOrizz.setLabel("Tipi di frutta");  
        NumberAxis asseVert = new NumberAxis();  
        asseVert.setLabel("Vendite");  
  
        BarChart<String,Number> chart = new BarChart<>(asseOrizz,asseVert);  
        chart.setTitle("Andamento vendite frutta");  
  
        XYChart.Series<String,Number> modena = new XYChart.Series<>();  
        modena.setName("Modena");  
        modena.getData().add( new XYChart.Data<>("Mele", 30));  
        modena.getData().add( new XYChart.Data<>("Pere", 15));  
        modena.getData().add( new XYChart.Data<>("Arance", 50));  
        chart.getData().add(modena);  
        panel.getChildren().add(chart);  
        stage.setScene(new Scene(panel));  
    }  
}
```

Tipizzato

Una serie

Popolamento serie
coi dati di vendita

Aggiunta serie (popolata) al grafico



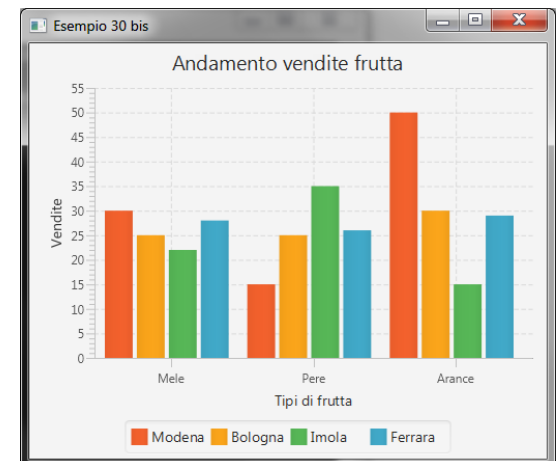
IL COMPONENTE BarChart

Variante con più serie

```
public class EsJavaFX30bis extends Application {  
    public void start(Stage stage){  
        ...  
  
        XYChart.Series<String,Number> modena = new XYChart.Series<>();  
        XYChart.Series<String,Number> bologna = new XYChart.Series<>();  
        XYChart.Series<String,Number> imola = new XYChart.Series<>();  
        XYChart.Series<String,Number> ferrara = new XYChart.Series<>();  
  
        ... // popolamento di tutte le quattro serie  
        chart.getData().add(modena);  
        chart.getData().add(bologna);  
        chart.getData().add(imola);  
        chart.getData().add(ferrara);  
  
        panel.getChildren().add(chart);  
        stage.setScene(new Scene(panel));  
    }  
}
```

Quattro serie

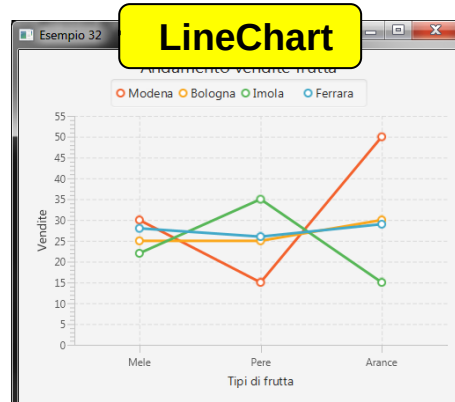
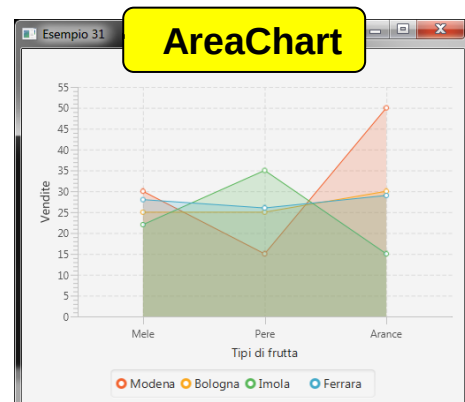
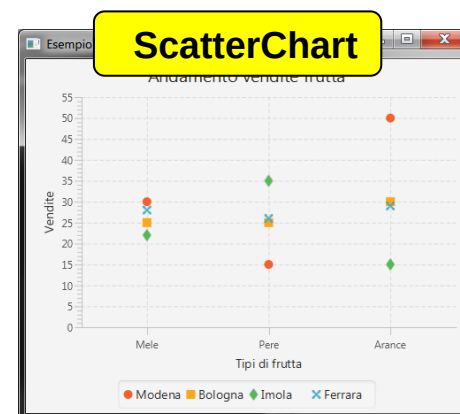
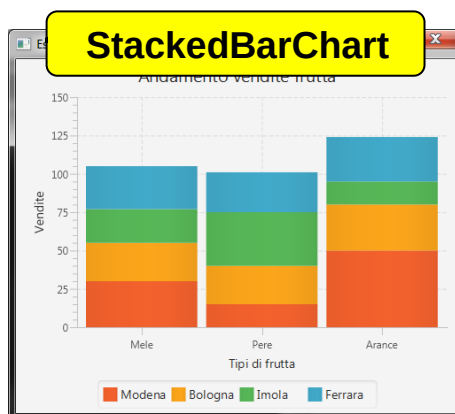
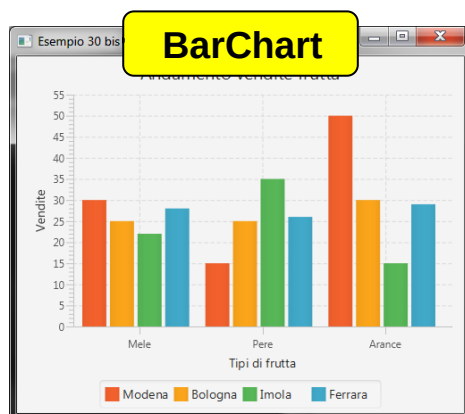
Aggiunta delle
quattro serie al
grafico



ALTRI TIPI DI GRAFICI

Per cambiare tipo di grafico basta creare un diverso chart

```
BarChart<String,Number> chart = new BarChart<>(asseOrizz,asseVert);
```





IL COMPONENTE BubbleChart

Questo chart richiede assi *numerici* (esempio: funzioni)

```
public class EsJavaFX34 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 34");  
        FlowPane panel = new FlowPane();  
  
        NumberAxis asseX = new NumberAxis(); asseX.setLabel("x");  
        NumberAxis asseY = new NumberAxis(); asseY.setLabel("y");  
  
        BubbleBarChart<Number,Number> chart = new BubbleBarChart<>(asseX,asseY);  
        chart.setTitle("Funzioni varie");  
  
        XYChart.Series<Number,Number> parabola = new XYChart.Series<>();  
        parabola.setName("parabola");  
        parabola.getData().add( new XYChart.Data<>(0,0));  
        parabola.getData().add( new XYChart.Data<>(10,10));  
        parabola.getData().add( new XYChart.Data<>(15,22));  
        parabola.getData().add( new XYChart.Data<>(20,40));  
        parabola.getData().add( new XYChart.Data<>(30,90));  
  
        ...  
    }  
}
```

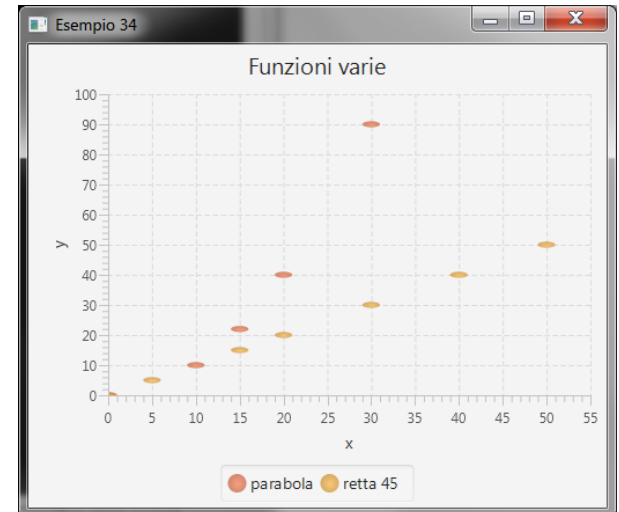
IL COMPONENTE BubbleChart

...

```
retta45.setName("retta 45");  
retta45.getData().add( new XYChart.Data<>(0,0) );  
retta45.getData().add( new XYChart.Data<>(5,5) );  
retta45.getData().add( new XYChart.Data<>(15,15) );  
retta45.getData().add( new XYChart.Data<>(20,20) );  
retta45.getData().add( new XYChart.Data<>(30,30) );  
retta45.getData().add( new XYChart.Data<>(40,40) );  
retta45.getData().add( new XYChart.Data<>(50,50) );
```

```
chart.getData().add(parabola);  
chart.getData().add(retta45);  
panel.getChildren().add(chart);  
stage.setScene(new Scene(panel));
```

```
}
```





PieChart: SCHEMA GENERALE

Passi da compiere

1. non ci sono assi: è un grafico a torta!
2. **prima** si predispone la **serie di dati** (**una sola**)
3. **poi** si crea l'oggetto PieChart con quella serie, e se ne fissa il titolo

Esempio: le solite vendite di frutta

- una sola serie = una sola città (es. Modena)
- la serie contiene i valori di frutta venduta a Modena

IL COMPONENTE PieChart

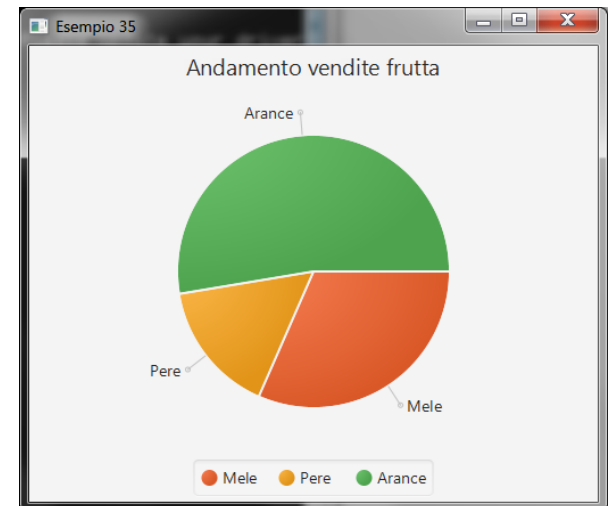
Questo chart richiede assi *numerici* (esempio: funzioni)

```
public class EsJavaFX35 extends Application {  
    public void start(Stage stage){  
        stage.setTitle("Esempio 35");  
        FlowPane panel = new FlowPane();  
        ObservableList<PieChart.Data> dati =  
            FXCollections.observableArrayList(  
                new PieChart.Data("Mele", 30),  
                new PieChart.Data("Pere", 15),  
                new PieChart.Data("Arance", 50)  
            );  
        PieChart chart = new PieChart(dati);  
        chart.setTitle("Andamento vendite frutta");  
        panel.getChildren().add(chart);  
        stage.setScene(new Scene(panel));  
    }  
}
```

Pre-popolamento serie sotto forma di
ObservableList di PieChart.Data

NON tipizzato

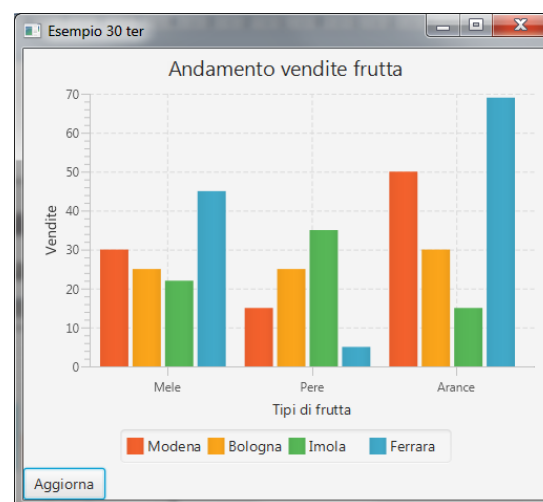
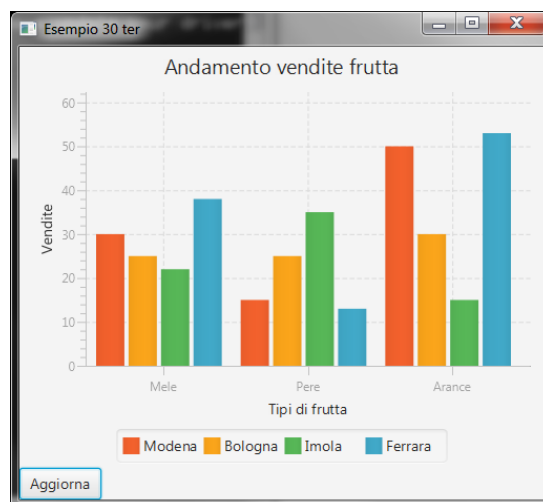
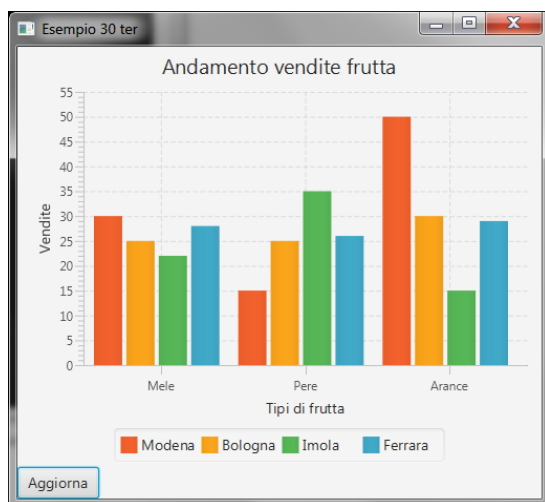
Una serie



GRAFICI ANIMATI

Le serie dati possono essere modificate e, volendo, le modifiche possono essere *animate*.

Per vederlo, aggiungiamo un bottone che cambia i dati di Ferrara:



```
chart.setAnimated(true);
```

```
Button button = new Button("Aggiorna");
```

```
button.setOnAction(event -> {
```

```
    ferrara.getData().get(0).setYValue(45); // era 28
```

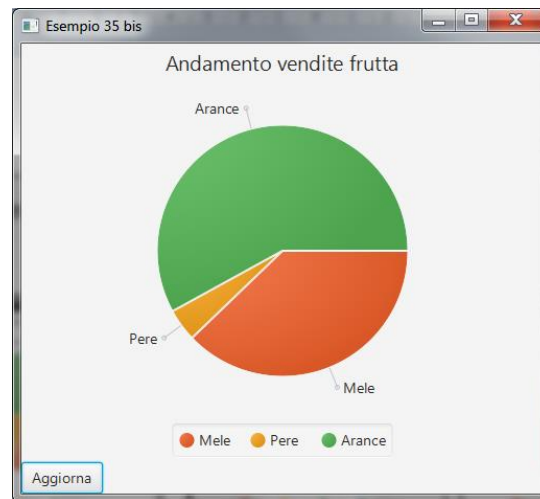
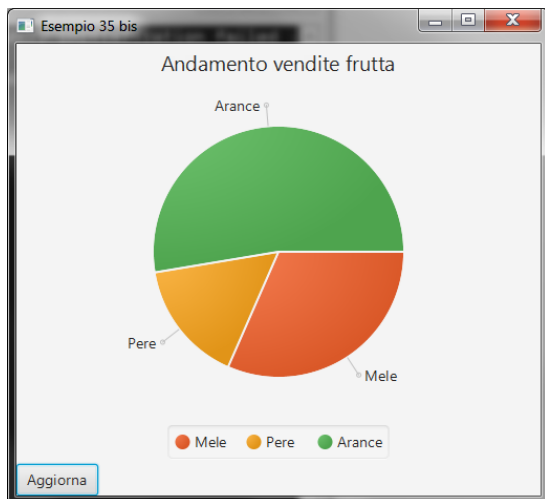
```
    ferrara.getData().get(1).setYValue(5); // era 26
```

```
    ferrara.getData().get(2).setYValue(69); // era 29
```

```
});
```

GRAFICI ANIMATI

Analogamente si può fare col grafico a torta:



```
chart.setAnimated(true);  
Button button = new Button("Aggiorna");  
button.setOnAction(event -> {  
    dati.get(0).setPieValue(45); // era 30  
    dati.get(1).setPieValue(5);  // era 15  
    dati.get(2).setPieValue(69); // era 50  
});
```

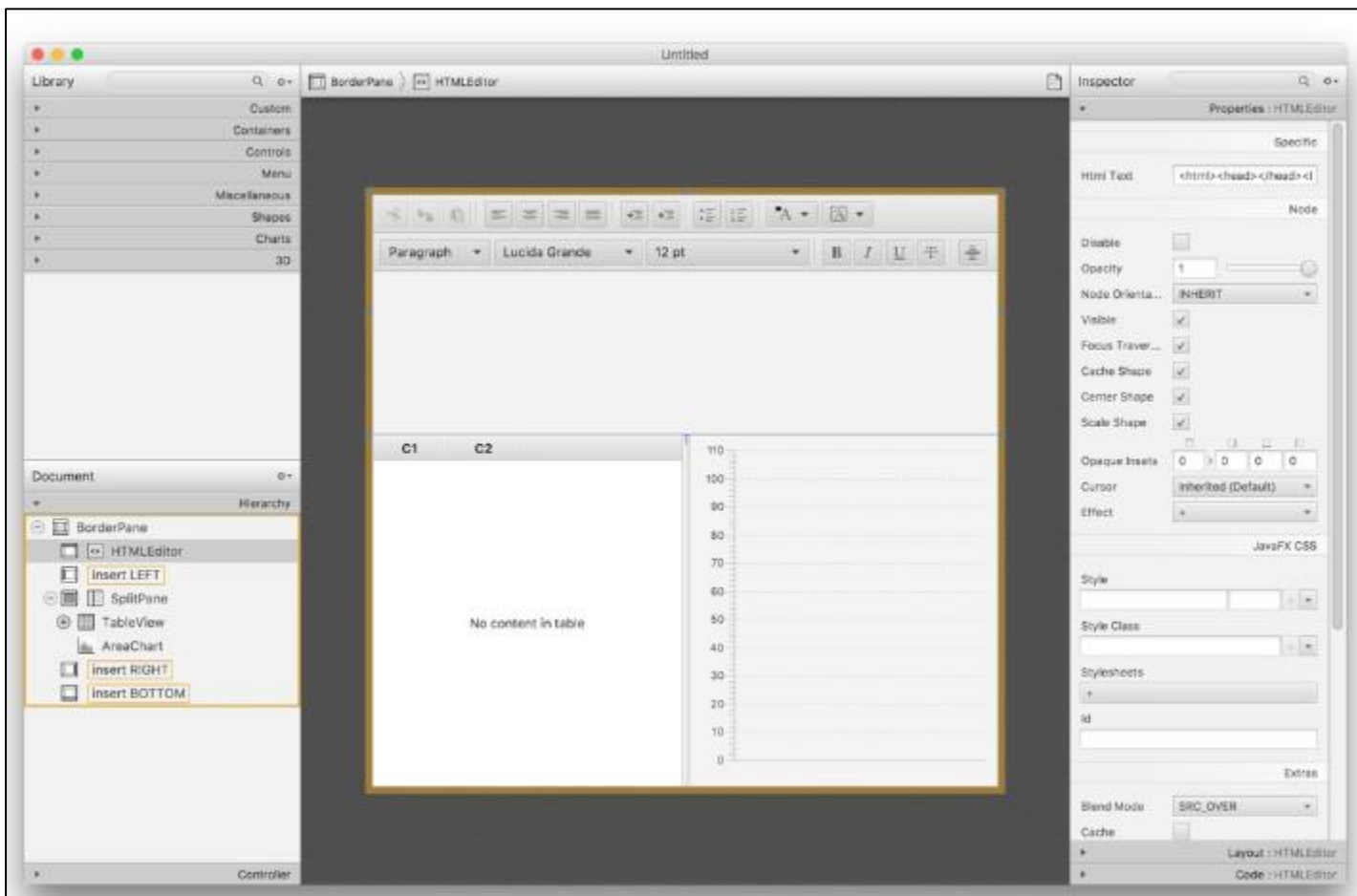
Strumenti per il progetto e la definizione di GUI



OLTRE IL DISEGNO MANUALE

- In realtà, nessuno disegna a codifica GUI "a mano"
 - i componenti non appaiono come/dove si vorrebbe
 - *necessità di iterare più volte lo sviluppo*
- Si utilizzano appositi **strumenti visuali**
 - si disegna la GUI e si vede subito "come viene"
 - editing diretto delle proprietà grafiche
 - GUI *descritta tramite file (F)XML anziché cablata nel codice*
- **JavaFX è supportato dallo strumento SceneBuilder**
 - <https://gluonhq.com/products/scene-builder/>
 - documentazione: <https://docs.gluonhq.com/>
 - usabile da Eclipse, con apposito plugin da installare a parte
 - https://docs.gluonhq.com/#_the_gluon_plugin_for_eclipse

SCENE BUILDER





GUI DESCRITTE VIA FXML

- Questi strumenti però adottano un NUOVO APPROCCIO
 - la *parte strutturale* della GUI non è più "programmata" in Java:
è descritta in un particolare formato testo, chiamato *FXML*
 - il codice Java viene usato solo per la *parte comportamentale*
 - *massima separazione* fra *struttura* e *comportamento*
 - massimo riuso su più piattaforme (Java SE, Android...)
- Architettura dell'applicazione
 - il file FXML *descrive la GUI* in termini di widget → *SceneBuilder*
 - il file FXML *specifica la classe* che funge da *controller*
 - il *controller Java* contiene la *business logic* (gestione eventi)
 - il *main Java* carica la GUI e inizializza il controller.

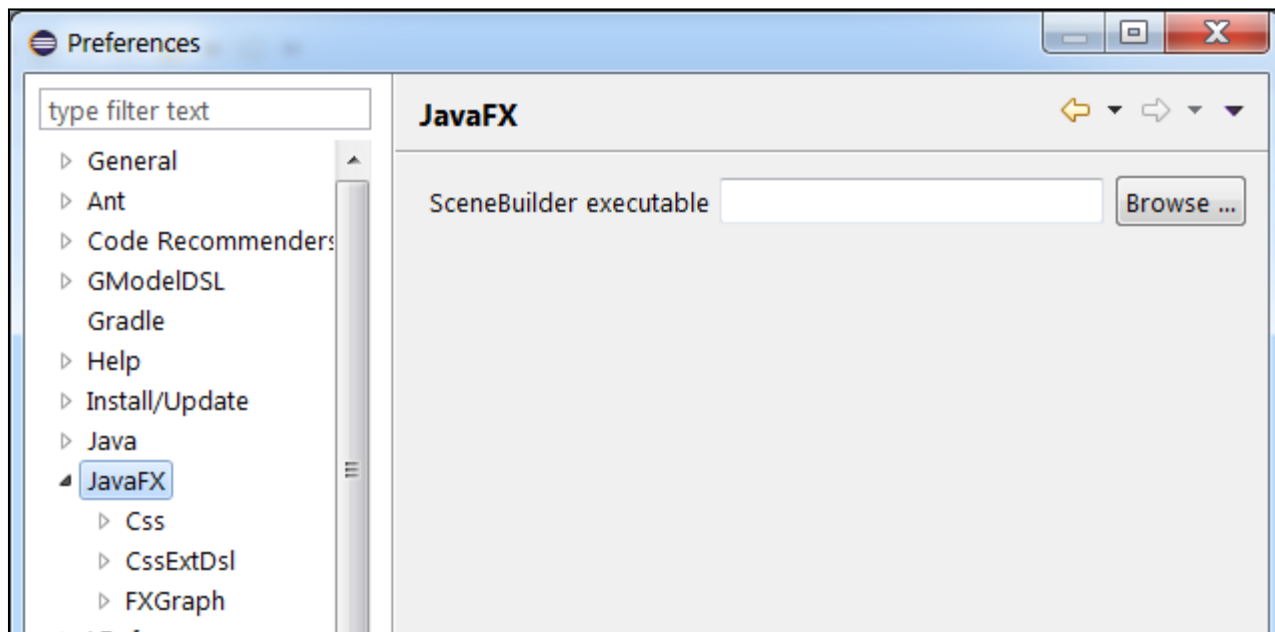


INSTALLAZIONE di SCENE BUILDER

- Scene Builder non è incluso in Eclipse di default
 - va scaricato da Gluon e installato per conto proprio
 - poi, va installato il supporto JavaFX in Eclipse
- Scaricare da:
 - <https://gluonhq.com/products/scene-builder/>
 - documentazione: <https://docs.gluonhq.com/>
- Integrazione con Eclipse
 - Installare prima il plugin e(fx)clipse di cui si è già parlato e seguire le istruzioni alle prossime slide
 - In alternativa, c'è l'apposito plugin Gluon sull'Eclipse Marketplace (occhio alla licenza!)
https://docs.gluonhq.com/#_the_gluon_plugin_for_eclipse

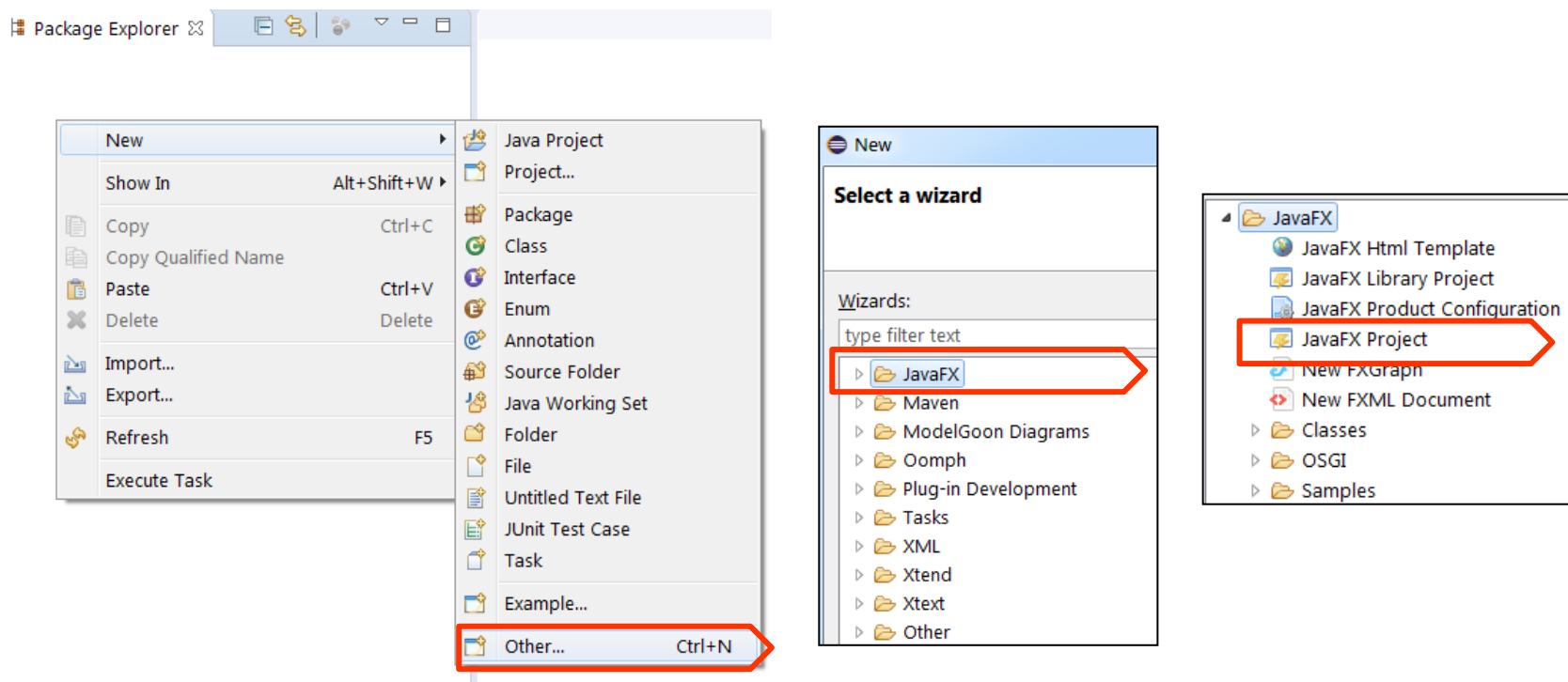
CONFIGURAZIONE di SCENE BUILDER

- Prima di essere usato, Scene Builder va configurato
 - *Eclipse >> Window >> Preferences >> JavaFX*
 - sfogliare alla ricerca dell'eseguibile
`C:\Users\Enrico\AppData\Local\SceneBuilder`



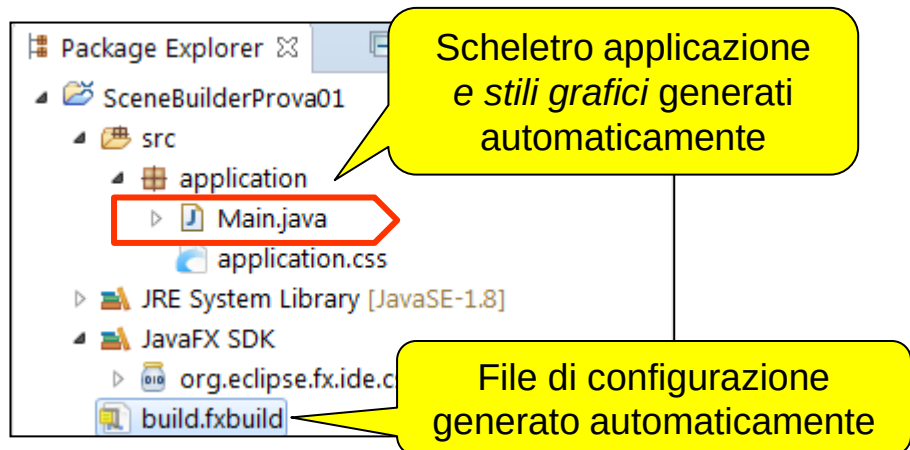
SCENE BUILDER (1)

1. New > Java Project
2. New > Other > JavaFX > Application
3. scegliere il nome del package, quello della classe e poi.. via!



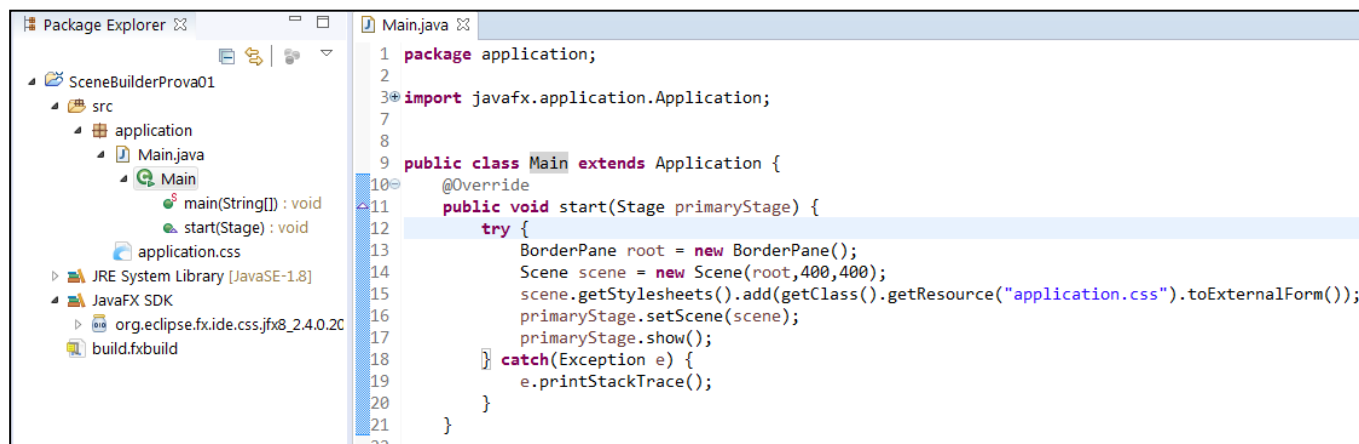
SCENE BUILDER (2)

Viene generata l'applicazione, con relativo main (e file di configurazione)



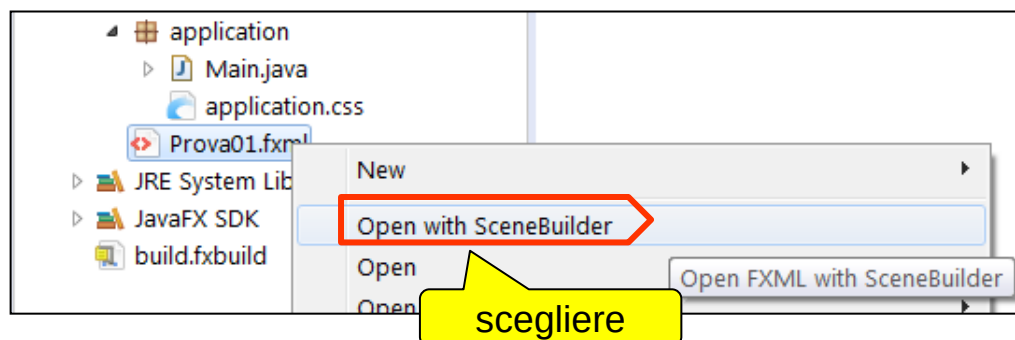
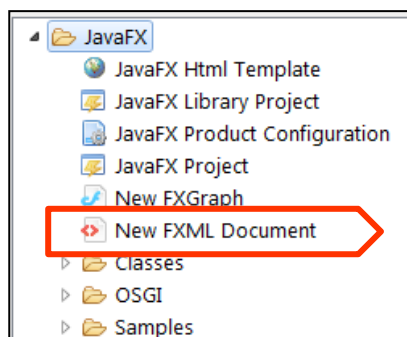
Possiamo sbirciare il main di base, ma è solo un modello standard **che dovremo modificare.**

Il bello deve ancora venire.



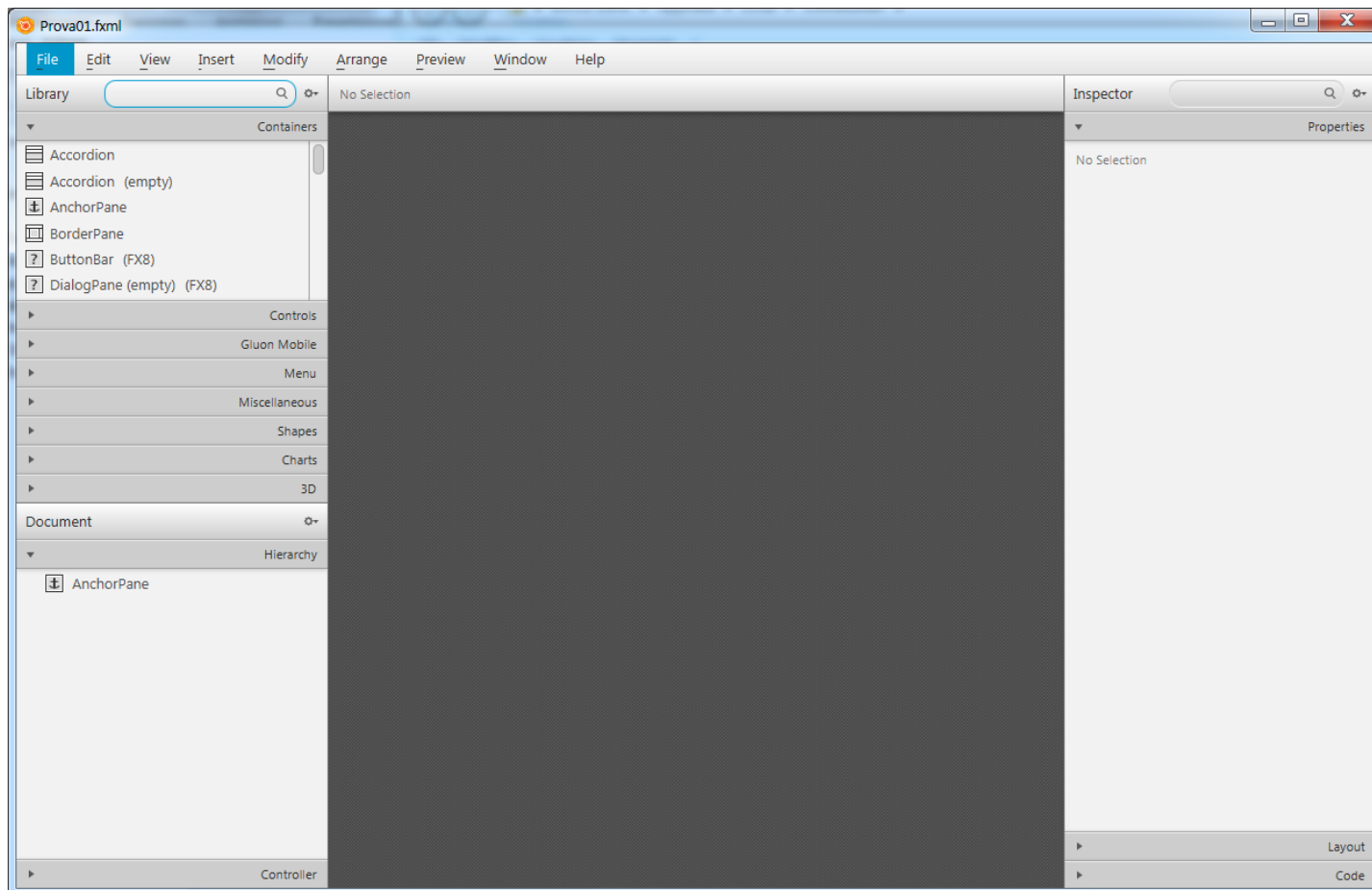
SCENE BUILDER (3)

Per usare lo strumento visuale, dobbiamo prima creare il file FXML destinato a contenere la descrizione della GUI. Fatto ciò...



SCENE BUILDER (4)

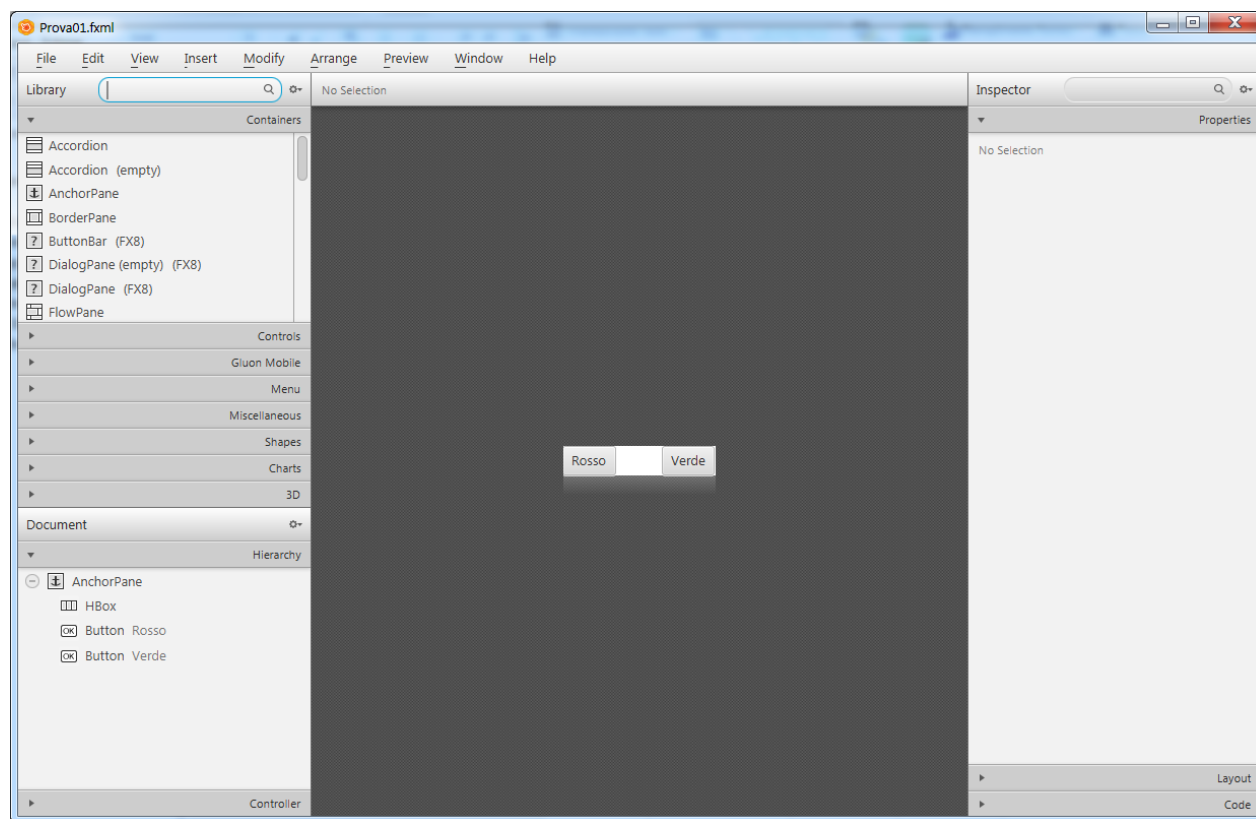
Ora, cliccando col tasto destro sul file FXML, si apre Scene Builder





UN'APP CON SCENE BUILDER (1)

Applicazione Rosso/Verde: costruiamo un **HBox** con due pulsanti e una **Region** (trasparente e vuota) fra loro, per tenerli distanziati:





UN'APP CON SCENE BUILDER (2)

Riaprendolo in Eclipse, vediamo la *descrizione della GUI in FXML*

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.scene.control.Button?>
4 <?import javafx.scene.layout.AnchorPane?>
5 <?import javafx.scene.layout.HBox?>
6 <?import javafx.scene.layout.Region?>
7
8 <AnchorPane xmlns="http://javafx.com/javafx/8.0.111" xmlns:fx="http://javafx.com/fxml/1">
9   <children>
10    <HBox minWidth="154.0" prefHeight="29.0" prefWidth="154.0" style="-fx-background-color: white;">
11      <Button layoutX="-1.0" mnemonicParsing="false" text="Rosso" />
12      <Region opacity="0.0" prefHeight="22.0" prefWidth="50.0" />
13      <Button layoutX="97.0" mnemonicParsing="false" text="Verde" />
14    </HBox>
15  </children>
16 </AnchorPane>
```

Per poter agganciare alla GUI la *business logic* (gestione eventi), occorre:

- dare un **nome simbolico** ai **singoli componenti**
(in modo da poterli poi referenziare nel codice dell'ascoltatore)
- indicare il **nome del metodo dell'ascoltatore** che gestirà l'evento

UN'APP CON SCENE BUILDER (3)

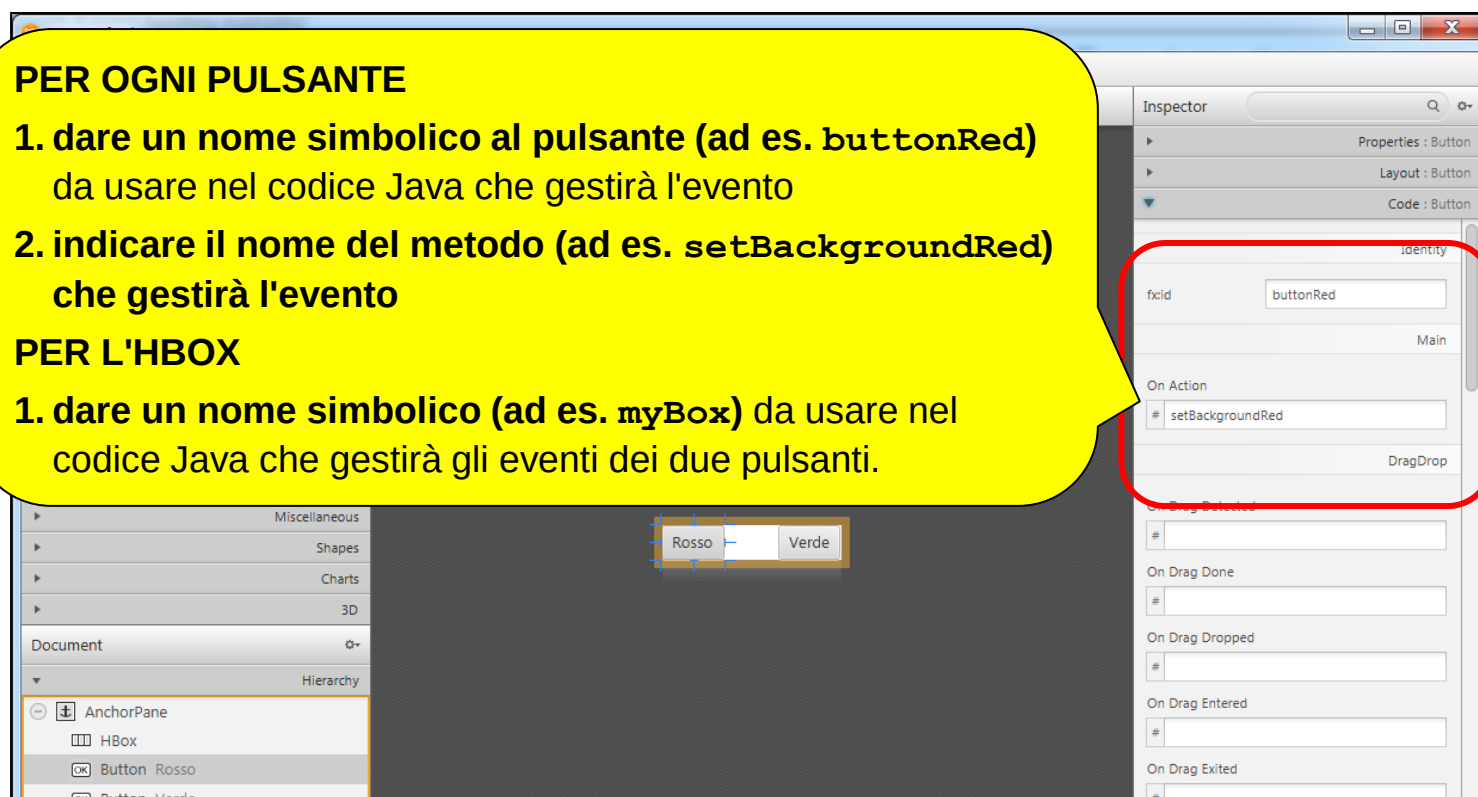
Per aggiungere i nomi simbolici, fare clic via via sui vari elementi e per ciascuno scegliere (nel menù a destra) la **scheda Code**:

PER OGNI PULSANTE

1. dare un nome simbolico al pulsante (ad es. `buttonRed`) da usare nel codice Java che gestirà l'evento
2. indicare il nome del metodo (ad es. `setBackgroundRed`) che gestirà l'evento

PER L'HBOX

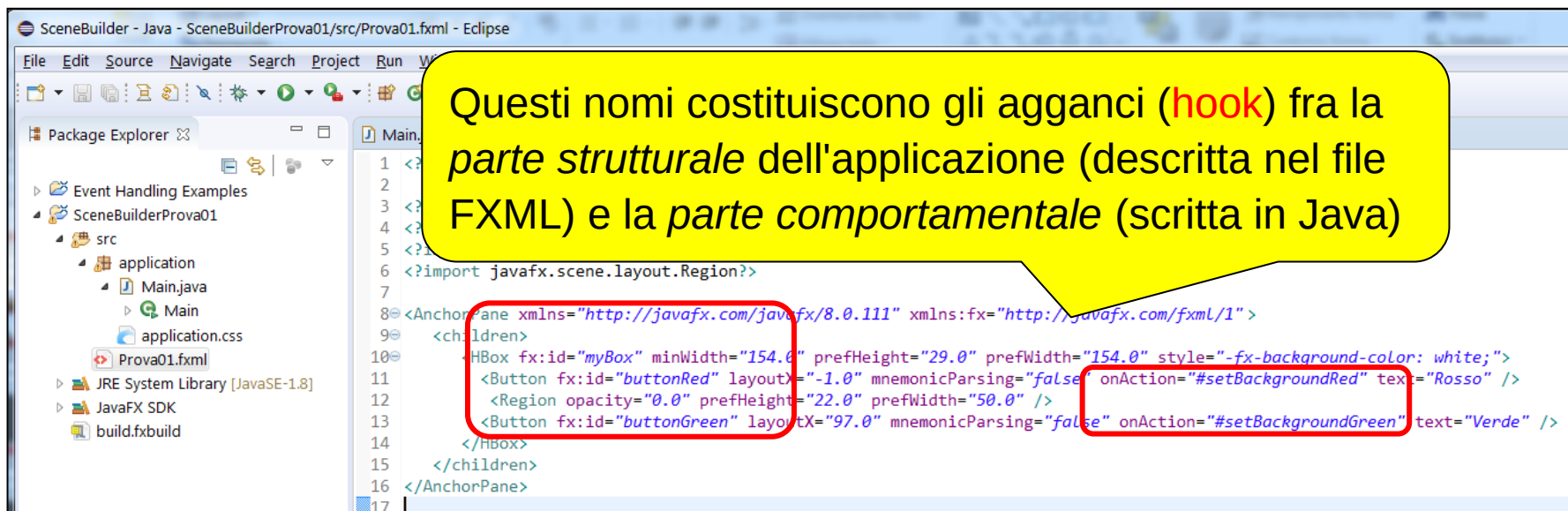
1. dare un nome simbolico (ad es. `myBox`) da usare nel codice Java che gestirà gli eventi dei due pulsanti.



UN'APP CON SCENE BUILDER (4)

Le modifiche fatte in Scene Builder si riflettono nel file FXML:

- i nomi inseriti appaiono come *attributi*, rispettivamente nella forma **fx:id="nome"** per i nomi pulsanti e **onAction="nome"** per i metodi.
- in alternativa, tali nomi si possono aggiungere direttamente come testo del file FXML, senza far uso di Scene Builder.





UN'APP CON SCENE BUILDER (5)

La business logic dev'essere contenuta in una classe *controller*

- la GUI deve sapere chi sia il suo controller: lo specifica l'attributo **fx:controller="nome"** dell'elemento di top-level (**AnchorPane**)
- naturalmente, anche in questo caso l'attributo può essere impostato sia tramite Scene Builder sia direttamente nel testo FXML.

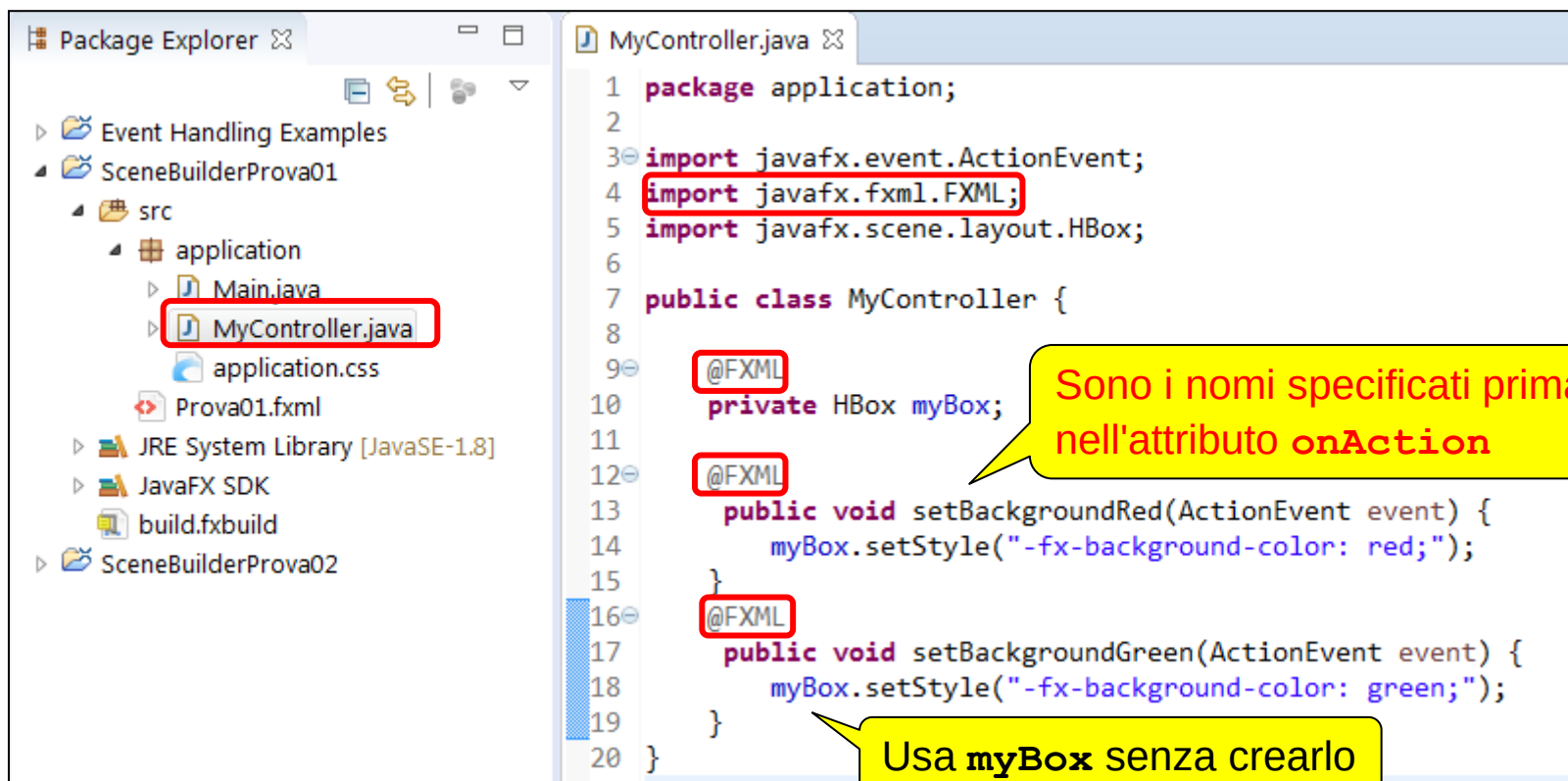
```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.scene.control.Button?>
4 <?import javafx.scene.layout.AnchorPane?>
5 <?import javafx.scene.layout.HBox?>
6 <?import javafx.scene.layout.Region?>
7
8 <AnchorPane xmlns="http://javafx.com/javafx/8.0.111" xmlns:fx="http://javafx.com/fxml/1" fx:controller="application.MyController">
9     <children>
10         <HBox fx:id="myBox" minWidth="154.0" prefHeight="29.0" prefWidth="154.0" style="-fx-background-color: white;">
11             <Button fx:id="buttonRed" layoutX="-1.0" mnemonicParsing="false" onAction="#setBackgroundRed" text="Rosso" />
12             <Region opacity="0.0" prefHeight="22.0" prefWidth="50.0" />
13             <Button fx:id="buttonGreen" layoutX="97.0" mnemonicParsing="false" onAction="#setBackgroundGreen" text="Verde" />
14         </HBox>
15     </children>
16 </AnchorPane>
17
```

Ora dobbiamo implementare in Java questo controller.

UN'APP CON SCENE BUILDER (6)

Il controller è una comune classe di nostra scelta

- apposite **annotazioni** della forma **@FXML** identificano i widget che il **controller dichiara ma non costruisce**, in quanto specificati via FXML



```
1 package application;
2
3 import javafx.event.ActionEvent;
4 import javafx.fxml.FXML;
5 import javafx.scene.layout.HBox;
6
7 public class MyController {
8
9     @FXML
10     private HBox myBox;
11
12     @FXML
13     public void setBackgroundRed(ActionEvent event) {
14         myBox.setStyle("-fx-background-color: red;");
15     }
16
17     @FXML
18     public void setBackgroundGreen(ActionEvent event) {
19         myBox.setStyle("-fx-background-color: green;");
20     }
21 }
```

Sono i nomi specificati prima nell'attributo **onAction**

Usa **myBox** senza crearlo

UN'APP CON SCENE BUILDER (7)

Per finire, resta da sistemare il **Main**, che deve:

- caricare il file **FXML** (occhio al path!!)
- recuperare il nodo radice (AnchorPane) e impostarlo sulla scena

```
public void start(Stage primaryStage) {  
    try {  
        FXMLLoader loader = new FXMLLoader(Main.class.getResource("/Prova01.fxml"));  
        AnchorPane page = (AnchorPane) loader.load();  
        Scene scene = new Scene(page);  
        scene.getStylesheets().add(getClass().getResource("application.css").toExternalForm());  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

And the winner is... 😊



UN'APP CON SCENE BUILDER (8)

In alternativa, si può non specificare nella GUI il gestore dell'evento, lasciando al metodo **initialize** del controller (chiamato automaticamente da JavaFX) il compito di collegarli:

```
<AnchorPane xmlns="http://javafx.com/javafx/8.0.111" xmlns:fx="http://javafx.com/fxml/1" fx:controller="application.MyController">
  <children>
    <HBox fx:id="myBox" minWidth="154.0" prefHeight="29.0" prefWidth="154.0" style="-fx-background-color: white;">
      <Button fx:id="buttonRed" layoutX="-1.0" mnemonicParsing="false" text="Rosso" />
      <Region opacity="0.0" prefHeight="22.0" prefWidth="50.0" />
      <Button fx:id="buttonGreen" layoutX="97.0" mnemonicParsing="false" text="Verde" />
    </HBox>
  </children>
</AnchorPane>
```

Tolti gli attributi **onAction**

```
MyController.java
1 package application;
2
3 import javafx.fxml.FXML;
4
5 public class MyController {
6
7     @FXML
8     private Button buttonRed, buttonGreen;
9
10    @FXML
11    private HBox myBox;
12
13    /**
14     * Initializes the controller class. This method is automatically called
15     * after the fxml file has been loaded.
16     */
17    @FXML
18    private void initialize() {
19        buttonRed.setOnAction( event -> myBox.setStyle("-fx-background: red;") );
20        buttonGreen.setOnAction( event -> myBox.setStyle("-fx-background: green;") );
21    }
22
23 }
24
25
26
```

Classiche lambda expression

UN'APP CON SCENE BUILDER (9)

Giochino finale:

cambiamo il colore dei pulsanti ogni volta che vengono premuti

```
@FXML
private void initialize() {
    buttonRed.setOnAction( event -> {
        myBox.setStyle("-fx-background-color: red;");
        buttonRed.setStyle("-fx-background-color: " + changeColorTo());
    });
    buttonGreen.setOnAction( event -> {
        myBox.setStyle("-fx-background-color: green;");
        buttonGreen.setStyle("-fx-background-color: " + changeColorTo());
    });
}

private String changeColorTo(){
    int r,g,b;
    java.util.Random gen = new java.util.Random();
    r = gen.nextInt(256); g = gen.nextInt(256); b = gen.nextInt(256);
    String hexcolor = String.format( "%02X%02X%02X", r,g,b);
    System.out.println(hexcolor);
    return hexcolor;
}
```

Sintetizza una stringa-colore
RGB in formato esadecimale

